

게임 개발 현장 백서

조용히 틀리는 것들

다섯 직무가 9주 동안 밟은 지뢰와, 그 뒤에 세운 원칙 180개

게임을 만들기 시작한 사람, 그리고 이제 막 현업에 들어서는 사람에게

2026년 10월

머리말

이 책은 무엇인가

작은 팀이 9주 동안 웹·모바일용 턴제 전술 게임 하나를 만들었다. 캐릭터는 체스 기물처럼 정해진 규칙으로 움직이고, 행동 순서는 각자의 대기 시간이 다 차는 순서로 정해진다. 온라인 대전은 서버가 판정하고, 캐릭터 뽑기(가차)와 육성, 도시를 키우는 메타 게임이 그 위에 얹힌다. 화면은 열 개 언어로 나간다.

개발하는 동안 매일 기록을 남겼다. 9주 뒤 그 기록에서 **다른 게임을 만들 때도 통하는 교훈 353개**를 뽑았고, 게임 개발의 다섯 직무로 나눠 **원칙 180개**로 다시 묶었다. 이 책은 그 결과다. 우리 게임을 몰라도 읽을 수 있도록, 사례는 어느 게임에서나 생길 수 있는 장면으로 다시 썼다.

왜 「조용히 틀리는 것들」인가

가장 비싼 실수는 요란하게 깨지는 버그가 아니었다. **화면은 멀쩡해 보이고, 테스트는 통과하고, 오류 메시지도 없는데 실제로는 틀려 있는 것이었다.** 아무것도 확인하지 않던 테스트, 실제 데이터와 다르게 남아 버린 기획 문서, 번역이 빠지면 말없이 원문으로 물러나는 화면, 데이터에는 적혀 있는데 엔진이 읽지 않아 3주 동안 효과가 없던 장비. 이런 실수는 대개 발견이 늦고, 발견했을 때는 이미 여러 곳에 퍼져 있다.

이 책의 거의 모든 원칙은 하나의 질문으로 모인다 — 「**이것이 틀렸다면, 누가 언제 알아챌까?**」

누구를 위한 책인가

- 게임 업계에 들어오려는 사람 — 현장에서 실제로 무엇이 어긋나는지 미리 보고 싶은 사람.
- 다른 게임을 만들고 있는 주니어·미들급 개발자 — 자기 직무의 함정과, 옆 직무와 맞닿는 경계를 점검하고 싶은 사람.
- 작은 팀의 리드 — 사람이 적어 한 사람이 여러 역할을 겸할 때, 무엇을 놓치기 쉬운지 알고 싶은 사람.

읽는 법

- **1장 「모든 직무 공통」**을 먼저 읽는다. 검증과 일하는 방식에 관한 원칙이다.
- 그다음 **자기 직무의 장**을 읽는다. 장 첫머리의 「이 장의 원칙」 목록만 훑어도 요지가 잡힌다.
- 마지막으로 **이웃 직무의 장**을 읽는다. 사고는 대개 두 직무의 경계에서 났다 — 기획이 넘긴 수치를 엔지니어가 코드에 다시 적을 때, 엔진의 판정을 화면이 따로 계산할 때, 디자인의 치수가 코드의 값과 따로 놀 때.
- 원칙마다 실제로 겪은 **사례**가 붙어 있다. 무슨 일이 있었고, 왜 아무도 몰랐고, 어떻게 잡았는지의 순서다. 장 끝의 **점검표**는 작업 후후에 스스로 묻는 질문이다.

다섯 직무

직무	말는 것	장
게임 디자이너	규칙과 콘텐츠 기획, 밸런스와 경제, 화면 기획, 시나리오와 번역 관리	2장
시스템 엔지니어	게임 규칙 엔진, 성장·경제 로직, 기획 데이터를 코드로 옮기는 파이프라인, 시뮬레이션 도구	3장
서버 엔지니어	실시간 대전 서버, 계정과 데이터베이스, 서버 판정과 치팅 방지, 인프라	4장
클라이언트 엔지니어	화면 기능과 상태, 전투 연출 코드, 사운드 연결, 다국어 처리	5장
디자인 엔지니어	화풍과 UI 비주얼, CSS·레이아웃, 그림 리소스를 게임용으로 굽는 파이프라인	6장

작은 팀에서는 한 사람이 둘 이상의 직무를 겸한다. 장은 사람이 아니라 일의 종류로 나뉘었다.

차례

1장 모든 직무 공통	6
1.1 검사를 믿기 전에 실패를 본다	7
1.2 검사가 그 상태에 실제로 도달하는가	9
1.3 검증 층을 나누고, 자동 검사를 살려 둔다	11
1.4 실패를 만났을 때 — 원인을 가른다	13
1.5 확인하지 못한 것을 적는다	15
1.6 작업을 나누고, 정본을 하나로 둔다	16
1.7 저장소와 도구를 다룰 때	18
2장 게임 디자이너	21
2.1 기획 결정을 주고받는 법	22
2.2 숫자와 이름은 한 곳에서만 정한다	24
2.3 중간 상태와 예외를 만들지 않는 규칙	26
2.4 숫자는 혼자 틀리지 않는다 — 시계·관계·경계	28
2.5 우회로가 정식 통로보다 싸지 않게 — 보상·패널티·치팅	30
2.6 밸런스와 경제는 시뮬레이션으로 끝까지 민다	32
2.7 화면은 규칙을 거짓 없이 말해야 한다	34
3장 시스템 엔지니어	38
3.1 순수성과 시간	39
3.2 판정의 단일 출처	41
3.3 데이터 구동 시스템 — 적힌 것이 읽히는가	44
3.4 저장 형식과 마이그레이션	45
3.5 빌드와 생성 파이프라인	48
3.6 측정과 회귀	50
4장 서버 엔지니어	54
4.1 서버 권위의 경계	55
4.2 결정적 재생으로 얻는 권위	57
4.3 동시성 · 순서 · 원자성	58
4.4 실시간 방과 전선	60
4.5 실패를 읽는 법	61
4.6 외부 서비스 · 계정 · 비밀	63
4.7 서버를 검증하는 법	65

5장 클라이언트 엔지니어	67
5.1 연출의 시간축 — 화면은 결과를 시간에 맞춰 풀어 놓는다	68
5.2 다시 그리기와 화면 상태	70
5.3 프레임워크 밖의 자원, 연결, 실행 환경	71
5.4 화면의 통로 — 엔진이 허용하는 모든 수에 손이 닿는가	74
5.5 화면이 말하는 숫자의 출처	75
5.6 다국어 — 열 언어에서 같은 말을 하게 하기	77
5.7 화면 검사 — 브라우저 스모크를 거짓말하지 않게 만들기	78
 6장 디자인 엔지니어	 82
6.1 CSS는 틀려도 말하지 않는다	83
6.2 레이아웃 — 누가 줄고 누가 늘어나는가	85
6.3 숫자는 재서 정하고, 이유와 함께 지운다	87
6.4 에셋 파이프라인 — 원본에서 화면까지	89
6.5 자르기와 배경 제거 — 이미지 처리를 재는 법	92
6.6 눈으로 확인하는 법	94

1장

모든 직무 공통

검증과 일하는 방식 — 직무와 상관없이 모두 읽는 장

이 책의 사례는 한 팀이 두 달 동안 만든 게임에서 나왔다. 체스 기물처럼 움직이는 캐릭터들이 싸우는 턴제 전술 게임이고, 행동 순서는 캐릭터마다 다른 대기 시간으로 정해진다. 서버가 판정하는 온라인 대전, 가차와 캐릭터 육성, 열 개 언어가 붙어 있다. 이 장은 직무와 상관없이 검사를 쓰고, 결과를 보고하고, 저장소와 문서를 만지는 사람 모두에게 해당한다. 현장에서 가장 자주, 가장 조용히 틀리는 것은 기능이 아니다. **기능을 지킨다고 믿었던 검사다.** 실패할 수 없게 짜인 검사, 겨냥한 상태까지 한 번도 가지 못한 검사, 앞에서 막혀 뒤가 전부 안 되는 검사가 모두 초록불을 낸다. 그다음으로 흔한 것은 「확인했다」와 「확인하지 않았다」의 경계를 적지 않아 생기는 오해, 그리고 문서와 저장소를 다루다 아무 경고 없이 잃는 작업이다. 이 장의 원칙은 장르와 엔진이 달라도 그대로 통한다.

이 장의 원칙

1. 새 검사는 한 번 실패하는 것을 본 뒤에 믿는다
2. 기대값은 기본값과 다른 값으로 잡는다
3. 결함을 심어도 결과가 안 바뀌면 입력이 둔감한 것이다 — 결과가 뒤집히는 입력을 만든다
4. 「일부러 깨 보기」가 예상과 다르면 조작이 실제로 먹었는지부터 본다
5. 목록을 도는 검사는 비어 있지 않음을 먼저 단언하고, 기대값은 화면 밖 원천에서 가져온다
6. 확률이 끼는 동작은 결과가 아니라 결정적으로 일어나는 부분으로 판정한다
7. 겨냥한 상태에 도달하지 못하는 검사는 돌지 않는다 — 흉내 낼 통로를 검사와 함께 만든다
8. 조건부 검사는 조건이 성립하는 데이터를 검사 옆에 심고, 「건너뛴」을 따로 센다
9. 흉내 낸 상태는 사람이 실제로 만들 수 있는 상태여야 한다
10. 검사가 우연한 진행 순서에 기대지 않게 한다 — 필요한 전제는 검사 안에서 만든다
11. 검증 층을 나누어 각각 둔다 — 층마다 잡는 버그가 다르다
12. 움직임과 타이밍은 눈이 아니라 시간표와 시계열로 켤다
13. 고정 대기 시간을 쓰지 않는다 — 실제 상태를 기다린다
14. 선택자는 글자·순서·모양이 아니라 뜻 속성에 걸고, 자리는 실측 좌표로 본다
15. 첫 실패에서 멈추는 검사의 막힘은 그날 풀고, 테스트 수가 줄면 실패로 본다
16. 「이 실패는 원래 있던 것인가」는 변경을 걸어 낸 기준선으로 답한다
17. 테스트가 실패하면 규칙보다 먼저 시작 상태를 의심한다
18. 이상이 보이면 코드보다 바깥부터 켤다
19. 체감값과 임계값은 먼저 숫자로 만든다 — 정상 분포와 대조군을 함께 켤다
20. 확인하지 못한 것을 목록으로 남긴다

21. 검사가 지키지 못하는 성질은 「안 잡힌다」고 사실대로 적는다
22. 자동화가 볼 수 없는 것은 담당자를 정해 사람에게 넘기고, 사람이 잡은 버그는 검사로 굳힌다
23. 큰 작업은 동작이 안 바뀌는 단계로 쪼개고, 감쌀 흐름을 먼저 완성한다
24. 같은 정보는 한 곳만 정본으로 둔다
25. 「없다」는 근거로 쓰기 전에 다시 찾는다
26. 규칙을 거스르거나 되돌리기 비싼 결정은 확인받고, 모호한 낱말은 대상을 꼭 잡는다
27. 비밀 파일은 거르지 말고 필요한 키 하나만 읽는다
28. 되돌리기 명령은 범위를 좁히고, 커밋하지 않은 분량을 먼저 본다
29. 도구로 파일을 고치면 줄 끝과 서식이 조용히 바뀐다 — 바꿀 부분만 바꾼다
30. 검사 도구는 상태를 쓰지 않는 길로 관찰한다

1.1 검사를 믿기 전에 실패를 본다

초록불은 두 가지를 뜻할 수 있다. 「잘 고쳤다」와 「검사가 아무것도 안 본다」. 둘은 결과만 봐서는 구분되지 않는다. 우리 팀에서는 같은 종류의 교훈이 두 달 동안 열 번 넘게 다시 나왔다. 그만큼 잊기 쉽다. 이 장에서 「회귀 테스트」는 한 번 고친 버그나 정한 규칙이 다시 깨지지 않게 지키는 자동 테스트를, 「스모크 테스트」는 실제 화면을 띄워 처음부터 끝까지 눌러 보는 자동 검사를 말한다.

새 검사는 한 번 실패하는 것을 본 뒤에 믿는다

검사를 넣었으면 고친 코드를 잠깐 되돌리거나 지워서 그 검사가 빨개지는지 본다. 실패하는 것을 본 적 없는 테스트는 검증된 테스트가 아니다. 깎 때는 **정확히 그 자리에서** 실패하는지도 본다. 「몇 건이 깨지는가」도 정보다. 하나만 깨져야 할 곳에서 여럿이 깨지면 검사들이 서로 기대고 있다는 뜻이다. 다 본 뒤에는 깎 흔적이 남지 않았는지(diff 0) 확인하고 되돌린다.

사례 · UI 레이아웃

무슨 일이 캐릭터 카드가 목록의 칸 밖으로 넘치는 버그를 고치고 「카드 액자 높이 > 칸 높이면 실패」 검사를 넣었다. **잡은 방법** 수정을 되돌려 248px > 185px로 걸리는 것을 확인하고서야 검사를 믿었다. 캐릭터가 칸을 건너뛰어 순간이동하는지 프레임마다 훑는 탐지기도, 수정을 되돌리자 이동할 때마다 걸리는 것을 보고 믿었다.

사례 · 검사 점검

무슨 일이 한 주 동안 넣은 새 검사마다 대상 로직을 일부러 깼다. 패배 보상으로 주는 카드의 등급을 바꾸고, 레벨 상한을 무한으로 풀고, 보관함으로 옮기는 호출을 지웠다. **결과** 각각 정확히 해당 스모크 테스트 줄에서 멈췄다. 클라이언트가 덮어쓰지 못하게 서버가 소유하는 필드 목록에서 둘을 빼자 권한 회귀 5건이 전부 깨졌다. 마지막으로 깎 흔적을 표시해 둔 문자열이 코드에 0개이고 diff도 0인 것까지 확인하고 마쳤다.

기대값은 기본값과 다른 값으로 잡는다

기대값이 기본값과 같으면 기능이 죽어도 결과가 같아서 초록불이 뜬다. 저장·적용·정산을 확인하는 검사에는 「기능이 없을 때 나오는 값」과 다른 값을 넣어야 한다. 검사가 기대는 전제(「상한이 한 번에 사는 묶음 크기보다 크다」 같은 것)도 검사 안에서 먼저 단언한다.

사례 · 저장과 불러오기

무슨 일이 플레이어가 전투 전 캐릭터 배치를 미리 저장해 두는 기능이 있다. 「저장한 배치가 실제 판에 깔리는가」 스모크 테스트가 통과했는데, 알고 보니 저장한 배치가 기본 배치 그대로였다. **왜 조용했나** 저장한 배치가 안 깔려도 엔진이 기본값으로 같은 자리에 세우므로, 무슨 일이 있어도 통과하는 검사였다. **잡은 방법** 캐릭터 하나를 한 칸 옮겨 저장하게 고쳤다. 그리고 배치 적용 코드를 지워 저장 [12,17 17,17 5,15] / 판 [12,17 17,17 7,17] 로 실패하는 것까지 봤다.

사례 · 경제 시스템

무슨 일이 시간이 지나면 차오르는 재료가 있고, 상점에서 재료를 사기 전에 먼저 그동안 찬 양을 정산해야 한다. 「구매 전에 정산한다」 검사는 창고 상한(20)과 한 번에 사는 묶음 크기(20)가 같아서, 정산 코드를 통째로 지워도 통과했다. 같은 주에 「결정타 추가 데미지」 검사는 시험에 쓴 무기에 그 효과가 아예 없어서, 기능을 지워도 아무것도 안 깨졌다. **잡은 방법** 새 규칙마다 해당 줄을 지우거나 끄고 돌려 보는 변이 확인(mutation check)을 거의 매 세션 했다. 한 세션에 데이터 추출기 6건, 게임 엔진 12건, 인벤토리 13건을 깨 본 적도 있다.

결함을 심어도 결과가 안 바뀌면 입력이 둔감한 것이다 — 결과가 뒤집히는 입력을 만든다

일부러 깬데 통과하면 그 테스트 입력은 그 결함에 둔감하다. 작은 차이가 결과(순서·승패)를 실제로 뒤집는 경계 입력을 만들어 넣는다. 테스트 픽스처(테스트가 공통으로 쓰는 준비 코드와 데이터)가 새 규칙을 흡수하도록 고치면, 테스트 전체가 그 규칙과 상관없이 통과할 수 있다. 그때는 그 규칙을 엔진에서 걷어 내고 한 번 돌려 본다. 깨 봐도 통과로 남는 항목은 그 규칙과 상관없다는 뜻이고, 이것도 기록한다.

사례 · 턴 순서

무슨 일이 앞으로 누가 어떤 순서로 행동할지 미리 보여 주는 예보 함수에 회귀 테스트를 넣고, 예보 코드를 네 군데 일부러 깬더니 둘이 안 잡혔다. **왜 조용했나** 깬 곳 중 둘은 대기 시간을 조금 바꾸는 작은 보정이었다. 그 정도 차이로는 행동 순서가 안 바뀌었고, 검사는 순서만 비교했다. **잡은 방법** 그 작은 차이가 순서를 실제로 뒤집는 경계 입력을 만들어 넣었다.

사례 · 스킬 규칙 추가

무슨 일이 「스킬은 쓰고 나서 일정 시간 뒤에 발동한다」 는 새 규칙을 넣으면서, 기존 회귀 10건을 살리려고 픽스처가 그 지연을 대신 처리하게 만들었다. 이러면 엔진에서 지연을 지워도 아무 테스트도 안 깨질 수 있다. **잡은 방법** 엔진의 지연 값을 0으로 바꿔 돌렸다. 전용 회귀 10건 중 8건이 깨졌고, 남은 2건은 지연과 상관없는 항목이라고 적었다.

「일부러 깨 보기」가 예상과 다르면 조작이 실제로 먹었는지부터 본다

깨 봤는데 걸린 검사가 0건이면, 검사가 죽었다고 단정하기 전에 깨는 조작 자체가 적용됐는지 확인한다. 도구가 아무 말 없이 대입을 건너뛴 수 있다. 파서처럼 갈래가 여럿인 코드는 갈래마다 그 갈래를 타는 입력 표본을 둔다. 한 번도 안 도는 갈래는 그 모양의 입력이 들어오기 전까지 드러나지 않는다.

사례 · 데이터 파이프라인

무슨 일이 기획 데이터는 엑셀로 관리하고 추출기가 그것을 게임용 JSON으로 바꾼다. 「설명 칸을 비운 엑셀이 들어오면 추출기가 거부하는가」를 보려고 Python 라이브러리 openpyxl로 칸을 비웠는데, `cell(row, col, value=None)` 은 값이 None이면 대입을 아예 하지 않는다. 칸은 비워지지 않았고, 검사가 죽은 줄 알았다. **잡은 방법** `.value = None` 으로 직접 대입해 다시 확인했다. 같은 날 공유 문자열 표가 없는 형식의 엑셀이 처음 들어오자, 코드에는 있었지만 한 번도 돈 적 없던 파서 갈래가 죽었다.

목록을 도는 검사는 비어 있지 않음을 먼저 단언하고, 기대값은 화면 밖 원천에서 가져온다

빈 목록 위의 반복문은 아무 말 없이 지나간다. 「검사한 대상 수」를 단언하거나 로그로 남겨 0이 되면 의심한다. 기대값을 검사 대상 자신에서 뽑으면, 대상이 줄어도 볼 항목이 하나 줄 뿐 통과한다. 기대 목록은 손으로 적고, 화면이 그린 값은 API·데이터 같은 화면 밖 원천과 대조한다. 검사마다 「이 기능을 지우면 이 검사가 깨지는가」를 묻는다. 시간 값은 보이는지가 아니라 줄어드는지를, 필터는 걸렸을 때 0이 되는지를 본다.

사례 · 실패할 수 없는 검사

무슨 일이 메인 화면의 건물 간판을 특정 `data-*` 속성으로 찾아 하나씩 확인하는 검사가 있었는데, 화면에 그 속성이 아예 없어 목록이 늘 비어 있었다. 「서버 소유 필드」 목록을 돌며 권한을 확인하는 회귀는 소스의 목록에서 항목 하나를 빼도 통과했다. **왜 조용했나** 하나는 확인할 대상이 0개였고, 다른 하나는 검사 대상에서 기대값을 뽑았다. **잡은 방법** 화면에 속성을 붙이자 첫 검사가 곧바로 걸렸다. 둘째는 기대 목록을 손으로 적게 바꿨다.

사례 · 화면 값 검증

무슨 일이 다친 캐릭터의 치료 남은 시간을 확인하는 검사는 「60분」이 보이는 것까지만 봤다. 기획자가 「그 숫자가 줄어드는가」를 물었고, 65초를 기다려 값이 바뀌는지 보는 검사를 더했다. 「0분」을 찾는 정규식 `/0분/` 은 「60분」에도 걸려 멀쩡한 화면을 실패로 찍었다. 숫자 앞에 경계 `(^|\d)` 를 두어 고쳤다. 부대 목록 필터 검사는 전체 줄 수를 세고 있어 필터 코드를 지워도 통과했다. 걸렸을 때 0줄이 되는 조건을 골라 다시 짰다.

확률이 끼는 동작은 결과가 아니라 결정적으로 일어나는 부분으로 판정한다

성공과 실패가 확률로 갈리는 행동을 「결과가 나왔나」로 검사하면 정상 동작을 버그로 잡는다. 자원 소모나 로그 기록처럼 결정적으로, 즉 매번 똑같이 일어나는 부분으로 성사를 판정한다. 검사를 쓰기 전에 검증 기준 자체가 규칙과 맞는지 먼저 따진다.

사례 · 확률 판정

무슨 일이 상대에게 상태이상을 거는 마법형 스킬의 시전 검사를 「상태이상이 붙었나」로 걸었다. 그런데 규칙상 상대가 저항하면 상태이상 없이 MP만 소모된다. 검사가 정상 동작을 버그로 잡았다. **잡은 방법** 기획서 규칙을 다시 대조하고, 판정 기준을 「MP가 나갔나」로 바꿨다.

1.2 검사가 그 상태에 실제로 도달하는가

도달하지 못한 상태를 보는 검사는 실패하지 않는다. 건너뛰거나 헛돌 뿐이고, 그 결과는 「통과」와 구별되지 않는다. 1.10 「검사가 틀린 화면에서 실패하는가」였다면 이 절은 「검사가 그 화면까지 가기는 하는가」다.

겨냥한 상태에 도달하지 못하는 검사는 돌지 않는다 — 흉내 낼 통로를 검사와 함께 만든다

검사를 쓰기 전에 그 검사가 겨냥한 갈래에 실제로 들어가는지 확인한다. 아직 붙지 않은 기능(서버·상대·외부 서비스)에 걸린 검사는 엉뚱한 조기 반환을 확인하고 끝낸다. 도달할 수 없으면 그 상태를 흉내 낼 통로를 검사와 같이 만든다. 회귀 테스트 안에만 있는 가짜 구현이면 제품 화면을 더럽히지 않고 낚지도 않는다. 판정을 하는 층과 그것을 그리는 화면 층은 각각 따로 깨 봐야 둘 다 살아 있는지 안다.

사례 · 아직 없는 서버

무슨 일이 매칭된 상대를 거절하고 다시 찾는 기능은 온라인 대전에만 있는데, 그때는 온라인 서버가 아직 없어 매칭이 늘 AI 상대로 떨어졌다. 그 기능의 완료 조건 두 줄이 한 번도 실행되지 않았다. 다음 날에는 전투 재생기의 「연출이 도는 동안에는 대기열에서 다음 메시지를 꺼내지 않는다」 잠금을 지켜야 할 회귀가, 그 줄을 지워도 통과했다. 오프라인 대전의 전송 계층은 요청해야 답하는 구조라 대기열이 늘 비어 있었기 때문이다. **잡은 방법** 온라인 매칭을 흉내 내는 개발용 URL을 만들었다. 재생기 쪽은 회귀 안에만 있는 가짜 서버를 두어 한 판 분량의 메시지를 통째로 밀어 넣었고, 연출 중에 N통을 한꺼번에 꺼냈다 로 잡혔다.

조건부 검사는 조건이 성립하는 데이터를 검사 옆에 심고, 「건너뛰」을 따로 센다

건너뛰 것은 실패로 보고되지 않는다. 조건이 맞을 때만 도는 검사는 그 조건이 성립하는 입력을 명시적으로 고른다. 새 계정이나 최소 데이터로는 안 그려지는 갈래가 많다. 확인 도구마다 도달시켜야 할 갈래 목록을 먼저 적고, 그 갈래를 만드는 시드 데이터(검사 전에 미리 심어 두는 데이터 — 쪽 수·레벨·인원)를 검사 옆에 둔다. 데이터를 늘리면 새 흐름(반복해서 뜨는 팝업 등)이 드러날 수 있으니 함께 처리한다.

사례 · 건너뛰 검사

무슨 일이 캐릭터의 고유 스킬 설명을 확인하는 검사는 살펴볼 캐릭터가 고유 스킬을 가졌을 때만 돌아왔다. 그런데 캐릭터를 아무나 고르다 보니 고유 스킬이 없는 하위 등급 캐릭터가 뵈혀 검사가 건너뛰어졌다. **잡은 방법** 스모크 테스트 코드를 다시 읽다 발견했고, 고유 스킬을 가진 캐릭터를 우선 고르게 고쳤다.

사례 · 시드 데이터

무슨 일이 무기 아이템을 둘만 심으면 목록의 쪽 나눔 단추가 아예 그려지지 않았다. 새 계정은 제작 건물이 1레벨이라 최고 레벨 제작 목록 검사가 안 돌아왔다. 무기를 종류마다 한 자루씩만 심으면 「같은 무기 둘」 갈래가 한 번도 그려지지 않았다. **잡은 방법** 무기 7개, 캐릭터 124명, 제작 건물 레벨 지정으로 시드를 늘렸다. 그러자 「완성!」 팝업이 받지 않은 품목마다 떠서, 팝업을 한 번만 달던 스모크 테스트가 죽었다. 그것도 같이 고쳤다.

흉내 낸 상태는 사람이 실제로 만들 수 있는 상태여야 한다

사람이 도달할 수 없는 상태를 검사하면 거짓 버그가 나온다. 「검사할 상태에 도달하지 못하는 검사」의 반대쪽 함정이다. 흉내는 사람이 실제로 조작할 수 있는 요소만 조작한다.

사례 · 팝업 위치

무슨 일이 상점 화면의 팝업 위치를 눈으로 확인하면서 「목록을 아래로 내린 뒤 팝업을 연다」를 흉내 내려고, 화면 요소의 `scrollTop` 을 코드로 밀었다. 팝업이 278px 벗어나 「제자리에 없다」가 나왔다. **왜 조용했나** 그 요소는 `overflow: hidden` 이라 사람은 그 상태를 만들 수 없었다. 버그가 아니라 흉내가 틀린 것이었다.

검사가 우연한 진행 순서에 기대지 않게 한다 — 필요한 전제는 검사 안에서 만든다

「다음 차례는 A다」 「목록 첫 줄의 캐릭터는 아직 아무 데도 배치되지 않았다」 같은 전제가 우연히 늘 참이면, 검사가 그 우연에 기대고 있다는 사실이 드러나지 않는다. 필요한 전제는 검사 안에서 확인하고, 맞지 않으면 직접 만든다. 「첫 줄」이나 고정 id 대신 실제로 고른 요소의 속성을 읽는다.

사례 · 규칙 변경의 여파

무슨 일이 대기 시간이 같은 캐릭터끼리의 순번을 정하는 규칙을 바꾸자, 테스트에 쓰던 고정 난수 시드에서 행동 순서가 달라졌다. 「턴을 넘기면 AI가 행동해 말풍선이 뜬다」를 전제한 스모크 테스트가 거짓 실패했다. 다음 차례도 사람 쪽 캐릭터라 아무도 행동하지 않았다. **잡은 방법** 사람 차례면 한 번 더 넘기도록 전제를 검사 안에서 만들었다. 연출 길이처럼 들쭉날쭉한 시간은 넉넉한 상한을 두고 실제 상태를 기다리게 바꿨다.

1.3 검증 층을 나누고, 자동 검사를 살려 둔다

단위 테스트·통계·화면 연동·사람의 플레이는 서로를 대신하지 않는다. 그리고 화면을 처음부터 끝까지 누르는 E2E(End-to-End) 검사, 우리 팀이 「스모크」라고 부르던 것은 가장 넓게 잡지만 가장 쉽게 낚는다. 낚는 방식이 조용해서 「원래 깨져 있다」로 받아들여지기 쉽다.

검증 층을 나누어 각각 둔다 — 층마다 잡는 버그가 다르다

데이터 정합성·순수 로직·통계·화면 연동·전체 흐름·눈검사를 따로 둔다. 각 단위가 개별 테스트를 통과해도 문제는 대량 반복이나 단위 사이의 이음매에 숨어 있다. 「규칙이 옳은가」(단위 테스트)와 「그 규칙이 실제 진입점에서 불리는가」(끝단 검사)는 다른 검사다. 각 층이 무엇을 맡는지 문서에 갈라 적는다. 한쪽이 다른 쪽을 대신한다고 쓰면 과장이 된다.

사례 · 통계와 연동

무슨 일이 AI끼리 5000판을 자동으로 붙이는 밸런스 시뮬레이션이 「행동 순서 스케줄러가 멈추는 버그」와 「공격 대상 판정이 빠지는 버그」를 잡았다. 메타 화면 스모크 테스트(편성·보상·저장 같은 전투 밖 화면을 훑는 검사)는 「규칙도 맞고 전투도 도는데 화면이 돌을 잊지 못하는」 버그를 잡았다. 편성한 부대가 엔진에 안 넘어가고, 저장이 안 되고, 전투 결과가 계정에 반영되지 않았다. 셋 다 단위 테스트는 통과하던 것이다.

사례 · 저장 형식 마이그레이션

무슨 일이 게임을 업데이트하면 계정 저장 형식이 바뀐다. 옛 형식을 새 형식으로 바꿔 주는 마이그레이션 함수를 단위 테스트로 조목조목 검증했다. 그러나 그 테스트는 「계정을 불러올 때 그 함수를 실제로 부르는가」는 모른다. 옛 코드의 「버전이 다르면 계정을 버린다」한 줄이 되살아나도 단위 테스트는 통과한다. **잡은 방법** 스모크 테스트가 옛 1버전 저장 데이터를 브라우저에 실제로 넣고 새로고침하게 했다. 옛 동작을 잠깐 되살려 $\times$ v1 저장분을 만나자 계정을 버렸다 로 잡히는 것까지 봤다.

움직임과 타이밍은 눈이 아니라 시간표와 시계열로 잔다

눈은 0.2초와 0.3초를 구분하지 못하고, 한 장면은 언제나 정상처럼 보인다. 연출 로직을 렌더러 없이 돌 수 있게 분리해 시간표를 단위 테스트한다. 실제 화면 확인이 필요하면 프레임 단위 시계열을 남겨 규칙으로 판정한다. 스모크 테스트는 「보이는가·제자리인가」만 말한다. → 5장 클라이언트 엔지니어

사례 · 연출 테스트

무슨 일이 전투 연출을 지휘하는 모듈과 카메라 모듈을 렌더러를 부르지 않게 짰더니, 화면 없이(헤드리스로) 시간표를 채는 첫 클라이언트 테스트가 생겼다. 캐릭터가 칸을 건너뛰는 순간이동은 30초 동안 프레임마다 그려지는 칸을 훑어 「인접하지 않은 칸으로 건너뛰면 실패」로 탐지했다.

고정 대기 시간을 쓰지 않는다 — 실제 상태를 기다린다

고정 대기는 연출 길이를 테스트에 한 번 더 적은 것이다. 연출이 바뀌어도 아무도 알려 주지 않는다. 「연출 끝남」 「카메라 정지」 「서버가 그 값을 본다」 같은 관측 가능한 조건을, 상한 시간을 두고 기다린다.

사례 · 막힌 스모크 테스트

무슨 일이 스킬을 쓸 때 뜨는 배너를 확인하는 UI 스모크 테스트가 1300ms를 기다렸는데, 연출을 늘리면서 실제 배너 길이가 2000ms가 됐다. 그때부터 그 검사는 **산술적으로 통과할 수 없었다**. 스모크 테스트는 첫 실패에서 멈추므로, 그 뒤에 있던 검사 20여 건이 2주 동안 한 번도 돌지 않았다. **잡은 방법** 고정 대기를 Playwright의 `waitForFunction` 으로 실제 연출 단계를 기다리는 방식으로 바꿨고, 2주 만에 처음으로 끝까지 완주시켰다.

선택자는 글자·순서·모양이 아니라 뜻 속성에 걸고, 자리는 실측 좌표로 본다

화면 글자·`nth-child`·모양 클래스로 요소를 고른 검사는 UI가 바뀌면 실패하지 않는다. 대상이 0개가 되어 조용히 통과한다. `data-side`·`data-action` 같은 뜻 속성에 건다. 마크업을 나누면 상태 속성이 어느 노드로 옮겨 갔는지 보고, 검사도 같은 커밋에서 옮긴다. 속성은 코드가 의도한 값일 뿐 렌더 결과가 아니다. 「어디에 있나」는 박스 좌표로, 「누를 수 있나」는 그 좌표의 최상위 요소(`elementFromPoint`)로 본다. 이미지 같은 예제는 「지정했나」와 「실제로 받아오나(HTTP 200)」를 따로 본다. CSS 배경 이미지가 404면 오류 없이 바탕색으로 물러난다.

사례 · 글자로 고른 선택자

무슨 일이 「상대가 가진 스킬은 정보 팝업에서 가린다」 스모크 테스트가 팝업의 「적군」이라는 글자로 상대 캐릭터를 골랐다. 팝업에서 그 글자를 빼자 조건이 늘 거짓이 되었고, 무엇을 노출해도 통과했다. **잡은 방법** 검사 로그의 「스킬 가림」이 「스킬 0중」으로 바뀐 것을 보고 알았다.

사례 · 속성과 실제 위치

무슨 일이 전투 커맨드 패널이 화면 좌상단에 눌러앉았는데 요소에는 `data-x="right" data-y="bottom"` 이 붙어 있었다. 그것을 보고 「자리는 맞다」고 판단해 엉뚱한 곳을 뒤졌다. 스킬 대상 칸을 눌러도 반응이 없던 것은, 그 칸 위에 「취소」 단추가 떠서 클릭을 가로채고 있었기 때문이다. 이것도 `elementFromPoint` 로 찍고서야 보였다.

첫 실패에서 멈추는 검사의 막힘은 그날 풀고, 테스트 수가 줄면 실패로 본다

앞에서 실패 하나가 나면 뒤의 검사는 「실패」가 아니라 「실행 안 됨」이 된다. 막힌 채 두면 뒤쪽 검사가 화면 변경을 못 따라와 전부 낡는다. 막힘은 그날 풀고, 끝까지 완주한 것을 보고 나서 통과라고 말한다. 규칙이나 화면 구조를 바꾸는 커밋은 그것을 지나는 검사도 함께 바꾼다. 검사 안에 숫자를 박지 말고 제품 코드의 상수를 읽게 하고, 오류는 첫 건만이 아니라 전부 찍는다. 통과 여부만 보지 말고 테스트 수가 이전보다 늘거나 같은지도 본다. 파일째 로드되지 않은 테스트는 빨간불을 내지 않는다.

사례 · 「원래 깨져 있다」

무슨 일이 메타 화면 스모크 테스트가 여러 세션에 걸쳐 첫 화면에서 멈춰 있었다. 게임 이름 변경, 지원 언어 5개에서 10개로 확장, 모양 클래스로 건 선택자 등 아홉 군데가 원인이었다. 그 탓에 AI 대전에서 입장 비용이 빠지는 구멍을 아무도 못 봤다. 나중에는 한 자리에서 12일을 멈춰 있는 동안 뒤쪽 열한 자리가 났었다. 숨어 있던 콘솔 오류 8건과 `<p>` 안의 `<h2>` 같은 마크업 오류도 막힘이 풀린 뒤에야 보였다. **왜 조용했나** 팀이 「스모크는 원래 깨져 있다」 고 받아들였다.

사례 · 사라진 테스트

무슨 일이 테스트 파일 하나가 이미 지운 상수를 import해서 파일째 로드되지 않았는데, `npm test` 는 통과로 끝났다. 테스트 파일이 TypeScript 타입 검사 대상이 아니어서 컴파일러도 못 잡았다. **잡은 방법** 테스트 합계가 662에서 646으로 줄어든 것을 보고 알았다.

1.4 실패를 만났을 때 — 원인을 가르다

검사가 빨개지면 손이 먼저 코드로 간다. 그러나 우리가 겪은 실패의 상당수는 코드가 아니라 시작 상태·환경·검사 자신에 있었다. 무엇이 틀렸는지 가르는 순서를 정해 두면 엉뚱한 곳을 덜 뒀다.

「이 실패는 원래 있던 것인가」는 변경을 걸어 낸 기준선으로 답한다

실패가 내 변경 탓인지는 기억이 아니라 깨끗한 트리에서 다시 돌려서 답한다. 검사를 고치기 전에도 원래 코드로 먼저 완주시켜 「내 변경 탓」과 「원래부터의 결함」을 가르다. 기존 실패를 두면 매 세션 이 확인 비용을 치르고, 새 실패가 그 사이에 섞여 숨는다. 빨리 고치거나, 이유와 담당자를 붙여 할 일 목록에 올린다.

사례 · 방치된 실패

무슨 일이 게임 제목을 바꾼 작업이 테스트 기대값을 갱신하지 않아 2건이 실패했다. 다음 작업자는 자기 변경을 걸어 내고 돌려 무관함을 확인했지만, 범위 밖이라며 고치지 않았다. 그 실패는 주 끝까지(444건 중 442건 통과에서 463건 중 460건 통과로) 매 세션 따라다녔다. 다음 주에도 기존 실패 3건 때문에, 매 세션 변경분을 걷고 같은 실패가 나는지 확인한 뒤 되돌리는 일을 반복했다.

사례 · 기준선 먼저

무슨 일이 막힌 스모크 테스트를 고치기 전에 변경 전 코드로 먼저 완주시켜 봤다. 막힌 두 곳은 내 변경 탓이었지만, 「상태 배지 설명」 검사는 변경 전에도 이미 건너뛰고 있던 갈래였다. **처리** 그 갈래는 「통과」로 치지 않고 별도의 할 일로 로드맵에 올렸다.

테스트가 실패하면 규칙보다 먼저 시작 상태를 의심한다

검사는 시작 상태에 대한 암묵적 전제를 품고 있다. 절대값 대신 앞뒤 차이로 재서 그 전제를 줄인다. 시작 상태를 바꾸는 기능을 넣을 때는 그 상태에 기대던 기존 검사를 찾아본다.

사례 · 시작 상태

무슨 일 「승리하면 대전 입장에 드는 소모 자원을 1 돌려받는다」 검사가 열두 조합 중 넷에서 실패했다. 규칙을 세 번 다시 읽었는데, 원인은 새 계정이 그 자원의 상한(20)에서 시작해 받을 자리가 없었던 것이다. 같은 날 「배치를 미리 저장하는 기능」이 들어와 캐릭터를 맨 윗줄로 옮기자, 옛 이동 검사가 배치 구역 밖으로 깨졌다. 그 검사는 캐릭터가 늘 같은 행에서 시작한다고 말없이 전제하고 있었다. **잡은 방법** 테스트의 시작 자원을 0으로 고쳤다. 덕분에 「상한에 걸리면 들어간 만큼만 기록한다」가 따로 회귀로 굳었다. 입장 비용 검사는 앞뒤 차이를 재게 바꿨다.

이상이 보이면 코드보다 바깥부터 쟀다

코드를 의심하기 전에 환경(에셋 로딩·DB 지연·떠 있는 프로세스)·데이터·검사 자신을 먼저 본다. 서버를 고쳤으면 「지금 도는 프로세스가 그 코드인가」부터 확인한다 → 4장 서버 엔지니어. 이상해 보이는 그림은 미리보기가 아니라 파이프라인 산출물과 픽셀 통계를 판단한다 → 6장 디자인 엔지니어. 출처를 모르는 파일은 지우기 전에 읽어서 왜 있는지 파악한다.

사례 · 바깥의 원인들

무슨 일 한 주에 네 번 코드를 의심했는데 넷 다 바깥이었다. 그림이 없어 보인 스크린샷은 CSS 문제가 아니라 그림을 아직 다 받지 못한 순간에 찍힌 것이었다. 로그인에서 멈춘 스모크 테스트는 DB 요청 한 번이 12.8초 걸렸다. 떠 있던 API 서버는 파일 변경 감시가 없어 옛 코드로 돌고 있었고, 그래서 새로 만든 경로가 404였다. 「목록 첫 줄 캐릭터」를 고른 스모크 테스트는 그 캐릭터가 이미 다른 자리에 배치돼 있어서, 규칙이 맞게 거부한 것이었다. **잡은 방법** 계산된 스타일·응답 시간·프로세스를 먼저 확인했다. 테스트 쪽이 틀린 것은 테스트를 고쳤다.

사례 · 그림 파일

무슨 일 가차 화면에 쓸 캐릭터 그림을 이미지 미리보기로 열었더니 배경에 노이즈가 보였다. 그러나 알파 채널(투명도) 분포는 정상이었고, 노이즈는 미리보기 프로그램의 압축 아티팩트였다. 폴더에 커밋되지 않은 전처리 스크립트가 있었는데, 지우기 전에 읽어 보니 전처리가 이미 끝났다는 증거였다.

체감과 임계값은 먼저 숫자로 만든다 — 정상 분포와 대조군을 함께 쟀다

「~같다」는 피드백은 측정 가능한 지표로 바꾼 뒤 원인을 찾고, 수정 결과도 같은 지표로 보고한다. 임계값을 정하기 전에는 정상값이 어디까지 흔들리는지와, 막고 싶은 실패(대조군)가 어디에 있는지를 함께 재서 그 사이에 선을 긋는다. 대조군이 없으면 원래 안 커지는 것을 안 커진다고 확인하는 검사가 된다.

사례 · 체감을 숫자로

무슨 일 「전투 로그 대화창이 판의 진행을 못 따라가는 것 같다」는 피드백을 받았다. 60초 동안 계속해 「밀린 줄 수 평균 2.7, 최대 8」로 바꾸고서야 원인 둘이 나왔다. 줄 사이 간격이 1초로 고정돼 있었고, 대기 시간 계산이 연출 시간만 세고 있었다. 고친 뒤 「평균 1.51, 행동 사이마다 0으로 따라잡는다」로 보고했다.

사례 · 임계값 찾기

무슨 일 온라인 대전에서 서버가 보내는 상태 스냅샷은 전투가 길어져도 크기가 커지면 안 된다. 이것을 「뒤쪽 스냅샷 ≤ 앞쪽 스냅샷」으로 적었더니, 5대5 대전에서 깨졌다(3,401 → 3,823 B). 상태이상 같은 정보 때문에 조금은 늘어나는 것이 정상이었다. **잡은 방법** 난수 시드 40개로 정상(가장 큰 것이 첫 것의 1.33~1.54배)과, 전체 전투 로그를 함께 실은 대조군(21~23배)을 같이 쟀다. 사이가 열 배 넘게 벌어져 있어 임계값을 2배로 그었다.

1.5 확인하지 못한 것을 적는다

적어 두지 않은 공백은 「확인됐다」로 읽힌다. 검증 보고의 값어치는 통과 목록보다 그 경계에 있다. 우리 팀은 이 절의 원칙을 주간 QA 기록의 「확인하지 못한 것」 칸으로 굳혔다.

확인하지 못한 것을 목록으로 남긴다

검증 보고에는 통과한 것과 함께, 어떤 방법으로도 확인하지 않은 것, 흉내 통로로만 본 것, 고치고도 돌리지 않은 것을 나란히 적는다. 돌리지 않았으면 통과라고 말하지 않고 「안 돌렸다」로 적는다. 간헐 실패는 「원인 미확인」으로 남긴다. 검사 대상이 사라진 스모크 테스트 구간은 지우지 말고 「안 돈다」 표시를 남긴다. 각 항목은 로드맵의 특정 단계에 배정해 잊히지 않게 한다. 개발용 통로에서 잘 돌면 실제 경로도 된다고 착각하기 쉬운데, 통로는 발동 조건 자체를 건너뛰는 거다.

사례 · 주간 검증 기록

무슨 일이 한 주의 검증 기록에 「소리를 브라우저로 직접 듣지 않았다」 「아홉 개 언어 번역은 대표 8건만 검수했다」 「마법형 스킬 팝업에는 스모크 테스트가 없다」 「계정 전적 구간은 들어갈 화면이 없어서 안 돈다」가 따로 적혔다. 실제로 그 주의 소리 타이밍 버그는 직접 들어 보지 않은 영역에서 나왔다.

사례 · 흉내 통로로만 본 것

무슨 일이 대기 시간이 같을 때의 순번 규칙 변경은 단위 회귀와 UI 스모크 테스트로만 확인했고, 메타 화면·이벤트 전투·온라인 스모크 테스트는 돌리지 않았다. 순번을 정하는 주사위 연출은 개발용 URL로만 봤고, 실제 계정으로 들어간 대전의 배치 화면에서는 보지 못했다. 둘 다 「확인하지 못한 것」 칸에 적어 다음 단계로 넘겼다.

검사가 지키지 못하는 성질은 「안 잡힌다」고 사실대로 적는다

한 번도 돌지 않는 검사를 하나 더 만드는 것보다, 무엇이 검사로 안 잡히는지와 어떻게 확인했는지(코드 검토만 함, 실행 안 함)를 남기는 편이 낫다. 일부러 깨 보는 표(무엇을 깼나 / 잡혔나)에 「안 잡힌다」를 그대로 적는다. 지금 발동시킬 입력을 만들 수 없는 방어 코드는 가정이다. 남기더라도 「지금은 여기서 걸리는 일이 없다. 두 단계가 갈라지는 순간 잡으려고 둔 것」이라고 주석으로 적는다.

사례 · 과장된 주석

무슨 일이 전투 재생기에는 같은 함수가 자기 자신을 다시 부르지 못하게 막는 재진입 잠금이 있다. 이 잠금은 지워도 회귀가 통과한다. 잠금이 막는 것은 재귀 깊이이지 처리 순서가 아니어서, 결과가 같기 때문이다. 그런데 문서에는 「재진입 잠금이 이 함수의 요점」이라고 적혀 있었다. **처리** 깨 보기 표에 「안 잡힌다」를 적고 주석의 과장을 고쳤다.

사례 · 중복 방어

무슨 일이 「이 아이템을 전투에서 쓸 수 있는가」를 판정하는 함수에서, 아이템 종류가 액티브인지 보는 줄은 지워도 아무 검사가 깨지지 않았다. 패시브 아이템에는 효과 데이터가 없어서 바로 다음 줄이 먼저 걸려 났기 때문이다. 두 줄은 뜻이 달라(하나는 규칙, 하나는 데이터 방어) 둘 다 남겼다. 대신 테스트로는 둘을 구분할 수 없다는 것과, 실제로 지키는 검사가 어느 것인지를 적었다.

자동화가 볼 수 없는 것은 담당자를 정해 사람에게 넘기고, 사람이 잡은 버그는 검사로 굳힌다
스크린샷이 확인하는 것은 「동작하는가」 까지다. 소리·박자·색감·밀도·원어민 수준의 번역 같은 감각 검증은 담당자를 지정해 넘긴다. 끝까지 진행해 봐야 나는 버그도 사람의 플레이가 잡는다. 화면 작업 뒤에는 실제 플레이 확인을 요청하고, 사람이 잡은 버그는 그 자리에서 자동 검사 항목으로 굳혀 다음에는 기계가 잡게 한다. 실제 CSS만 얹은 목업 화면은 잘림·깜빡임 같은 결함 판단에만 쓰고, 「답답하다」 같은 느낌은 실제 크기 화면에서 판단한다 → 6장 디자인 엔지니어.

사례 · 사람의 플레이

무슨 일이 한 주의 버그 셋이 모두 기획자의 실제 플레이로 잡혔다. 낼 수 있는 행동이 「턴 종료」 하나뿐인데 그 단추가 없어 게임이 멈추는 교착, 대화창 글자가 사라지는 문제, 자유 이동 횟수가 소모되지 않는 문제다. 왜 조용했나 셋 다 한 턴을 끝까지 진행해 봐야 하거나, 특정 화면 크기·특정 구성에서만 나는 종류라 스크린샷과 단위 테스트가 지나쳤다.

1.6 작업을 나누고, 정본을 하나로 둔다

여러 세션·여러 사람에 걸친 작업에서 가장 흔한 사고는 두 가지다. 같은 정보가 두 곳에 적혀 한쪽만 낡는 것, 그리고 앞 사람이 남긴 「없다」 「미룬다」를 다시 확인하지 않는 것이다. 둘 다 아무 경고도 내지 않는다.

큰 작업은 동작이 안 바뀌는 단계로 쪼개고, 감쌀 흐름을 먼저 완성한다

겉으로는 동작이 하나도 안 바뀌는 단계의 값어치는, 나중에 깨질 곳을 좁혀 두는 데 있다. 한 세션에 한 단계씩 진행하고, 매 세션 「문서 읽기 → 진도 확인 → 진행 중 표시 → 구현 → 검증 → 완료 표시·기록·커밋」 절차를 입구 문서에 고정한다. 네트워크 코드처럼 무언가를 감싸는 층은 그것이 실어 나를 단계와 검증할 규칙이 확정된 뒤에 짤다. 기획 결정은 미룬다고 더 잘 정해지지 않으니 답할 수 있을 때 답하고, 「어떻게 짤까」는 코드를 열 때까지 미룬다.

사례 · 작업 순서

무슨 일이 원래 계획은 실시간 대전 서버를 먼저 만드는 것이었다. 이것을 뒤집어 「부대 편성 → 배치 → 경찰 → 전투 → 보상 → 저장」으로 이어지는 오프라인 한 바퀴를 먼저 완성하기로 했다. 화면 없이 서버부터 만들면 편성·배치 화면을 붙일 때 통신 프로토콜을 다시 손대게 된다. 편성 규칙도 결국 서버가 검증할 대상이라, 오프라인에서 먼저 굳히는 편이 낫다고 봤다. 순서만 바꿨을 뿐 기존 설계는 하나도 버리지 않았다. → 4장 서버 엔지니어

사례 · 단계 쪼개기

무슨 일이 온라인 대전 작업을 전송 계층(서버 코드 0줄)·대전 방·계정 세 단계로 쪼갰다. 전송 계층은 브라우저 없이 회귀 테스트만으로 완전히 검증됐다. 서버를 붙인 뒤 문제가 생겨도 「전송 때문인가, 방 때문인가, 시계 때문인가」를 가릴 수 있었다. 첫 단계가 끝난 뒤 다음 단계에 해당하는 기획 질문 넷이 나왔는데, 미루지 않고 바로 답했다. 그 문답에서 「대전 도중 연결을 끊는 쪽이 매칭을 거절하는 쪽보다 손해가 적으면 안 된다」는 규칙이 나왔다.

같은 정보는 한 곳만 정본으로 둔다

진도·수치·판정을 두 곳에 적으면 한쪽만 낡아도 아무 경고가 없다. 문서는 입구(절차)·진도·설계처럼 파일마다 「정하는 것」을 하나씩 갖게 가른다. 화면은 엔진·서버의 판정 함수 결과를 읽기만 하고, 같은 기능의 입구는

하나만 둔다. 계획 문서의 완료 조건도 작성 이후의 결정에 밀려 낡는다. 구현 전에 기존 코드·회귀와 대조하고, 어긋나면 문서를 고친다. 결정이 뒤집혀도 그 시점의 기록은 고치지 않는다. 정본 문서의 결정표에 「뒤집힘」을 표시한다. 그래야 다음 사람이 같은 안을 다시 제안하지 않는다. 기획 수치의 정본 이야기는 → 2장 게임 디자이너.

사례 · 낡은 완료 조건

무슨 일이 전적 기능의 완료 조건에 「계정 전적의 합 = 캐릭터 전적의 합」이라고 적혀 있었다. 그런데 한 판에 캐릭터 3~5명이 함께 뛰므로 둘은 같을 수 없다. 회귀에는 이미 반대 방향의 검사가 있었다. **왜 조용했나** 그대로 믿고 구현했다면 화면에는 「계정 전적이 좀 크네」로만 보였을 것이다. **잡은 방법** 구현 전에 완료 조건을 기존 회귀와 대조했다. 조건을 ≠ 로 고쳤고, 스모크 테스트가 「계정 6전 ≠ 그 캐릭터 4전」을 화면 숫자로 확인한다.

사례 · 문서 가리기

무슨 일이 전투 UI 개편을 「한 세션에 한 단계」로 진행하면서 문서를 셋으로 갈랐다. 마스터 문서(세션 절차와 규칙), 로드맵(단계 표와 체크 항목), 설계 문서(요구 사항과 확정 사항)다. 설계 문서에 있던 진행 표는 지우고 로드맵을 가리키는 한 줄로 바꿨다.

「없다」는 근거로 쓰기 전에 다시 찾는다

「없다」는 「있다」보다 검증이 싸다. 앞 세션이나 동료가 남긴 「없음」 「자료 미도착」을 근거로 쓰기 전에 한 번 다시 찾는다. 같은 내용을 담은 구조화된 소스(CSV·표)가 있는지를 산문 파싱보다 먼저 찾는다. 찾아낸 결정 이력은 해당 코드의 머리말 주석에 남겨, 다음 사람이 같은 발견을 반복하지 않게 한다.

사례 · 잘못된 「없음」

무슨 일이 앞 세션이 「게임 안에 달력 개념이 없다」 「로그인 정보가 없다」고 남겼다. 실제로는 기획서에 「게임 시간 1초 = 하루」가 적혀 있었고, 환경 설정 파일에 로그인 값이 다 있었다. 그 「없음」 때문에 실제 로그인을 타는 스모크 테스트를 하루 미뤘고, 아이템 제작 기간의 단위도 잘못된 전제 위에서 정했다. **잡은 방법** 기획자가 「1주라는 건 게임 세상의 1주 아니냐」고 되물었고, 그 말을 듣고 문서를 다시 찾았다.

사례 · 다국어

무슨 일이 캐릭터 소개문을 게임에 연결하는 첫 작업은 한국어만 있고 「다른 언어는 미룬다」고 남겼다. 그런데 원본 CSV에는 처음부터 아홉 개 언어 열이 다 있었다. 그보다 앞서서는 산문 문서를 정규식으로 긁어내다가, 같은 내용을 열로 갈라 둔 CSV를 뒤늦게 발견하기도 했다.

규칙을 거스르거나 되돌리기 비싼 결정은 확인받고, 모호한 낱말은 대상을 콕 집는다

팀이 정해 둔 규칙을 거스르는 결정, 파일 이름 변경처럼 파급이 따르는 결정은 근거를 적어 확인받는다. 읽기 전용 원본은 그대로 두고 정규화 계층에서 고친다. 같은 낱말이 두 사람에게 다른 것을 가리킬 수 있으니, 기술 용어는 「무엇을」 가리키는지 콕 집어 다시 말한다. 화면이 서버 설정에 따라 달라져야 하면 화면이 추측하지 말고 서버에 묻는 길을 둔다.

사례 · 이름 표기 통일

무슨 일이 팀 규칙은 「캐릭터 이름 표기의 표준은 초상화 파일명」이었고, 그대로 따르면 게임 데이터 쪽 표기가 맞았다. 그러나 기획자가 캐릭터 소개문을 새로 쓰며 검증한 CSV 쪽 표기가 맞다는 확신이 있었다. 초상화와 연출 파일의 이름까지 바뀌야 하는 일이라 확인을 받은 뒤 8명을 CSV 표기로 통일했다. 원본 엑셀은 건드리지 않고, 추출기의 이름 정정 표에서 고쳤다.

사례 · 같은 낱말, 다른 대상

무슨 일이 테스트용으로 캐릭터를 다치게 만드는 개발 기능이 있었다. 개발자에게 「스위치」는 이 기능을 켜고 끄는 서버의 환경 변수였는데, 기획자는 화면에 있는 [개발용: 부상시키기] 줄을 스위치로 읽었다. 그 줄은 서버 쪽 스위치를 끈 뒤에도 늘 그려져 있었고, 누르면 「꺼져 있다」는 응답만 받았다. 양쪽 다 「스위치를 껐다」고 말하면서 서로 다른 것을 확인하고 있었다. **잡은 방법** 화면이 서버의 상태 확인용 엔드포인트에 개발 기능이 켜졌는지 묻게 했다. 켜져 있을 때만 그 줄을 그리고, 서버에 못 닿거나 404면 숨긴다.

1.7 저장소와 도구를 다룰 때

이 절의 사고는 기능과 상관없이 누구에게나 일어난다. 비밀이 새고, 작업이 사라지고, 파일이 통째로 바뀐다. 셋 다 명령은 성공으로 끝나서 그 자리에서는 아무도 모른다.

비밀 파일은 거르지 말고 필요한 키 하나만 읽는다

「위험한 낱말이 든 줄은 빼고 출력」하는 필터는 줄 이름만 보므로, 목록에 없는 이름의 비밀은 그냥 통과한다. 비밀 파일은 걸러서라도 통째로 출력하지 않는다. 블랙리스트가 아니라 화이트리스트로, 필요한 키 하나만 정확히 읽는다. 새면 곧바로 알리고 교체한다.

사례 · 비밀 유출

무슨 일이 환경 설정 파일(.env)을 `grep -v "KEY\|SECRET\|PASSWORD"` 로 걸러 터미널에 출력했다. 그런데 Supabase DB 연결 문자열 줄은 이름에 세 낱말이 하나도 없어서, 비밀번호가 든 연결 문자열이 그대로 찍혔다. **처리** 출력을 보자마자 팀에 알리고 DB 비밀번호 교체를 안내했다.

되돌리기 명령은 범위를 좁히고, 커밋하지 않은 분량을 먼저 본다

되돌리거나 임시 보관(`git stash`) 전에 `git status` 와 `git diff --stat` 으로 무엇이 들어가는지 본다. 범위는 문제의 파일로 좁힌다(`git stash -- <경로>`, 또는 별도 워크트리). 꼬이면 통째로 되돌리지 말고 충돌한 파일만 파일 단위로 복구한다. 스테이징하지 않은 변경은 git이 기억하지 않는다. 여러 세션에 걸쳐 쌓이는 작업(번역 등)은 중간중간 커밋한다.

사례 · 사라진 번역

무슨 일이 파일 하나의 잘못된 편집을 되돌리려고, 다국어 문자열 디렉터리 전체에 `git checkout --` 를 실행했다. 마지막 커밋 뒤 일곱 차례에 걸쳐 쌓인 편집과, 한 번도 커밋하지 않은 아홉 개 언어의 문자열이 전부 사라졌다. 스테이징한 적 없는 변경이라 reflog로도 찾을 수 없었다. **복구** 작업 세션의 기록(3만 줄대)에 남아 있던 모든 편집의 전후 문자열을 순서대로 다시 적용해 되살렸다. 커밋본과 diff해서 정확히 그 변경분만 돌아온 것을 확인했다(타입 오류 47건 → 0, 41개 키가 10개 언어에서 일치). 운이 좋았던 것이고, 이것은 최후 수단이지 작업 방식이 아니다.

사례 · 꼬인 stash

무슨 일이 작업 전부터 수정돼 있던 바이너리 문서(.pptx) 때문에 `stash pop` 이 파일 삭제 오류로 걸렸다. 다음 날에는 자동 생성 파일의 충돌로 `pop`이 실패해, 세션의 모든 변경이 `stash` 스택에 갇혔다. 두 번 다 `git checkout stash@{0}` -- <파일> 식의 파일 단위 복구로 손실 없이 빠져나왔다.

도구로 파일을 고치면 줄 끝과 서식이 조용히 바뀐다 — 바꿀 부분만 바꾼다

파싱한 뒤 다시 직렬화해 저장하면, 라이브러리 입장에서는 「같은 내용」을 저장한 것이라 오류가 없다. 하지만 사람이 넣어 둔 구조(빈 줄로 나눈 묶음, 공유 문자열 표)는 지워진다. 사람이 관리하는 파일은 바꿀 부분만 텍스트 수준에서 바꾼다. Windows에서 Python으로 파일을 쓸 때는 `newline='\n'` 을 명시한다. 일괄 편집 뒤에는 `diff`가 의도한 줄뿐인지 본다. → 3장 시스템 엔지니어

사례 · 통째로 바뀐 파일

무슨 일이 Windows의 Python `open(p, 'w')` 가 줄 끝을 CRLF로 바꿔, 파일 11개가 `diff`상 통째로 바뀌었다. 저장소는 LF로 통일돼 있었고, 같은 실수를 한 주에 두 번 했다. 번역 JSON을 파싱했다가 다시 쓰자 묶음 사이의 빈 줄 39개가 사라졌다. 그보다 앞서서는 `openpyxl`로 엑셀을 전체 재저장했더니 공유 문자열 표가 통째로 사라져, 그 표를 읽던 자체 파서가 깨졌다.

검사 도구는 상태를 쓰지 않는 길로 관찰한다

검사 도구도 경쟁 상태(race condition)를 만든다. 「읽기」라고 믿은 호출이 실제로는 읽으면서 되쓰는 경로일 수 있다. 서버 값 확인은 부수 효과 없는 경로(DB 행 직접 읽기)로 하고, 고정 대기 대신 원하는 값이 될 때까지 폴링한다. 브라우저 자동화 도구의 조용한 기본값도 의심한다. 도구가 드러낸 경쟁 상태가 실제 사용자 경로에도 있는지는 따로 판정해 기록한다. → 4장 서버 엔지니어

사례 · 관찰이 만든 버그

무슨 일이 서버에 값이 반영됐는지 보려고 300ms마다 계정 읽기 API를 폴링했다. 그런데 그 API는 읽은 값을 정리해 다시 저장하는 경로여서, 그 사이에 도착한 클라이언트의 저장 요청을 덮어썼다. 값이 간헐적으로 저장되지 않았다. 또 브라우저 자동화 도구 Playwright는 `window.confirm` 대화창을 기본으로 자동 취소해서, 단추가 고장 난 것처럼 보였다. 반대로 대화창을 조용히 수락하도록 설정하자, 이번에는 교체 확인 팝업이 죽어도 스모크 테스트가 통과했다. **잡은 방법** 서버 값을 DB 행에서 날것으로 읽게 바꾸자 0.8~1.4초 안에 안정적으로 반영이 확인됐다(두 번 연속 완주). 브라우저 기본 대화창은 게임 안의 팝업으로 바꾸고, 「브라우저 대화창이 한 번도 뜨지 않는다」를 따로 검사한다.

작업 전후 점검표

- ☐ 새로 넣은 검사가 실패하는 상태를 직접 만들어, 정확히 그 자리에서 빨개지는 것을 보았는가?
- ☐ 기대값이 「기능이 없을 때 나오는 값」(기본값·상한·빈 값)과 다른가?
- ☐ 목록을 도는 검사가 「대상이 0개가 아니다」를 먼저 단언하는가? 기대값을 검사 대상 자신에서 뽑고 있지 않은가?
- ☐ 이 기능을 지우면 이 검사가 깨지는가? 화면 값을 화면 밖 원천(API·데이터)과 대조하는가?
- ☐ 검사가 겨냥한 갈래에 실제로 도달하는가? 도달하지 못하면 흉내 통로를 함께 만들었는가? 그 흉내는 사람이 만들 수 있는 상태인가?
- ☐ 건너뛴 검사의 수를 따로 세는가? 시드 데이터가 그 갈래를 실제로 그리는가?
- ☐ 검사가 우연한 순서·「첫 줄」·고정 id에 기대고 있지 않은가?
- ☐ 검사 코드에 연출 길이나 고정 대기 시간이 적혀 있지 않은가?
- ☐ 선택자가 화면 글자·순서·모양 클래스가 아니라 뜻 속성에 걸려 있는가? 자리를 실측 좌표로 보는가?
- ☐ 스모크 테스트를 끝까지 완주한 것을 보았는가? 테스트 합계가 이전보다 줄지 않았는가?
- ☐ 이 실패가 원래 있던 것인지 변경을 걸어 낸 기준선으로 확인했는가? 남긴 기존 실패에 담당자를 적었는가?
- ☐ 코드를 의심하기 전에 시작 상태·떠 있는 프로세스·에셋 로딩·검사 자신을 보았는가?
- ☐ 보고에 「확인하지 못한 것」 「흉내 통로로만 본 것」 「안 돌린 것」 「검사가 안 잡는 것」을 적었는가?
- ☐ 같은 정보(진도·수치·판정)가 두 곳에 적혀 있지 않은가? 앞 사람이 남긴 「없다」를 다시 찾아보았는가?
- ☐ 되돌리기·임시 보관 전에 커밋하지 않은 분량을 확인하고 범위를 파일로 좁혔는가? 비밀 파일을 통째로 출력하지 않았는가? 일괄 편집 뒤 diff가 의도한 줄뿐인가?

2 장

게임 디자이너

무엇을 만들지와 그 숫자를 정하는 사람

이 장은 규칙과 수치를 정하는 사람을 위한 것이다. 그 결정을 받아 구현하면서 기획자와 말을 주고받는 개발자도 읽어야 한다. 이 자리의 실수는 오류를 거의 내지 않는다. 문서의 숫자가 데이터와 갈라져도 게임은 멀쩡히 돈다. 보상 규칙에 난 우회로는 누가 찾기 전까지 아무 말도 하지 않는다. 진행 조건이 딱 막혀도 화면에는 「캐릭터가 모자란다」로만 보인다. 기획 결정은 하나씩 보면 옳은데 서로 만나서 틀리고, 그 틀림은 대개 플레이어가 먼저 찾는다. 아래 원칙은 그것을 결정하는 자리에서 미리 잡는 방법이다. 사례는 한 게임에서 나왔다. 체스 기물처럼 움직이는 캐릭터로 싸우고 행동 순서는 대기 시간으로 정해지는 턴제 전술 게임이다. 서버가 판정하는 온라인 대전, 가차와 캐릭터 육성, 열 개 언어를 갖췄다. 이 게임을 몰라도 같은 장면은 어느 게임에서나 만난다.

이 장의 원칙

1. 열린 물음은 모아서, 선택지와 파급을 붙여 한 번에 묻는다
2. 두 갈래로 읽히는 문장은 해석을 고르지 말고 묻고, 답은 기호가 아니라 뜻을 지킨다
3. 원안의 낱선 표기와 너무 깔끔한 해석은 무엇을 가정했는지 원작자에게 확인한다
4. 지정은 의도부터 맞추고, 목표점과 극단 사례에 넣어 계산해 본다
5. 기획 수치의 정본은 데이터 하나이고, 문서는 따라 적는 곳이다
6. 다른 표에서 나오는 숫자·가격·색은 옮겨 적지 말고 파생시킨다
7. 수치나 이름을 바꾸면 그것을 말하는 문장까지 바꾼다
8. 한 개념에는 이름 하나, 한 이름에는 뜻 하나를 둔다
9. 절차가 둘로 갈라지려 하면 되돌리는 범위를 넓혀 하나로 만든다
10. 파생 가능한 값은 복사해 두지 말고 원본을 참조한다
11. 효과의 범위는 예외 목록이 아니라 구조로 정한다
12. 큰 위험은 조건으로 관리하지 말고 빼되, 엔진 능력은 남겨 둔다
13. 시간 제한은 어느 시계에 거는지부터 정한다
14. 상수 하나보다 상수 사이의 관계가 규칙이다 — 불변식으로 적어 고정한다
15. 수량 제한·가격·효과 포화는 한 묶음이다
16. 문턱 규칙은 가장 작은 용량에서 시험한다
17. 패널티마다 「이것이 없으면 어떤 우회가 싸지는가」를 함께 적는다
18. 보상 차등은 고를 수 있는 변수에만 걸고, 공짜로 반복할 수 있는 길은 파밍된다고 가정한다
19. 화면에 보이는 지표에 넣은 것은 최적화와 악용의 대상이 된다
20. 서버가 검증할 수 없는 값은 나르지 말고, 그 값이 필요 없는 규칙으로 닫는다

21. 결과는 겨냥한 지표만이 아니라 그대로여야 할 것·단조여야 할 것으로 검산한다
22. 진행 조건은 성향별로 끝까지 밀어 교착과 역유인을 찾는다
23. 매칭 지표는 예상 승률 눈금으로 정의하고, 모든 상대를 같은 눈금으로 뽑는다
24. 상위 등급은 하위 등급의 효과를 포함하게 만든다
25. 보여 주기와 시험 편의를 위해 규칙과 측정 기준선을 건드리지 않는다
26. 놓치면 사라지는 선택과 저절로 벌어지는 일은 그 직전에 화면이 알린다
27. 아직 없는 것은 종류에 따라 숨기거나, 잠그거나, 자리를 그려 둔다
28. 「없음」도 값으로 보여 주되, 설명할 수 없거나 비교하면 안 되는 숫자는 내지 않는다
29. 화면마다 주 행동을 먼저 정하고, 그 행동의 자리를 UI가 덮지 않게 한다
30. 결말은 사람의 손이 내리게 하고, 써 봐서 버그처럼 보이면 설계를 다시 연다

2.1 기획 결정을 주고받는 법

기획 오류의 상당수는 결정 자체보다 결정이 오가는 길에서 생긴다. 묻는 쪽이 해석을 혼자 고르거나, 답을 글자 그대로 옮기거나, 지정을 계산해 보지 않고 받아들이면 틀린 규칙이 정상인 것처럼 구현된다.

열린 물음은 모아서, 선택지와 파급을 붙여 한 번에 묻는다

질문마다 「선택지 A와 B가 있고, 각각 무엇이 따라온다」를 붙인다. 파급이 보이면 결정권자는 그 자리에서 고른다. 질문을 모아서 올리면 맥락을 바꾸는 횟수가 줄고, 번호 매긴 표로 올리면 한 번에 확정된다. 확정된 내용은 설계 문서 한 곳에 모은다. 새 이름이 기존 규칙의 이름만 바꾼 것이라면 그렇다고 적어 둔다. 그래야 구현자가 규칙을 새로 만들지 않는다.

사례 · 기획 질의

무슨 일이 최상급 캐릭터 전용 스킬 30종에 사양이 비어 있는 곳이 많았다. 이를 질문 12개로 줄여서 물었다. 미결 셋(1:1 대전을 돌지, 레벨업 실패를 돌지, AI 대전 보상)도 선택지와 파급을 함께 보여 한 번에 정리됐다. 「이 캐릭터 이름을 하나로 통일한다」가 파일 이름만 바꾸라는 것인지 이름 표준까지 바꾸라는 것인지 해석이 갈린 적도 있다. 표준까지 바꾸면 내부 식별자도 바뀐다는 파급을 보여 주자 바로 결정됐다. **잡은 방법** 질문마다 「무엇이 따라오는가」를 붙였다.

사례 · UI 개편

무슨 일이 전투 화면을 개편하면서 열린 물음 12개(동점일 때 순서를 언제 추가하나, 이동 범위를 언제 보여 주나, 봉인 상태는 무슨 색인가, AI 대전에서도 남은 시간을 보여 주나 등)를 표로 올렸고 기획자가 한 번에 확정했다. 새로 생기는 [턴 가져오기] 단추는 새 규칙이 아니라고 적었다. 기존 「강제로 차례 넘기기」 규칙의 이름과 위치만 바꾼 것이다. **잡은 방법** 확정 사항을 설계 문서 한 곳에 모았다.

두 갈래로 읽히는 문장은 해석을 고르지 말고 묻고, 답은 기호가 아니라 뜻을 지킨다

서술형 규칙에서 순서·방향·포함 관계가 두 가지로 읽히면 구현하기 전에 묻는다. 답이 나오면 문서를 고쳐서 다음 사람이 같은 갈림길에 서지 않게 한다. 받은 답이 기존 구조와 어긋날 때도 있다. 그때는 답에 쓰인 기호와 답이 뜻하는 내용 중 어느 쪽이 결정의 본체인지 판단한다. 뜻을 지키고, 그렇게 판단했다고 기록한다.

사례 · 규칙 해석

무슨 일이 기획서에는 「이동한 뒤 다음 중 하나를 한다」 라고만 적혀 있었다. 그래서 전용 스킬도 그 선택지 가운데 하나로 구현했는데, 실제 뜻은 「차례 맨 앞에 따로 있는 단계」 였고 고쳤다. 행동 순서 막대가 「흰색에서 회색으로」 변한다는 문장도 두 방향으로 읽혀서 확인했다. 결론은 「다음이 누구냐」 를 한눈에 읽을 수 있는 「차오르는」 방향이었다. **잡은 방법** 해석을 고르지 않고 물었다.

사례 · 진영 표기

무슨 일이 「1P는 북쪽 진영(판 위쪽), 2P는 남쪽 진영(판 아래쪽)」 이라는 답을 받았다. 그런데 엔진은 2P를 위, 1P를 아래에 두고 있었다. 이 답에서 지킬 것은 「북쪽은 위, 남쪽은 아래」 라는 뜻이었다. 그래서 1P/2P와 진영의 대응만 뒤집었다. **왜 조용했나** 기호대로 따랐다면 화면의 카드 줄과 판 위 기물이 서로 다른 말을 했을 것이다. 눈에는 「내 편이 왜 위에 있지」 정도로만 보였을 것이다.

원안의 낯선 표기와 너무 깔끔한 해석은 무엇을 가정했는지 원작자에게 확인한다

기획 원안의 이상한 표기나 제약은 숨은 규칙의 흔적일 수 있다. 지우거나 단순화하기 전에 왜 그렇게 적었는지 묻는다. 반대로 해석이 너무 깔끔하게 맞아떨어질 때도 의심한다. 어떤 가정 위에서 그린 표는 그 가정대로 읽으면 모든 숫자가 맞는다. 그건 발견이 아니라 같은 말을 되풀이한 것이다. 결론을 회귀 테스트(고친 뒤에도 예전 동작이 그대로인지 다시 확인하는 자동 검사)로 고정할 때는 결과 숫자를 적지 않는다. 「규칙대로 끝까지 돌린 뒤 남는 것을 센다」 는 관계를 적는다. 그래야 상한이 바뀌어도 테스트의 숫자를 고칠 일이 없다.

사례 · 목표 해석

무슨 일이 목표에는 공격력이 2-4 처럼 범위로 적혀 있었다. 「왜 범위지?」 하고 넘긴 채 숫자 하나로 단순화했다. 그런데 그 범위는 「때릴 때마다 따로 내림한다」 는 규칙(2.5를 두 번 때리면 4)을 플레이어에게 보여 주려는 장치였다. **잡은 방법** 기획자가 되돌리라고 했다.

사례 · 기획 자료 해석

무슨 일이 기획 슬라이드에 있는 격자를 「건물 레벨별 해금 조건표」 로 읽었다. 그 해석으로 구현하고 기획서와 회귀 테스트까지 적었다. 「도시 레벨마다 정확히 3칸」 이 「증축할 때마다 건설 기회 3회」 와 맞물린다는 결론도 냈다. 실제로 그 격자는 기획자가 가상의 플레이어로 건물을 순서대로 지어 본 시뮬레이션이었다. **왜 조용했나** 표 자체가 「한 번에 3회씩 쓴다」 는 가정으로 그려져 있었다. 그러니 그렇게 읽으면 전부 맞았다. **잡은 방법** 기획자의 설명을 듣고 해석을 뒤집었다.

지정은 의도부터 맞추고, 목표점과 극단 사례에 넣어 계산해 본다

공식의 분포가 의도와 맞지 않아 보이면 숫자보다 의도를 먼저 묻는다. 분석을 돌리기 전에는 기준 시점·단위·포함 여부 같은 규칙 한 줄을 기획자와 맞춘다. 가정이 틀려도 표와 퍼센트는 그럴듯하게 나오기 때문이다. 지정을 받으면 거기 적힌 목표점을 전부 같은 식에 넣어 본다. 최대 레벨이나 최소 진행 같은 극단에서도 결과를 계산한다. 모순이나 우회로가 보이면 하나를 임의로 고르지 말고 구현과 함께 보고한다. 효과 지속시간을 언제부터 셀지(시전한 때인지 발동한 때인지) 맞추자 분석 결론이 「가장 아프다」 에서 「차이 없음」 으로 뒤집혔다. 연구 시설을 되돌리는 규칙을 극단에서 계산하자, 한 번도 연구하지 않은 레벨의 대기 시간까지 건너뛰는 길이 드러났다. 기획자가 범위를 좁혔다.

사례 · 확률 공식

무슨 일이 아군을 돕는 마법의 성공률은 $\text{clamp}(0, 100, \text{시전자 지능} + \text{대상 지능})$ 이었다. 이 공식을 두고 「지능이 중앙값(58)만 넘어도 거의 다 100%라 지능 높은 캐릭터를 우대하는 효과가 없다」고 지적했다. 그러자 기획자가 의도를 바로잡았다. 목적은 지능 높은 캐릭터를 우대하는 것이 아니었다. 지능 낮은 캐릭터를 돕는 마법에서 걸러내서, 힘만 센 캐릭터로 짠 조합을 줄이려는 것이었다. **잡은 방법** 수치를 지적하자 의도가 드러났다. 돕는 효과마저 50~60% 확률의 도박이면 아무도 쓰지 않는다. 대다수는 확실히 성공하고 걸러 낼 소수(하위 약 37%)만 실패하는 공식이 의도에 맞았다.

사례 · 대기 시간 설계

무슨 일이 부상 치료 시설에 「즉시 완치, 재사용 대기 5분」 안이 나왔다. 이 안을 목표 두 점(시설 1레벨에서 15분, 3레벨에서 1분 이내)에 함께 넣어 봤다. 즉시 완치면 1레벨 10분·3레벨 0분이고, 치료에 5분이 걸리면 1레벨 15분·3레벨 5분이었다. 어느 쪽도 두 점을 함께 맞추지 못했다. **왜 조용했나** 한 점씩만 보면 둘 다 말이 됐다. 즉시 완치였다면 3레벨에서 「치료하러 간다」는 행동이 화면에서 사라졌을 것이다. **잡은 방법** 모순을 보고하자 기획자가 「치료 1분, 재사용 대기 5분」으로 고쳤다. 1레벨 13분, 3레벨과 5레벨은 정확히 1분이 됐다.

2.2 숫자와 이름은 한 곳에서만 정한다

같은 숫자를 두 곳에 적으면 한쪽만 낚는다. 코드가 정본을 읽는 한 게임은 멀쩡히 돌아가므로, 낚은 쪽은 실행으로도 회귀 테스트로도 잡히지 않고 사람의 눈에만 걸린다.

기획 수치의 정본은 데이터 하나이고, 문서는 따라 적는 곳이다

수치의 정본(기준이 되는 원본)은 한 곳에만 둔다. 이 프로젝트에서는 엑셀에서 추출한 JSON이다. 기획서와 화면 문구는 그것을 따라 적거나 읽어 오는 곳이다. 수치가 의심스러우면 생성된 데이터를 보고, 어긋난 곳을 찾으면 문서를 고친다. 같은 종류의 어긋남이 두 번 나오면 규칙 문서에 따로 절을 만들어 다음 사람에게 남긴다.

사례 · 기획서와 데이터

무슨 일이 기획서에 적힌 레벨업 카드 누적량(116)과 건물 증축 자재 누적량(370)이 엑셀과 달랐다. 실제 값은 100과 225였다. 레벨업 화면에는 「공격력 +1」이 손으로 적혀 있었는데, 데이터와 엔진의 값은 0.5였다. 세 번째도 있었다. 기획서는 칸에 거는 지형 효과가 「1일마다」 정산된다고 적었지만, 엔진과 화면은 처음부터 0.9일이었다. **왜 조용했나** 코드는 처음부터 데이터를 읽고 있어서 동작에는 문제가 없었다. 그림 파일은 추출 도구가 대조해 주지만, 문서 본문과 화면 문구를 대조하는 장치는 없었다. **잡은 방법** 구현하던 사람이 문서와 코드를 직접 대조하다가 찾았다.

다른 표에서 나오는 숫자·가격·색은 옮겨 적지 말고 파생시킨다

다른 표에서 나오는 값은 「계수 × 기존 함수」로 적는다. 옮겨 적은 값은 지금은 출처와 같아서 문제가 없다. 출처가 바뀌는 날에야 한쪽만 낚는다. 이 원칙을 알아도 같은 주에 다시 어기게 되므로 코드 리뷰 항목으로 둔다.

사례 · 경제 상수

무슨 일이 가차 한 바퀴의 슬롯 수는 200장(최대 레벨까지 드는 카드 100장 × 2)이다. 이 200을 숫자로 적지 않고, 최대 레벨까지 드는 카드 수를 계산하는 기존 함수로 계산했다. 등급 색도 서술형 가이드를 보고 손으로 색상 코드를 적지 않았다. 연출 그림에서 채도가 가장 높은 픽셀을 찾아 거꾸로 뽑았다. 그런데 같은 주에 상점 화면에서는 육성 초기화 아이템의 가격을 숫자 10으로 박아 넣었다. **잡은 방법** 리뷰하다가 스스로 알아챘다. 가격 상수를 불러 쓰게 고치고, 상수를 참조하는 다른 화면 값도 전부 다시 확인했다.

사례 · 가격 책정

무슨 일이 「자재 10개에 유료 화폐 10」이라는 가격을 따로 적지 않았다. 「증축 1단계에 드는 자재 수 × 1」로 계산되게 했다. 「10개 묶음」이라는 상수를 따로 두는 안은 기각했다. 증축 표가 바뀌면 그런 상수만 낡는다.

수치나 이름을 바꾸면 그것을 말하는 문장까지 바꾼다

숫자는 표에 있고, 그 숫자를 설명하는 문장은 다른 파일에 있다. 수치를 고치면 설명문·키 이름·용어표도 같이 고친다. 데이터를 추출하는 단계에서 효과 값 표와 설명 문장을 서로 대조하고, 어긋나면 추출을 멈춘다. 원본 파일이 읽기 전용이면 정정 목록을 따로 둔다. 그리고 원본이 갱신되면 「이 정정은 이제 필요 없다」고 스스로 알리게 만든다. 공식을 설명하는 안내문은 코드에 정의된 공식 항목 목록에서 옮겨 쓴다. 이름을 바꿀 때는 이름 칸만 보지 말고, 모든 언어의 본문에서 옛 이름을 검색한다. 가능하면 본문 안의 상호 참조를 키 치환(`{skill:id}` 같은 자리 표시)으로 만든다. 여러 언어의 번역자가 서로 모르는 채 같은 원문을 의심하면, 원문에 오타자가 있다는 신호다. 실제로 기술 이름의 한자 한 글자(이빨 齒)가 그렇게 해서 맞는 글자(말 탈 騎)로 고쳐졌다.

사례 · 밸런스 패치

무슨 일이 상대의 행동 자원을 깎는 스킬의 효과를 1에서 4로 올렸다. 그런데 설명문은 읽기 전용인 엑셀과 다국어 CSV에 있었고, 열 개 언어 모두 「1」로 남았다. 화면은 「1 내린다」고 말하면서 실제로는 4를 깎았다. **왜 조용했나** 실행해도, 회귀 테스트를 돌려도 효과는 맞았다. 데이터 안에서 효과와 설명만 서로 갈라져 있었다. **잡은 방법** UI 스모크 테스트(실제 화면을 띄워 끝까지 눌러 보는 자동 검사)가 화면에 실제로 그려진 글자를 읽어서 잡았다.

사례 · 다국어

무슨 일이 영어 화면의 마법 이름을 음차(한국어 발음을 로마자로 적은 것)에서 뜻 번역으로 바꿨다. 그런데 효과 설명 세 개가 다른 마법을 옛 음차로 부르고 있었다(`created by Hwa-gye` 같은 식). 여섯 언어 × 3종으로 18건이었다. 부대 편성 화면의 강함 지표 안내문은 「구성 유닛 종류와 레벨 조합에 따라 정해진다」였다. 하지만 유닛 종류는 한 달 전에 일부러 지표에서 뺐다. **왜 조용했나** 이름 칸만 보면 전부 바뀌어 있었다. 안내문도 글자에는 아무 문제가 없었고 사실과 다를 뿐이었다.

한 개념에는 이름 하나, 한 이름에는 뜻 하나를 둔다

한 개념에 이름이 둘이면 화면에 이미 박힌 라벨 쪽으로 맞춘다. 엔진 내부 단위는 공식의 근거를 보여 줘야 하는 곳에만 남긴다. 단위 변환은 데이터 파이프라인의 한 지점에서만 한다. 반대로 코드에서 이미 뜻이 정해진 이름을 다른 개념에 다시 쓰지 않는다. 원본 이름을 바꿀 수 없으면 코드 쪽 이름을 갈라 쓰고, 「이 이름은 언제나 이 뜻」이라고 문서에 적는다. 번역 용어집은 게임 화면에서 이미 쓰는 낱말에 묶는다. 외국어 표기는 두 규약으로 나눈다. 서사가 있는 고유명사(필살기 이름)는 원어 읽기를 살린다. 기능을 알아야 하는 용어(일반 마법)는 뜻으로

번역하고, 인물이 아니라 효과에 맞춰 짓는다. 「○○'s Charm」 대신 Charm으로 쓰는 식이다. 자동으로 만드는 문장의 띄어쓰기도 어미별 목록으로 갈라 적는다. 한데 묶어 처리하면 1.9일동안 같은 문장이 나온다.

사례 · 용어 통일

무슨 일이 플레이어에게 보이는 시간 단위는 전부 「일(日)」로 통일했다. 설명문에 쓰던 waiting time (유닛이 다시 행동하기까지 남은 시간)은 화면 막대의 라벨과 같은 wt로 바꿨다. 「대기시간」이라는 이름은 라벨과 어긋나서 기각했다. $WT = 190 - \text{통솔력}$ 표처럼 엔진 단위로 적어야 근거가 보이는 곳은 그대로 뒀다. 변환은 추출 도구가 출력하기 직전 한 곳에서만 했다.

사례 · 이름 충돌

무슨 일이 엑셀 시트 이름을 따라 화면에 겹쳐 그리는 그림 층을 「오라(aura)」라고 불렀다. 그러자 엔진의 「오라」와 섞였다. 엔진의 오라는 특정 캐릭터 주변 환경을 판정하는 기능이다. 「최상급 캐릭터의 오라」라고 하면 시전자에게 붙는 표식인지, 환경 안의 적이 보는 그림인지 구별할 수 없었다(실제로 두 그림은 다르다). **잡은 방법** 엑셀 이름은 그대로 두고, 코드에서 그림 층을 「시각 효과」라는 다른 이름으로 갈랐다. 같은 이유로 칸에 붙는 지형 그림과 유닛에 붙는 효과 링도 다른 층에 둔다.

2.3 중간 상태와 예외를 만들지 않는 규칙

규칙의 비용은 규칙 문장의 길이가 아니라 그 규칙이 만들어 내는 상태와 예외의 수로 정해진다. 원래 없던 상태를 만드는 규칙은 저장·검증·화면·마이그레이션(옛 저장 형식을 새 형식으로 옮기는 일)에 모두 흔적을 남긴다. 그중 한 곳만 빠뜨려도 깨진다.

절차가 둘로 갈라지려 하면 되돌리는 범위를 넓혀 하나로 만든다

일부만 되돌리는 절차는 전부 되돌리는 절차보다 언제나 비싸다. 되돌린 부분과 남은 부분의 경계를 사람이 이해해야 하기 때문이다. 되돌린 뒤에는 기존 절차를 그대로 다시 밟게 만든다. 값을 치르는 자리는 하나로 두고, 돌려주는 양은 쓴 양과 정확히 같게 한다. 모든 소비처가 알아야 하는 중간 상태(잡아 두기, 고르는 도중)는 한 곳만 빠뜨려도 깨지므로 피한다. 규칙을 먼저 굳힌 다음 목업을 읽는다. 목업의 한 줄이 「새로 만들라」는 뜻인지 「있는 것을 쓰라」는 뜻인지는 규칙이 정해져야 가릴 수 있다.

사례 · 육성 초기화

무슨 일이 육성 초기화 아이템은 캐릭터의 성장 선택을 다시 하게 해 주는 유료 아이템이다. 원안은 「레벨은 그대로 두고 능력만 다시 고르기」였다. 구현해 보니 따로 만들 것이 줄줄이 생겼다. 순서대로 다시 고르는 마법사 화면, 한 걸음 무르기, 어디에도 없던 「작성 중」 상태, 그 상태를 저장하면 안 되는 이유, 선택 검증이 하나씩 더 필요했다. 「7레벨인데 아무것도 고르지 않은 상태」라는, 원래 게임에 없는 상태가 생긴 것이다. **잡은 방법** 「쓴 카드를 전부 돌려주고 1레벨로 되감기」로 바꿨다. 그러자 그 뒤는 평범한 레벨업 절차가 됐다. 무승부 보상 중 하나를 고르는 화면도 「고른 뒤에야 반영」하게 해서 「고르는 도중」이라는 상태를 없앴다.

사례 · 유료 화폐

무슨 일이 유료 화폐를 「돈으로 산 것」과 「무료로 받은 것」으로 나누면 문제가 생긴다. 환불처럼 화폐를 되돌리는 모든 길이 「어느 쪽에서 낸 화폐인가」를 따라가야 한다. 「마지막 1분에 결제한다」는 안은 그동안 화폐를 잡아 두는 예치 상태가 필요했다. 그러면 화폐를 쓰는 일곱 곳 이상이 모두 예치액을 빼고 계산해야 했다. **잡은 방법** 코드를 훑어 결제 말고 화폐가 늘어나는 곳을 찾았다. 개발용 지급과 제작 주문 취소 환불, 두 곳이었다. 보상은 유료 화폐가 아닌 것으로만 주기로 했다. 제작 시설은 「확정하면 바로 결제, 환불 없음, 확정 전에 한 번 묻기」로 바꿨다. 이제 화폐가 생기는 길은 결제 하나, 되돌아오는 길은 0이다.

파생 가능한 값은 복사해 두지 말고 원본을 참조한다

원본에서 복사해 둔 값은 원본이 바뀌면 낡는다. 사용자는 그것을 버그로 읽는다. 복사본이 주던 기능이 필요하면 원본 쪽의 다른 자리로 옮긴다. 옛 기능을 되살리는 방식으로 풀지 않는다.

사례 · 데이터 모델

무슨 일이 부대가 캐릭터의 레벨을 따로 들고 있게 했다. 부대마다 레벨을 낮춰 출전할 수 있게 하려는 것이었다. 그러자 캐릭터가 레벨업해도 부대에는 옛 레벨이 남아서, 부대마다 손으로 올려 줘야 했다. 사용자는 「레벨업이 반영되지 않는다」는 버그로 읽었다. **잡은 방법** 부대는 「어느 캐릭터가 속해 있는가」만 들도록 뒤집었다. 「일부러 약하게 나가기」는 캐릭터 단위의 육성 초기화로 옮겼다. 그 대가로 그 캐릭터가 속한 모든 부대가 함께 약해진다.

효과의 범위는 예외 목록이 아니라 구조로 정한다

효과를 엔진이 이미 갖고 있는 분류(공격 피해, 시전 중 상태 등)에 맞춰 정의하면 새 콘텐츠가 자동으로 따라온다. 예외를 이름으로 나열하면 새 항목이 생길 때마다 조용히 빠진다. 「나머지 전부」같은 지정은 목록이 아니라 기본값 규칙으로 옮기고, 처음 보는 입력으로 시험한다. 서로 배타적이어야 하는 규칙은 방향마다 하나씩 막지 않는다. 모든 참여자가 같은 조건 하나를 쓰게 한다. 대상에 따라 뜻이 달라져서 화면이 경우마다 다르게 설명해야 하는 효과는 들이지 않는다.

사례 · 아이템·스킬 설계

무슨 일이 「상대의 피해를 모두 0으로 만드는 아이템」을 짜려니 즉사 기술과 특수 기술을 예외로 문자열마다 손으로 적어야 했다. 새 즉사 기술이 생길 때마다 조용히 뚫리는 구조였다. 한 캐릭터의 스킬에는 「이미 걸린 효과를 지운다」를 넣으려 했다. 그런데 엔진은 상태가 어디서 왔는지 기록하지 않았고, 기술 효과는 여러 유닛과 지형, 예약된 효과에 흩어져 있었다. 그래서 「지운다」의 뜻이 기술마다 달라졌다. **잡은 방법** 아이템은 「공격 피해만 0」으로, 스킬은 「대상이 시전 중이면 무산시킨다」로 바꿨다. AI도 고칠 필요가 없었다.

사례 · 기본값 규칙

무슨 일이 배경음악 지정은 「아래 네 화면이 아니면 메인곡」이었다. 이것을 화면 목록이 아니라 「모르는 화면이면 메인곡」이라는 규칙으로 구현했다. 그리고 'shop'이나 빈 문자열처럼 없는 화면 이름으로 시험했다. 칸에 거는 지형 마법에서는 질문 하나로 비대칭이 드러났다. 불 지형만 다른 지형 칸을 덮을 수 있었다. 반대 방향도 막아 달라는 요청이 이어졌다. **잡은 방법** 「지형이 없는 칸에만」이라는 필터 하나를 세 효과에 똑같이 걸고, 양방향 회귀 테스트를 넣었다. 자기 기술끼리도 서로 막히는 부작용이 생겼지만 대칭인 규칙 하나가 더 단순했다.

큰 위험은 조건으로 관리하지 말고 빼되, 엔진 능력은 남겨 둔다

강한 효과를 넣을 때는 승리·패배 조건과 겹치는 조합부터 뺀다. 승부가 나지 않는 판 같은 큰 위험은 조건을 붙여 관리하기보다 아예 빼는 편이 싸다. 그러면 달려 있던 예외 처리까지 함께 사라진다. 다만 엔진의 기반 능력은 남겨 두어, 다시 필요할 때 되살릴 자리를 둔다. 같은 대상을 건드리는 효과끼리는 어느 쪽이 덮어쓰는지 우선순위를 정한다. 비용이 큰 쪽을 보호한다. 「어느 편 소속인가」와 「누가 조종하는가」를 나눠 정의하면 승패 판정이 단순해진다.

사례 · 승패 조건

무슨 일이 오래가는 무적 스킬을 가진 캐릭터는 「잡히면 지는 기물」 자리에 둘 수 없게 했다. 거기에 두면 아무도 그 기물을 잡을 수 없다. 같은 이유로 버그 하나를 막았다. 싼 일시 조종 효과가 비싼 영구 조종 효과를 덮어써서 날려 버리는 버그였다. 영구 조종은 소속을 바꾸는 것이 아니라 조종권만 넘기는 것으로 정의했다. 승패는 원래 편으로 썼다.

사례 · 콘텐츠 삭제

무슨 일이 칸에 거는 지형 마법은 겨울 상대가 없어서 성공률이 $\text{지능} \times 2$ 였다. 지능 50이면 100%다. 진입할 수 없는 칸을 만드는 마법을 여러 번 쓰면 승부가 나지 않는 판이 생길 수 있었다. **잡은 방법** 이 마법과 짝 마법을 조건으로 묶지 않고 삭제했다. 그러자 「특정 레벨에서만 한 쌍으로 배운다」는 예외도 같이 사라졌다. 그 예외는 저장 형식, 마이그레이션, 검증, 레벨업 화면 네 곳에 흔적을 남기고 있었다. 엔진의 「진입할 수 없는 지형」 판정은 남겼다. 대상이 없는 효과에는 「대상과 비교하는」 공식 대신 별도 공식($\text{내 지능} - 10$)을 뒀다.

2.4 숫자는 혼자 틀리지 않는다 — 시계·관계·경계

상수 하나하나가 그 자체로 옳을 때가 많다. 틀리는 곳은 둘이 만나는 자리, 다른 시계로 옮겨 가는 자리, 그리고 가장 작은 경우다. 이 셋은 값만 들여다봐서는 보이지 않고 시나리오로 굴러 봐야 보인다.

시간 제한은 어느 시계에 거는지부터 정한다

판 길이 제한을 어느 시계에 거느냐가 플레이 스타일 사이의 공정성을 정한다. 「생각하는 시간」과 「게임 진행」을 분리한 설계라면 제한도 게임 시계에 건다. 게임 내 시간 환산을 다른 화면으로 가져가기 전에는 그 화면에서 그 시계가 실제로 흐르는지 본다. 기간 값은 그 기간 동안 사용자가 해야 하는 행동(취소, 확인)을 할 수 있는지로 검증한다.

사례 · 판 길이 제한

무슨 일이 이 게임은 유닛이 행동하는 동안 게임 시계가 멈춘다. 무승부 상한은 실제 시간이 아니라 게임 내 시간에 걸었다. 그 시간이 다 되면 남은 체력 합으로 판정승을 준다. 실제 시간에 거는 안은 오래 생각하는 플레이어만 손해를 봐서 기각했다. 동점처럼 아주 드문 일은 실측 빈도(0.02%)를 보고 얼마나 공들여 처리할지 정했다.

사례 · 시간 단위

무슨 일이 장비 제작 기간을 「실제 n주」로 정했는데, 기획자가 「게임 내 1주」를 제안했다. 게임에서는 1초가 1일이니 7초다. 그런데 그 환산은 전투 시계의 것이고, 도시 화면에서는 그 시계가 흐르지 않는다. 실제 시계로 옮기면 1레벨 제작이 7초라서 기다릴 일이 없고, [주문 취소]를 누를 틈도 없었다. **잡은 방법** 각 안대로라면 화면에서 무슨 일이 일어나는지 하나씩 짚어 보고 「실제 n분」으로 정했다.

상수 하나보다 상수 사이의 관계가 규칙이다 — 불변식으로 적어 고정한다

시간 제한이 여럿이면 각 값보다 값 사이의 관계가 규칙이다. 숫자를 넣고 타임라인을 직접 굴러 본다. 그 관계를 코드의 불변식(언제나 참이어야 하는 조건)으로 적고 테스트로 묶는다. 그러면 하나만 바뀌도 테스트가 깨진다. 한 지점의 어색함을 고칠 때도 먼저 그 값이 들어가는 불변식(총량, 남는 칸)을 적는다. 그리고 불변식을 지키면서 돌릴 수 있는 다른 손잡이(상한, 시작점)를 찾는다. 시간이 지나면 차오르는 자원의 자투리·상한·뒤로 간 시계 처리도 이와 같은 구현 문제다 → 3장 시스템 엔지니어.

사례 · 온라인 대전 타이머

무슨 일이 온라인 대전의 재접속 유예 60초는 그것만 보면 옳았다. 문제는 「배치 30초가 지나면 자동으로 준비 완료」 규칙과 만날 때였다. 배치 시작 직후 끊긴 사람이 30초에 자동으로 준비 완료되고, 유예가 끝나는 60초에는 이미 다음 단계(정찰)에 들어가 있었다. 그래서 「배치 중 이탈」 규칙은 영원히 발동하지 않았다. **왜 조용했나** 도달할 수 없는 분기가 코드가 아니라 사양에서 나왔다. **잡은 방법** 「끊긴 사람에게는 자동 준비를 걸지 않는다」는 규칙을 더했다. 그리고 유예 = 상대 수락 대기, 유예 ≤ 한 차례 제한 × 3, 방 유희 제한 > 유예를 불변식 상수로 적어 회귀 테스트로 고정했다.

사례 · 총량 불변식

무슨 일이 「도시 1레벨에 건설 기회가 3회 있는데 지을 수 있는 건물이 없다」를 고치려 했다. 시작할 때 주는 3회를 없애면 전체 기회가 27에서 24로 줄고, 「최종 건물이 다섯 칸을 연다」가 여덟 칸이 된다. **잡은 방법** 대신 최종 건물의 레벨 상한을 한 단계 올려 증축 횟수(9)를 유지했다. 27과 5가 흔들리지 않았다.

수량 제한·가격·효과 포화는 한 묶음이다

가격은 어떤 총량 제한을 전제로 매겨진다. 그 제한을 풀 때는 가격과 효과 포화도 같은 전제 위에 있었는지 확인한다. 셋 중 하나만 움직이면 나머지 둘이 조용히 틀린다.

사례 · 상점 경제

무슨 일이 장비 제작 수량 제한을 없애자는 안이 나왔다. 그런데 가격은 「종류당 1개」를 전제로 같은 효과의 카드보다 3.2배 싸게 매겨져 있었다. 효과도 한 판에 5개를 넘으면 더 쌓이지 않았다. **잡은 방법** 무제한 안은 기각했다. 대신 「종류당 최대 개수 = 시설 레벨 - 해금 레벨 + 1」인 피라미드 구조로 바꿨다. 한 계정의 매출 상한은 750에서 1,495로 늘었고, 강함의 천장은 그대로였다.

문턱 규칙은 가장 작은 용량에서 시험한다

용량이나 묶음 크기에 걸린 조건은 용량이 가장 작은 경우에 실제로 통과할 수 있는지 본다. 「통째로 들어갈 때만」 같은 조건은 작은 용량에서 영원히 막힌다. 약간의 손해를 받아들이는 완화가 낫다면 그 손해가 최대 얼마인지 적어 둔다.

사례 · 구매 조건

무슨 일이 대전 입장에 드는 소모 자원을 사는 조건을 「한 묶음(20개)이 통째로 들어갈 때만」으로 짰다. 그러자 보관 상한이 20인 최저 레벨 창고는 자원이 0일 때만 살 수 있었다. **잡은 방법** 최저 레벨의 상한으로 계산해 보고 「빈자리가 16 미만이면 막는다」로 풀었다. 최대 4개가 넘쳐 버려지는 것은 받아들인 손해로 적었다. 회귀 테스트가 「문턱 미만이면 막고, 딱 문턱이면 사서 상한에 닿는다」와 문턱 < 묶음 크기 를 고정한다.

2.5 우회로가 정식 통로보다 싸지 않게 — 보상·패널티·치팅

현금 결제와 랭킹이 있는 게임에서 보상 규칙은 플레이어가 최적화하는 대상이다. 비정상 통로의 비용은 언제나 정식 통로보다 크게 설계해야 한다. 이런 공백은 화면에 아무 표시도 남기지 않으므로, 찾으려는 사람이 먼저 찾는다.

패널티마다 「이것이 없으면 어떤 우회가 싸지는가」를 함께 적는다

패널티를 문서에 적을 때는 「무엇을 지키는가」와 「무엇을 못 하게 하는가」를 따로 적는다. 「지킨다」만 적어 두면 언젠가 「너무 박하다」는 이유로 되돌려지고, 그 패널티가 막던 우회가 조용히 살아난다.

사례 · 이탈 패널티

무슨 일이 「배치 중에 사라진 쪽에게는 참가비를 돌려주지 않는다」는 규칙의 이유를 처음에는 「남은 쪽을 지킨다」로만 적었다. 기획자가 핵심을 짚었다. 매칭된 상대를 물리는 정식 통로인 「다시 찾기」는 참가 자원 1개를 낸다. 그런데 참가비를 낸 뒤 접속을 끊어서 돌려받을 수 있다면 같은 회피가 공짜가 되고, 거절 패널티는 통째로 무력해진다. 같은 이유로 배치 중 이탈이 나면 남은 쪽에게도 승리를 주지 않는다. 승리를 주면 「상대를 끊게 만드는 것」이 이득이 된다. 양쪽 다 끊기면 지킬 사람이 없으므로 양쪽 모두 환불한다. **왜 조용했나** 「지킨다」만 적힌 규칙은 되돌려도 아무것도 깨지지 않는 것처럼 보인다.

보상 차등은 고를 수 있는 변수에만 걸고, 공짜로 반복할 수 있는 길은 파밍된다고 가정한다

보상을 가르는 변수는 플레이어가 고르거나 실력으로 정해지는 것이어야 한다. 게임에 들어가는 흐름이 바뀌면, 그 흐름이 보장하던 「선택」에 기대고 있던 규칙을 다시 본다. 보상 경로마다 자동화로 파밍될 수 있는지 본다. 비용 없이 반복할 수 있는 입력이 결과를 정한다는 사실은 화면에 드러내지 않는다. 결정성(같은 입력이면 늘 같은 결과) 자체는 QA와 재현을 위해 남긴다. 성장 경로에 확률형 실패를 두면 플레이어의 불만과 법적 확률 공시 부담이 함께 생긴다. 이 프로젝트는 레벨업 실패를 없애 모든 레벨을 100% 성공으로 만들고, 난이도는 필요한 카드 수로만 조절했다.

사례 · AI 대전 보상

무슨 일이 「AI 대전은 소모 자원만 주고 전적과 카드는 주지 않는다」가 정해져 있었다. 봇으로 파밍하는 것을 막기 위해서였다. 그런데 출전하는 문이 [출정하기] 하나로 합쳐졌다. 30초 동안 온라인 상대를 찾고, 못 찾으면 AI와 붙는다. 이제 상대가 사람인지는 플레이어가 고르는 것이 아니라 「그때 접속한 사람이 있었는가」로 정해진다. 보상이 갈리면 그것을 정하는 것은 시간대가 된다. **잡은 방법** 문을 합치는 결정과 함께 보상 규칙을 다시 따졌고, 보상과 전적을 똑같이 주기로 했다. 이력의 [AI] 표시와 「사람과의 전적만 보기」 필터는 살렸다. 한산한 시간대에 카드가 쉽게 모일 수 있는 위험은 나중에 손덜 곳을 상수 하나로 모아 뒀다.

사례 · 결정적 시드

무슨 일이 「같은 도시 이름으로 시작하면 늘 같은 캐릭터 다섯 명을 받는다」는 안내 문구를 열 개 언어로 번역해 넣었다. 그 뒤에야 문제를 알아챘다. 계정 생성은 무료라서, 이 문구는 「좋은 이름을 찾을 때까지 계정을 만들라」는 파밍 안내였다. **잡은 방법** 사용자 질문에 답하다 발견했다. 문구만 지우고, 결정적으로 정해지는 로직은 QA용으로 남겼다.

화면에 보이는 지표에 넣은 것은 최적화와 악용의 대상이 된다

지표에 어떤 요소를 넣으면 플레이어는 그 지표를 움직이려고 그 요소를 고르거나 뺀다. 전술적 선택은 강함 지표에 넣지 않는다. 약자가 이겼을 때 주는 보상은 담합으로 꾸며 낼 수 있다고 보고 상한을 둔다. 보상을 명예(기록)로 주면 담합해도 얻을 것이 없다. 다만 그 기록을 랭킹 정렬 기준으로 쓰면 담합으로 1위를 만들 수 있으므로, 「보여 주기」와 「순위에 반영하기」를 가른다. 일시적 약화는 매칭 점수가 보지 않게 한다. 그래야 약해진 채로 출전하는 것이 순수한 손해가 된다.

사례 · 매칭 지표 설계

무슨 일이 매칭에 쓰는 강함 지표를 설계하면서 기획자와 의견이 네 군데서 갈렸다. 첫째, 「강함이 비슷한 상대와 붙으니 일부러 지는 계정은 스스로 망한다」며 카드 보상 상한을 한 등급 올리자는 안이 있었다. 기획자는 악성 유저끼리 짜고 큰 이벤트를 만들어 한 계정에 몰아줄 수 있다며 막았다. 둘째, 예상 승률에 따라 카드 등급을 나누자는 안은 「상한이 곧 파밍 목표가 된다」며 전적·랭킹 기록으로 바꿨다. 셋째, 유닛 종류를 지표에 넣자는 안은 뺐다. 기획자의 이유는 「사람이 그 숫자를 보고 유닛을 고르거나 뺀다. 유닛 선택은 전술이다」였다.

사례 · 상태 효과와 매칭

무슨 일이 부상(능력치 -10)을 출전 명단에서 미리 깎아 넘기면, 전투 화면이 「왜 약한지」를 설명할 수 없다. 반대로 매칭 지표에 부상을 반영하면 일부러 부상당한 채 나가 약한 상대와 붙는 길이 열린다. **잡은 방법** 출전 명단에는 「부상」 표시만 싣고, 실제 감소는 전투를 만들 때 적용했다. 매칭 지표는 온전한 능력치로 썼다. 두 경로를 모두 회귀 테스트로 고정했다.

서버가 검증할 수 없는 값은 나르지 말고, 그 값이 필요 없는 규칙으로 닫는다

치팅 공백을 메우는 코드보다 공백이 없는 규칙이 싸다. 소모와 지급은 서버가 권위 있게 알고 있는 순간에 일어나게 설계한다. 서버 권위란 결과를 서버가 정하고 클라이언트의 주장은 믿지 않는다는 뜻이다. 플레이어에게 불리한 일은 서버가 처리하는 지점, 즉 플레이어가 피할 수 없는 지점에서 일으킨다. 이탈·재시도·자원 소진 같은 출구마다 손해의 크기를 맞춰서 결과를 골라 받을 수 없게 한다. 화면이 허용하는 순서와 서버 규칙이 허용하는 순서도 같게 한다. 서버 규칙이 더 느슨하면 그 차이가 바로 치팅 표면이다. 실제로 연구 화면은 다음 레벨 하나만 보여 줬는데 서버 규칙은 아무 레벨이나 받아서, API를 직접 부르면 단계를 건너뛸 수 있었다. 구현 쪽 경계는
→ 4장 서버 엔지니어.

사례 · 아이템 소모

무슨 일이 「쓰지 않은 소모형 아이템은 돌려준다」를 지키려면 「누가 썼는가」를 정산할 때까지 날라야 했다.

그런데 무승부 보상을 고르는 경로는 판을 판정하던 쪽이 사라진 뒤에 들어와서, 서버가 그 값을 검증할 수 없었다.

「안 썼다」고 대면 쓴 아이템을 돌려받는다. **잡은 방법** 기획자와 규칙을 「참전하면 소모된다」로 바꿨다. 그 값을 나를 이유가 통째로 사라졌고, 정산 경로 세 곳은 한 줄도 바뀌지 않았다. 「판이 끝난 뒤 소모」였다면 같은 아이템을 여러 판에 쓸 수 있었을 것이다.

사례 · 불리한 이벤트

무슨 일이 NPC 습격(도시 자원을 약탈해 가는 방어전)을 클라이언트의 「습격을 봤다」 신고로 일으키면, 신고하지 않는 클라이언트는 영원히 약탈당하지 않는다. 같은 시드로 다시 들어갈 수 있으면 이길 때까지 되풀이할 수도 있었다. **잡은 방법** 습격은 서버가 계정을 읽는 순간에 일으켰다. 약탈량은 습격이 생길 때의 자원으로 정해져 $\min(\text{지금 자원}, \dots)$ 으로 상한을 뒤서, 미리 써 버려 피하는 길을 막았다. 항복의 손해를 패배보다 크거나 같게 해서 지는 판을 끊을 이유를 없앴다. 끊긴 판에는 다시 들어갈 수 없고, 시간이 지나면 서버가 항복으로 정산한다. 회귀 테스트가 「항복 손해 $\geq$ 패배 손해」를 전 구간에서 훑어 고정한다.

2.6 밸런스과 경제는 시뮬레이션으로 끝까지 민다

밸런스 수치와 진행 조건은 표로 보면 멀쩡하다. 문제는 특정 전략, 특정 성향, 특정 구간에서만 드러난다.

시뮬레이션은 언제나 그럴듯한 숫자를 내놓는다. 그러니 숫자를 믿기 전에 그 숫자를 검산할 방법부터 정해야 한다.

결과는 겨냥한 지표만이 아니라 그대로여야 할 것·단조여야 할 것으로 검산한다

조정 전후로 대조군 전략을 함께 켜다. 겨냥한 지표가 움직였는지와 함께 건드리지 않으려던 지표가 그대로인지 본다. 극단 사례의 문제는 일반 조정과 따로 다룬다. 경제 시뮬레이션이 이상한 숫자를 내면 규칙보다 「플레이어 전략」 모델을 먼저 의심한다. 「돈을 더 쓰면 더 빨리 크다」처럼 한 방향으로만 움직여야 하는 성질(단조성)이 지켜지는지는 기획자에게 숫자를 넘기기 전에 확인한다. 측정 자체의 함정은 3장에서 다룬다. 소수점까지 같은 값은 기능이 안 돌아갔다는 신호이고, 회귀 계수는 표준오차와 함께 읽어야 한다 → 3장 시스템 엔지니어.

사례 · 밸런스 패치

무슨 일이 레벨업 때마다 공격력을 1씩 올리게 하면 공격력에만 몰아 키우는 전략이 등급 사이의 승률 차를 10.5%p에서 3.0%p로 무너뜨렸다. 등급 설계의 근거가 사라지는 셈이다. 0.5로 낮추자 몰아 키우기는 6.3%p로 돌아왔고, 고르게 키우는 전략은 10.5%p로 변하지 않았다. 의도한 곳만 늘린 것이다. 능력치가 모두 1인 최약체 캐릭터의 승률 급락(44.2% → 30.8%)은 이 조정에 섞이지 않고 따로 검토하기로 땀다. **잡은 방법** 성장 전략별(몰아 키우기·고르게 키우기)로 시뮬레이션을 나란히 돌려 비교했다.

사례 · 경제 시뮬레이션

무슨 일이 카드 교환 기능(같은 등급 카드 3장을 1장으로 바꾸기)의 효과를 재는데, 모델 전략을 「등급마다 한 명씩 키우기」로 두자 기능이 있을 때가 없을 때보다 느리게 나왔다. 「전체 상위 다섯 명 키우기」로 두자 중간 과금이 최소 과금보다 느렸다. 둘 다 모델 전략이 카드가 없는 등급에 육성 자리를 준 탓이었다. **잡은 방법** 전략을 「하위 등급 키우기」와 「상위 등급 키우기」로 나누자 규칙 자체의 성질이 드러났다. 하위 등급은 전투로, 상위 등급은 가차로 크다.

진행 조건은 성향별로 끝까지 밀어 교착과 역유인을 찾는다

「보유 수」를 요구하는 진행 조건은 넣기 전에 플레이 성향 몇 가지로 계정을 끝까지 키워 본다. 조건이 요구하는 수를 늘리는 통로가 바로 그 조건 뒤에 잠겨 있지 않은지 보고, 같은 셈을 쓰는 다른 판정도 함께 맞춘다. 성장 조건을 플레이어가 조절할 수 있는 다른 값(시설 레벨 등)에 묶으면, 「일부러 안 하는 것」이 이득이 되는 역유인이 생긴다. 단계별 필요량을 표로 펼쳐 보고 갑자기 튀는 곳이 없는지 눈으로 확인한다.

사례 · 진행 교착

무슨 일이 도시 증축에 보유 캐릭터 수 조건이 붙었다. 이때 활성 명단만 세면 진행이 영원히 멈췄다. 명단이 하위 등급으로 차면 그 뒤 뽑은 상위 등급은 보관함으로 빠진다. 그런데 명단 크기를 늘릴 건설 기회는 증축을 해야 나온다. 시뮬레이션에서 전투를 먼저 하는 계정은 100%, 가차를 먼저 하는 계정은 30%, 자재를 사는 계정은 13%가 멈췄다. 기획자가 낸 「명단 시설을 강제로 먼저 올리게 하는」 안으로도 30%가 남았다. 「보유 = 명단 + 보관함」으로 세자 셋 다 0%가 됐다. **왜 조용했나** 화면에는 「캐릭터가 모자란다」로만 보였다. 정상적으로 진행이 막힌 것과 구별되지 않는다. **잡은 방법** 성향 세 가지로 4,000번씩 시뮬레이션을 돌렸다. 같은 셈을 쓰는 최종 목표 판정도 함께 맞췄다.

사례 · 역유인

무슨 일이 증축 조건 1안은 「명단 시설 수용 인원의 절반 이상」이었다. 그러면 명단 시설을 일부러 올리지 않는 것이 이득이었다. 2안은 특정 단계에서 필요 승리 수가 52에서 802로 15배 뛰었다. **잡은 방법** 단계별 표로 펼쳐 보고 세 번 고쳐서 확정했다.

매칭 지표는 예상 승률 눈금으로 정의하고, 모든 상대를 같은 눈금으로 뽑는다

「강함이 가까우면 대전이 성립한다」가 목적이면 지표의 뜻을 승률로 정의한다. 그리고 공식을 맞출 때 쓰지 않은 표본으로 그 승률이 맞는지 확인한다. 입력은 등급이 아니라 능력치와 레벨이다. 등급으로 뭉치면 같은 등급 안의 강약 차이가 사라지고 레벨이 들어갈 자리가 없다. 사람 상대와 AI 상대가 같은 눈금을 써야 「바뀌는 것은 상대가 사람인지뿐」이 성립한다. 원하는 범위의 상대가 없으면 막지 말고 가장 가까운 상대로 물러나되, 잔 값을 그대로 보여 준다. 한번 정해진 상대는 다른 화면에서 다시 만들지 말고 값으로 넘긴다. 계수를 다시 썼는데 표준오차 안에서만 움직였다면 옮겨 적지 않는다 → 3장 시스템 엔지니어.

사례 · 매칭 지표

무슨 일이 강함 지표를 $\text{예상 승률} = 1/(1+\exp(-\text{차이}/100))$ 꼴로 정의하고, 매칭 구간을 「예상 승률 45~55%」로 잡았다. 예전의 AI 상대 생성은 등급 점수를 썼다. 그래서 최상급 캐릭터 셋이 최대 레벨일 때와 1레벨일 때를 같은 상대로 봤다. 전투 화면이 매칭 화면과 같은 시드로 AI를 다시 만든다는 우연에 기대고 있기도 했다. **잡은 방법** 모드마다 2만 판을 돌려 공식을 맞추고, 쓰지 않은 새 시드로 5천 판을 돌려 맞는지 검사했다. 11개 구간 모두 $\pm 3\text{p}$ 안이었다. 인원이 다른 두 대전 모드는 계수가 달라서, 섞어 계산하면 예외를 던지게 했다.

상위 등급은 하위 등급의 효과를 포함하게 만든다

상위 등급이 하위 등급과 다른 종류의 효과를 주면 상성이 생겨서, 어떤 대전에서는 등급 순서가 뒤집힌다. 상위가 하위의 효과를 포함하는 누적형으로 만든다. 다른 공식을 거쳐 들어가는 효과는 먼저 최솟값·최댓값 제한(clamp)에 눌러 사라지는 비율을 잔다. 받는 피해를 깎는 방어 효과는 「무적」 구멍을 만들 수 있으니, 정해진 양을 흡수하는 배리어 같은 형태를 먼저 검토한다.

사례 · 장비 등급

무슨 일이 장비 효과 1안(힘 증가 + 체력 %)과 2안(치명타율 + 피해 감소) 모두, 어떤 대진이나 어떤 공격력에서는 상위 장비가 하위 장비보다 약했다. 피해가 한 자릿수 정수라서 효과가 계단처럼 꺾였다. 힘을 거쳐 들어가는 효과는 24.1%가 최솟값 제한에 눌러 아무 효과가 없었다. **잡은 방법** 캐릭터 260명 × 260명의 모든 대진을 돌려 실제 효과를 잰다.

보여 주기와 시험 편의를 위해 규칙과 측정 기준선을 건드리지 않는다

보여 주려는 설정과 재려는 설정은 목적이 다르다. 측정 기준선을 바꾸는 것은 언제나 따로 내리는 의도된 결정이어야 한다. 규칙이 테스트를 막으면 규칙을 느슨하게 하지 말고, 따로 표시된 시험용 통로를 둔다. 그 통로를 언제 지울지도 함께 적는다.

사례 · 데모와 측정

무슨 일이 화면에서 강화·약화 효과가 잘 보이도록 데모용 편성을 높은 레벨, 높은 체력, 마법 위주로 바꿨다. 밸런스 측정 도구는 1레벨 그대로 뒀다. 측정 도구까지 바꾸면 기존 5000판 실측치와 비교할 수 없다.

사례 · 시험용 통로

무슨 일이 AI 대전에서는 카드가 나오지 않게 정하자, 오프라인만으로는 성장 기능을 시험할 수 없었다. 규칙은 그대로 두고 캐릭터 관리 화면에 「개발용 카드 지급」을 붙였다. 온라인 대전과 상점이 붙으면 지운다고 적었다.

2.7 화면은 규칙을 거짓 없이 말해야 한다

플레이어는 규칙서가 아니라 화면을 읽는다. 규칙이 옳아도 화면이 그것을 말하지 않거나, 없는 것을 있는 척하거나, 설명할 수 없는 숫자를 띄우면 플레이어는 버그로 읽는다. 화면이 무엇을 말할지 정하는 것은 기획의 일이다. 구현 쪽 세부는 → 5장 클라이언트 엔지니어, 배치와 다국어 레이아웃은 → 6장 디자인 엔지니어.

놓치면 사라지는 선택과 저절로 벌어지는 일은 그 직전에 화면이 알린다

규칙상 특정 시점이 지나면 사라지는 선택지는 그 시점에 분명히 보여 주고 「보류」로 넘길 수 있게 한다. 플레이어에게 규칙의 순서를 외우게 하지 않는다. 사용자 모르게 상태를 바꾸는 규칙은 직접 눌러 보고, 놀라게 되는 지점에 예고를 둔다. 기능의 입구를 화면에서 걷어도 그 입구로 저장해 둔 상태는 계속 작동한다. 다른 화면에서 확인하고 되돌릴 길을 남기고 알린다. 할 수 있는 행동이 하나만 남는 상태에서도 그 행동을 할 단추가 있는지 확인한다 → 5장 클라이언트 엔지니어.

사례 · 행동 순서

무슨 일이 전용 스킬은 이동하기 전에만 쓸 수 있다. 그래서 먼저 이동해 버리면 「쓸 수 있었는데 날린」 상황이 생긴다. 차례 맨 앞에서 [스킬 발동] [보류]로 한 번 묻고 지나가게 했다.

사례 · 숨은 상태 변화

무슨 일이 장비 자동 교체 규칙을 실제로 써 보니, 아무 표시 없이 다른 캐릭터의 장비를 벗겼다. 다른 장비를 낀 캐릭터를 고를 때만 확인창을 띄우게 바꿨다. 출전 화면의 지참 아이템 판은 주 행동 단추를 화면 밖으로 밀어내서 걷어 냈다. 그런데 예전에 지정해 둔 아이템은 계정에 남아 다음 판마다 계속 소모됐다. **왜 조용했나** 소모는 계속 일어나는데 그것을 보여 주던 자리만 사라졌다. 사용자에게는 「아이템이 없어졌다」로만 보인다. **잡은 방법** 기능을 걷어 내면서 남는 상태를 따졌다. 다른 화면(상점)에서 회수할 수 있다는 경고를 문서에 적었다.

아직 없는 것은 종류에 따라 숨기거나, 잠그거나, 자리를 그려 둔다

내용이 빠진 글은 줄짜 감춘다. 아직 동작하지 않는 단추는 놀리게 두지 말고 비활성화한 뒤 이유를 화면에 적는다. 「놀리는데 아무 일도 없다」는 고장으로 읽힌다. 두 가지를 섞으면 둘 다 고장으로 읽힌다. 잠글지 말지는 「동작하는 데이터와 규칙이 있는가」로 정한다. 있으면 모양이 낡아도 열어 둔다. 검증 경로를 지키기 위해서다. 없으면 열지 않는다. 화면 잠금은 편의일 뿐이고 실제 판정은 로직 함수가 다시 한다. 잠긴 콘텐츠는 자리와 이름을 보여 줘야 목표가 된다. 숨기면 플레이어는 무엇을 향해 가는지 모른다.

사례 · 미완성 기능

무슨 일이 캐릭터 상세 화면의 「인물 소개: (없음)」은 데이터가 빠진 것인지 원래 없는 것인지 구별되지 않았다. 계획서 원안은 [레벨/스킬 관리] 단추도 잠그라고 했다. 하지만 그때 성장을 시험할 길은 그 화면의 개발용 지금 하나뿐이었다. 잠갔다면 오프라인 진행이 멈추고 레벨업 스모크 테스트도 죽었을 것이다. 레벨업은 규칙도 화면도 있었고, 전적은 데이터 모델이 없었다. 그래서 레벨업은 열고 전적은 잠갔다. 상점의 카드 교환·육성 초기화·화폐 상품은 규칙과 결제가 정해지지 않아 자리만 두고 잠근 뒤 이유를 적었다.

사례 · 잠긴 콘텐츠

무슨 일이 확장 도시에서 「짓지 않은 건물은 숨긴다」로 했다. 그러자 짓기 전에는 빈 들판뿐이라서, 치료 시설이나 제작 시설이 어디에 들어설지 알 수 없었다. **잡은 방법** 네 건물을 모두 그려 두었다. 지은 건물에는 소개 문장(「소모 자원 생산량을 늘린다.」)을, 짓지 않은 건물에는 「아직 열리지 않았다」를 이름 밑에 적었다.

「없음」도 값으로 보여 주되, 설명할 수 없거나 비교하면 안 되는 숫자는 내지 않는다

0과 「없음」도 하나의 상태다. 빼 버리면 그 항목만 본 사람은 그런 개념이 있다는 것조차 모른다. 강조색은 무언가를 구별할 때만 쓴다. 진짜 값이 없으면 지어내지 말고 지금부터 기록한다. 지어낸 과거 값은 화면이 거짓말을 하게 만든다. 반대로 내부 지표를 화면에 올리기 전에는 두 가지를 본다. 플레이어에게 설명할 수 있는가, 다른 정보로 이미 대신하고 있지 않은가. 표시용 대체 공식을 따로 만들면 두 숫자가 갈라질 자리가 하나 더 생긴다. 단위나 척도가 다른 수치는 목록을 묶음으로 갈라 표시한다. 코드의 검사가 막는 것은 계산이 섞이는 것뿐이다. 눈이 섞이는 것은 화면만 막을 수 있다. 누를 수 없는 선택지를 미리 골라 두지 않는다. 켜져 있는데 쓸 수 없는 상태도 화면의 거짓말이다.

사례 · 「없음」의 표시

무슨 일이 시전 지연이 0인 기술 27종에도 발동 시간을 「즉시」라고 띄웠다. 금색은 지연이 있을 때만 칠했다. 「즉시」까지 금색이면 색이 아무것도 구별하지 않는다. 장비의 제작일을 화면에 띄우려 했는데, 진짜 제작일은 어디에도 기록되어 있지 않았다. 기획자는 임의의 날짜를 넣자고 했다. 대신 기록을 시작한 시각을 한 번 찍었다. 그리고 그 값을 자주 다시 쓰는 지금 정보와 다른 칸에 두어, 다시 쓰다가 빠뜨리지 않게 했다.

사례 · 비교하면 안 되는 숫자

무슨 일이 인원이 다른 두 대전 모드의 강함 지표는 계수가 다른데 값의 범위가 겹친다. 목록처럼 부대 목록에 두 모드를 섞어 놓으면 눈으로 비교하게 된다. 그래서 목록을 모드별 묶음으로 갈랐다. 랭킹의 부대 「강함」 열은 회귀 계수로 만든 값이라 플레이어에게 설명할 수 없었다. 표시용 함수(등급 점수 × 레벨)를 따로 만들었다가 결국 지웠다. 행을 누르면 부대 구성이 펼쳐지므로 요약 숫자가 필요 없다는 기획 판단이었다. 열과 함수를 함께 지웠다.

화면마다 주 행동을 먼저 정하고, 그 행동의 자리를 UI가 덮지 않게 한다

각 단계의 주 행동(놓기, 보기, 고르기)을 먼저 정하고, 그 행동에 필요한 영역을 UI가 덮지 않게 한다. 카메라 초점이 잡히는 자리에는 고정 UI를 두지 않는다. 강조는 하나에만 줄 때 강조가 된다. 작은 배울에서 읽을 수 없는 정보는 판에서 빼고 카드나 팝업으로 보낸다. 화면은 「지금 어떤가」(현황)와 「무엇을 할까」(행동)를 나누고, 같은 목적지로 가는 문은 하나만 둔다. 목록은 고르는 화면이다. 지우기 같은 파괴적 행동은 대상 안으로 들어가서 하게 한다. 모바일이 대상이면 마우스 올림(hover)이나 키보드에만 기대는 상호작용은 덤으로도 만들지 않는다.

사례 · 전투 화면 배치

무슨 일이 행동을 고르는 창을 화면 아래 가운데에 두면 아래쪽 진영(하단 5행)을 가려서 오른쪽으로 옮겼다. 배치 단계에서 기물 정보 팝업이 판 한가운데를 덮었다. 그래서 배치는 「놓는 단계」, 정찰은 「보는 단계」로 나뉘었다. 칸이 30px대라 판 위 글자는 뭉개졌다. 체력 막대 하나만 남겼다. 열 명 모두에게 테두리를 두르면 「지금 누구 차례인가」가 묻혔다. 차례인 기물에만 테두리를 둘렀다. 상태 이상 22종은 칸에 개수 점만 찍고, 상세는 아무 기물이나 눌러 여는 팝업으로 보냈다.

사례 · 메타 화면 구조

무슨 일이 도시 관리 화면은 현황판으로 두고, 짓기와 증축은 [건물 관리]로 나뉘었다. 같은 곳으로 가던 다른 단추는 지웠다. 부대 [삭제] 단추는 두 번 옮겨졌다. 처음에는 목록 줄마다 붉은 단추가 있었다. 다음에는 목록 아래 단추 하나와 고르는 팝업이 됐다. 마지막에는 편성 화면 안의 [부대 삭제]가 됐다. 줄마다 단추를 두면 잘못 누를 자리가 부대 수만큼 생겼다. 팝업으로 옮겨도 목록 화면에 「지우는 일」이 남아 있었다. **잡은 방법** 단추를 옮길 때마다 문제가 없어졌는지, 자리만 바뀌었는지 따졌다.

결말은 사람의 손이 내리게 하고, 써 봐서 버그처럼 보이면 설계를 다시 연다

제재와 종료는 가능하면 기존 장치와 사람의 행동으로 일어나게 하고, 새 상수를 늘리지 않는다. 시스템이 대신 내린 결말은 「내가 안 했는데 졌다」는 기분을 남긴다. 누를수록 결과에 가까워지는 단추는 진행도를 보여 줘야 한다. 설정끼리 연동되는 규칙은 사용자가 결과를 보고 「의도된 것」으로 읽을 수 있는지로 판단한다. 직접 써 본 느낌이 예전 결정과 어긋나면 결정을 다시 연다.

사례 · 무응답 상태

무슨 일이 접속은 되어 있는데 아무 행동도 하지 않는 상태를 처리해야 했다. 첫 안은 「N초 동안 입력이 없으면 서버가 대신 항복시킨다」였다. 기획자 안은 「상대가 [차례 넘기기]를 3번 누르면 승리」였고, 이쪽을 택했다. 한 번에 20초인 차례 넘기기를 3번 하면 이미 최소 60초 동안 응답이 없었다는 뜻이다. 그래서 새 상수가 필요 없다. 누르는 사람이 있으면 방이 비어 있는 것도 아니어서 두 처리가 겹치지 않는다. 덧붙인 것은 두 가지다. 횟수는 누적한다(초기화하면 시간 끌기가 공짜가 된다). 그리고 「(2/3)」처럼 진행도를 보여 준다.

사례 · 설정 연동

무슨 일이 예전에 「한 번 고른 음성 더빙은 화면 언어를 바꿔도 유지된다」고 정했다. 실제로 써 보니 글자만 바뀌고 목소리는 그대로여서, 잘못 구현된 것처럼 보였다. **잡은 방법** 기획자가 직접 써 보고 결정을 뒤집었다. 언어를 바꾸면 더빙도 그 언어의 기본값으로 바뀐다. 직접 고른 더빙은 다음에 언어를 바꿀 때까지만 유지된다.

작업 전후 점검표

- ☐ 올리는 질문마다 선택지와 각 선택지의 파급을 붙여, 번호 표로 모아 한 번에 물었는가?
- ☐ 두 갈래로 읽히는 문장이나 낯선 원안 표기를 혼자 해석하거나 지우지 않았는가?
- ☐ 받은 지정을 명시된 목표점 전부와 최대·최소 극단에 넣어 계산해 봤는가? 모순과 우회로를 보고했는가?
- ☐ 문서·화면 문구에 데이터의 숫자를 손으로 다시 적은 곳이 있는가? 그 숫자는 출처에서 파생되는가?
- ☐ 수치나 이름을 바꿨다면 모든 언어의 설명 문장·키 이름·용어표에서 옛 값을 검색했는가?
- ☐ 이 규칙이 원래 없던 중간 상태(고르는 도중, 잡아 두기)를 만드는가? 기존 절차로 합칠 수 있는가?
- ☐ 효과의 범위를 이름 목록이 아니라 엔진의 분류나 기본값으로 정했는가? 승리·패배 조건과의 교차를 훑었는가?
- ☐ 시간 제한은 어느 시계에 걸려 있고, 그 화면에서 그 시계가 실제로 흐르는가?
- ☐ 여러 상수의 관계를 타임라인으로 굴러 보고 불변식 검사로 고정했는가?
- ☐ 문턱 조건이 가장 작은 용량에서도 통과 가능한가? 제한을 풀 때 가격과 효과 포화도 다시 봤는가?
- ☐ 패널티마다 「이것이 없으면 어떤 우회가 싸지는가」를 적었는가? 보상을 가르는 변수는 고를 수 있는 것인가?
- ☐ 서버가 검증할 수 없는 값에 기대는 규칙인가? 화면이 막는 순서를 서버 규칙도 막는가?
- ☐ 조정 뒤 그대로여야 할 지표가 그대로이고, 단조여야 할 것(더 쓰면 더 빠르다)이 단조인가?
- ☐ 진행 조건을 성향별 시뮬레이션으로 끝까지 밀어 교차과 역유인이 없는지 봤는가?
- ☐ 잠긴 것·없는 것·설명할 수 없는 숫자가 화면에서 어떻게 보이는가? 실제로 써 봤을 때 버그처럼 보이는 곳은 없는가?

3장

시스템 엔지니어

규칙 엔진·데이터 파이프라인·저장 형식·측정 도구를 만드는 사람

이 장은 게임 규칙을 코드로 옮기는 사람, 기획 데이터를 엔진이 읽는 형태로 바꾸는 사람, 계정 저장 형식과 그 변환을 맡는 사람, 밸런스를 재는 도구를 만드는 사람을 위한 것이다. 이 자리의 실수는 거의 오류를 내지 않는다. 판정은 맞는데 표시만 어긋난다. 데이터는 맞는데 아무도 읽지 않는다. 저장은 성공하는데 필드 하나가 매번 사라진다. 통계는 그럴듯한 숫자를 내놓는다. 사례는 한 턴제 전술 게임에서 나왔다. 체스 기물처럼 움직이는 캐릭터들이 싸우고, 행동 순서는 유닛마다 흐르는 대기 시간으로 정해진다. 서버가 판정하는 온라인 대전, 가차와 캐릭터 육성, 열 개 언어 지원이 붙어 있다. 이 프로젝트에서 시스템 쪽 교훈 55개를 모아 보니 뿌리는 여섯 갈래였다. 시계와 난수가 새는 자리, 같은 판정이 두 벌이 되는 자리, 적힌 데이터가 읽히지 않는 자리, 저장 형식이 조용히 깎이는 자리, 빌드 도구가 말없이 하는 일, 측정이 스스로를 속이는 자리다.

이 장의 원칙

1. 도메인 로직은 시계와 난수를 인자로 받고, 「지금」은 최상단 한 곳에서 넣는다
2. 시간으로 차오르는 값은 자투리·상한·되감긴 시계를 함께 설계하고, 요율이 바뀌기 전에 정산한다
3. 「나중에 일어날 일」을 절대시각으로 예약하기 전에, 그 시각을 미는 수단이 있는지 본다
4. 난수는 시드와 소비 위치로 다루고, 미리 보여 줄 값은 대상이 생기는 순간 뽑는다
5. 화면이 보여 주는 판정은 엔진 함수가 내고, 그 정답은 엔진을 실제로 돌려 얻는다
6. 보정이 붙는 값은 읽는 자리를 하나로 모으고, 결과는 로그가 아니라 끝난 상태에서 쏜다
7. 분기는 구체적인 갈래부터 하고, 공통 조건은 갈래 밖 맨 앞에서 한 번만 판정한다
8. 불가능한 상태는 예외로 터뜨리고, 안전장치 상한은 값으로 돌려준다
9. 한 번 쓰면 안 바뀌는 로그는 값을 복사하지 말고 공유한다
10. 데이터에 필드를 더했으면 그것을 읽는 자리와 깎는 자리를 끝까지 따라간다
11. 데이터와 예외 핸들러를 함께 읽고, 대조 스크립트도 본 파이프라인과 같은 정규화를 거친다
12. 여러 표의 열쇠인 열거형에 값을 더하지 말고 새 축으로 가른다
13. 텍스트는 그 값이 실제로 달라지는 단위에 붙인다
14. 기능을 없앴으면 그것만 쓰던 인자·필드·함수도 함께 지운다
15. 저장에는 결과가 아니라 원인을, 순서가 아니라 식별자를 남기고, 기본값은 「설정 없음」으로 적는다
16. 저장 형식에 필드를 더하면 마이그레이션과 왕복 회귀를 함께 고친다
17. 마이그레이션한 결과가 원문과 다르면 그 자리에서 다시 저장한다
18. 상대적 변환은 버전에 걸어 한 번만 일어나게 하고, 옛 형식 입력은 손으로 적어 고정한다
19. 이력은 DB 한 행처럼 적고, 영구 집계는 이력과 따로 쌓는다
20. 원본과 화면 사이에 변환 지점을 하나 둔다

21. 생성 도구는 말없이 지우지도, 말없이 풀백하지도 않는다
22. 검증은 지금 다시 돌린 결과만 믿고, 소비자가 생긴 검사는 실패로 올린다
23. 증분 빌드는 파일 시각이 아니라 내용으로 판단한다
24. 사람이 관리하는 파일은 통째로 다시 쓰지 말고 바꿀 부분만 바꾼다
25. 실행 환경이 받아 주는 문법과 모듈 로딩의 한계를 설계 제약으로 적는다
26. 회귀 계수는 표준오차와 함께 읽고, 오차 안의 움직임으로 상수를 바꾸지 않는다
27. 모형을 관측 범위 밖으로 외삽하지 않고, 테스트에 박은 직관도 실측으로 검증한다
28. 「변화 없음」이 나오면 먼저 측정 실패를 의심하고, 새 행동은 분포부터 찍는다
29. 수치를 맞추는 함수의 회귀는 전 구간을 훑고, 계수가 바뀌면 그 가정을 다시 잔다
30. 검사는 구현의 키가 아니라 결과를 재고, 일부러 깨서 실패하는 것을 본다

3.1 순수성과 시간

전투를 다시 재생하고, 서버가 판을 처음부터 다시 계산해 검증하고, 가짜 시계로 테스트하려면 규칙 계층이 시계와 난수를 스스로 읽지 않아야 한다. 같은 입력이면 언제 어디서 돌려도 같은 결과를 내는 엔진을 결정적(deterministic) 엔진이라고 한다. 이 원칙은 코드 머리말에 적어 뒀도 새 코드에서 다시 깨진다. 시간이 끼는 값은 경계가 셋 이상이라 설계 단계에서 따로 따져야 한다.

도메인 로직은 시계와 난수를 인자로 받고, 「지금」은 최상단 한 곳에서 넣는다

시간이나 난수에 기대는 계산은 그 값을 인자로 받는 순수 함수로 만든다. 「지금」을 넣는 자리는 그 값을 쓰는 모든 화면보다 위에 있는 한 곳이다. 한 화면에서만 넣으면 다른 화면이 낡은 값을 본다. 순수 계층에 새 모듈을 더할 때마다 이 점을 따로 점검한다. 타입 검사도 테스트도 「이 함수가 순수한가」를 막아 주지 않는다. 순수하게 둔 규칙은 서버와 클라이언트가 함께 쓸 수 있다. 「내 순위」처럼 내 데이터만으로 계산되는 값은 클라이언트가 직접 계산해 서버가 죽어도 보이게 한다. 어느 필드를 서버가 소유하는지 같은 권위 규칙도 DB 없는 순수 모듈에 두면 단위 테스트가 쉽다.

사례 · 시간 충전 자원

게임에는 대전에 들어갈 때마다 조금씩 쓰고, 시간이 지나면 다시 차오르는 자원이 흔하다. **무슨 일이** 이 자원의 정산 함수를 「계정 상태와 현재 시각을 받아 새 계정 상태를 돌려주는」 순수 함수로 두었다. 그런데 이 자원을 읽는 화면이 메인·출전 대기·부대 편성·도시 관리 넷이었다. **왜 조용했나** 도시 화면에서만 시각을 넣었다면 나머지 셋은 오류 없이 낡은 값을 보여 줬을 것이다. **잡은 방법** 시각은 앱 최상단 컴포넌트가 열릴 때와 1분마다 넣게 했다. 저장 형식 마이그레이션(옛 형식 데이터를 새 형식으로 바꾸는 변환)에도 현재 시각을 받게 하자는 안이 있었지만, 시각이 필요 없는 곳까지 시계가 번진다는 이유로 기각했다.

사례 · 가차

무슨 일이 가차 추첨 함수의 첫 구현이 계정별 가차 시드를 `Date.now()` 와 `Math.random()` 으로 스스로 만들었다. 그 패키지 머리말에는 「I/O·시계·난수를 안 쓴다」가 이미 적혀 있었다. **왜 조용했나** 타입 검사와 테스트가 모두 통과했다. **잡은 방법** 코드를 다 쓴 뒤 시드를 만드는 보조 함수를 다시 읽다가 알아챘다. 새 시드를 함수 인자로 받게 고쳤다. 이미 저장된 시드가 있으면 그 인자는 무시한다.

서버가 판을 다시 계산해 검증할 때 시각이 입력에 끼면, 그 시각도 결정적인 값으로 실어야 한다. 이 부분은
→ 4장 서버 엔지니어에서 다룬다.

시간으로 차오르는 값은 자투리·상한·되감긴 시계를 함께 설계하고, 요율이 바뀌기 전에 정산한다

시간 충전에는 경계가 적어도 셋 있다. 첫째, 정산 시각을 「지금」으로 찍으면 다음 단위가 차기 전에 자주 들르는 사람은 영원히 못 받는다. 정산 시각은 번 만큼만 민다(`at += earned * step`). 둘째, 상한에 닿아도 시각은 전진시키고 넘친 시간은 버린다. 시간을 쌓아 두면 자원을 쓰는 순간 몇 시간치가 한꺼번에 돌아온다. 셋째, 시계가 뒤로 가면 아무것도 하지 않는다. 뒤로 간 시계에 맞춰 정산 시각을 당기면 기기 시계를 앞뒤로 흔들어 같은 보상을 여러 번 받는다. 0이나 누락값은 epoch(1970년 1월 1일)가 아니라 「아직 정산 안 함」으로 읽는다. 요율이 바뀌는 함수는 시각을 인자로 받아 먼저 정산하게 규칙 층에 묶는다. 시간 단위를 바꿨으면 정산 주기가 그 해상도보다 촘촘한지도 본다.

사례 · 시간 충전 자원

무슨 일이 1시간에 하나씩 차는 입장 자원을 설계하면서 세 경계가 모두 틀릴 수 있었다. 30분마다 들르는 사람은 정산 시각이 매번 「지금」으로 당겨져 하나도 못 받을 판이었다. 게다가 정산 시각 필드가 0인 옛 계정을 epoch로 읽으면 1970년부터 충전돼 한 번에 상한까지 찰 판이었다. **왜 조용했나** 화면에는 전부 「자원이 좀 많네/적네」로만 보인다. 건물을 증축해 충전 속도가 올라갈 때의 선정산을 화면 쪽에 맡기면, 잊어도 예외 없이 조금 더 들어올 뿐이다. **잡은 방법** 설계 단계에서 경계를 따지고 회귀 테스트(한 번 고친 동작이 다시 깨지지 않게 고정해 두는 자동 테스트)로 고정했다. 입력은 59분 59초, 30분씩 두 번, 100시간 방치 뒤 소비, 시계 앞뒤로 흔들기였다. 증축 함수는 현재 시각을 인자로 받아 먼저 정산한다.

사례 · 단위 변경

무슨 일이 화면의 정산 주기는 시간 충전 자원에 맞춘 60초였다. 그 뒤 아이템 제작 기간이 주 단위에서 분 단위로 바뀌자, 1분짜리 주문의 완성 알림이 최대 1분 늦게 났다(실측 117초). **왜 조용했나** 기간이 주 단위일 때는 1분 오차가 보이지 않았다. 화면에는 「0초인데 아무 일도 안 일어난다」로만 보였다. **잡은 방법** 실제로 로그인하는 스모크 테스트(실제 화면을 띄워 끝까지 눌러 보는 자동 검사)에서 1분을 그대로 기다려 확인했다. 수정 후 59초가 됐다. 정산 로직은 두 별로 만들지 않고, 화면이 현재 시각을 넣어 기존 정산 함수 하나를 부르게 했다.

「나중에 일어날 일」을 절대시각으로 예약하기 전에, 그 시각을 미는 수단이 있는지 본다

예약의 뜻이 「시각 T에」인지 「내 차례가 돌아올 때」인지 먼저 정한다. 후자라면 절대시각 큐에 넣지 말고 제어권이 넘어오는 지점에 묶는다. 그러면 방해, 발동 시점의 대상 재선정, 시전 중 무산이 한꺼번에 풀린다. 미는 수단이 없는 사건(예: 사망 시각 기준으로 터지는 효과)만 절대시각으로 두고, 둘을 같은 통에 넣지 않는다.

사례 · 턴 스케줄러

이 게임은 유닛마다 다음 차례까지의 대기 시간(WT)이 흐르고, 0이 된 유닛이 행동하는 스케줄러로 돈다. **무슨 일이** 강한 스킬에 「시전하고 잠시 뒤 발동」 하는 지연을 붙이면서, 이를 절대시각 예약 큐에 넣었다. 그랬더니 상대가 시전자의 대기 시간을 크게 미는 스킬(30 → 180)을 써도 예약은 원래 시각에 그대로 터졌다. 예약이 처리되는 동안 엔진 시계가 옛 시각에 있어서, 효과의 지속시간이 시전 시점 기준으로 잡히기도 했다. **왜 조용했나** 결정성·리플레이·스냅샷이 다 맞게 돌아다. 미는 스킬과 겹치는 시나리오를 돌리기 전에는 틀린 데가 없었다. **잡은 방법** 기획서에 있던 시나리오(시전 중 대기 시간 30 → 180)를 회귀 테스트로 돌렸다. 「시전 중」 상태를 유닛에 두고, 그 유닛에게 제어권을 주는 함수에서 발동하게 옮겼다.

난수는 시드와 소비 위치로 다루고, 미리 보여 줄 값은 대상이 생기는 순간 뽑는다

큰 유한 난수 배열은 통째로 저장하지 말고 결정적 생성기의 입력(시드, 소비 개수)만 저장해 필요할 때 다시 만든다. 배열을 저장·복원하는 코드는 「재접속마다 다시 섞는」 실수를 부르기 쉽고 용량도 낭비한다. 화면이 미리 보여 줘야 하는 무작위 값은 「쓰는 순간」이 아니라 「대상이 생기는 순간」에 한 번 뽑아 확정한다. 이때 난수 소비 순서가 바뀌어 같은 시드로도 다른 판이 된다는 대가와, 옛 가정을 고정하던 테스트의 수정을 결정 기록에 함께 적는다.

사례 · 가차

무슨 일이 가차를 「정해진 장 수의 카드 더미를 섞어 위에서부터 뽑는」 방식으로 만들었고, 계정마다 25,200장짜리 배열이 필요했다. **잡은 방법** 배열 대신 {seed, drawn} (시드, 이미 뽑은 장 수)만 저장했다. 읽을 때마다 같은 시드로 다시 섞고 drawn 장을 건너뛴다. 전투 리플레이가 로그 대신 시드와 난수 소비 위치만 들고 다니는 것과 같은 패턴이다.

사례 · 동점 처리

무슨 일이 대기 시간이 동시에 0이 된 유닛들의 순서를 그 순간 시드 난수로 뽑고 있었다. 기획자가 「순번은 한 번 뽑아 계속 쓴다」로 정하자, 판을 만드는 함수에서 순번을 한 번 뽑도록 바꿨다. 설계 문서에는 「전투 시작 때」라고 적혀 있었지만, 그보다 이른 판 생성 시점으로 옮겼다. **왜 조용했나** 예전 방식에서는 배치 화면이 행동 순서를 미리 적을 수 없었는데, 그것은 버그가 아니라 기능 부재로만 보였다. **잡은 방법** 유닛이 생기는 자리가 하나뿐이고 배치 단계 동안 대기 시간이 안 바뀐다는 점을 따져, 그 자리가 가장 이른 확정 시점임을 확인했다.

3.2 판정의 단일 출처

같은 규칙이 두 곳에 있으면 한쪽만 바뀐다. 판정은 맞게 도는데 화면만 다른 말을 하거나, 보정 하나가 확률 판정 한 곳에서만 빠진다. 이 절의 원칙은 모두 「판정과 그 판정을 묻는 자리를 하나로 모은다」는 뿌리를 공유한다.

화면이 보여 주는 판정은 엔진 함수가 내고, 그 정답은 엔진을 실제로 돌려 얻는다

버튼 활성화 여부, 예상 피해, 상태 표시, 차례 예보처럼 화면이 묻는 것은 엔진이 질의 함수로 낸다. 화면과 판정이 둘 다 그 함수를 부른다. 화면이 규칙 사본을 들고 있으면 엔진이 바뀌어도 표시만 조용히 어긋난다. 그 함수의 회귀 정답은 손으로 쓴 기대값이 아니라 엔진을 실제로 진행시킨 결과로 삼는다. UI를 접더라도 함수와 「미리보기 = 실제 결과」 회귀는 남긴다.

사례 · 상태 표시

무슨 일이 「주변 몇 칸 안의 적 공격력을 절반으로」 같은 반경형 효과(오라)는 시전자에게만 표식을 두고, 피해를 계산할 때 거리를 다시 재는 구조였다. 그래서 당하는 쪽의 상태 목록이 비어, 「내 공격력이 왜 절반이지?»에 화면이 답할 수 없었다. **왜 조용했나** 판정은 맞았고 표시만 비어 있어서, 오류가 아니라 「설명 없는 현상」으로 보였다. **잡은 방법** 엔진이 「이 유닛에 지금 걸린 반경 효과 목록」을 돌려주는 질의 함수를 내고, 화면과 판정이 함께 부르게 했다. 버튼 활성화 여부도 엔진이 낸 후보를 엔진의 의도 검증 함수에 넣어 보고 정했다.

사례 · 차례 예보

무슨 일이 전투 UI를 고치면서 스킬 상태, 명령 버튼 활성화, 조준 후보, 「앞으로 누가 몇 번째로 움직이나」 예보를 엔진의 질의 모듈 하나로 모았다. 조준 로직은 화면 컴포넌트 안에 있던 사본을 옮긴 것이고, 예보는 엔진이 차례를 끝낼 때와 같은 순서로 센다. **잡은 방법** 예보 회귀가 3인·5인 부대 × 12개 시드에서 예보한 순서를 엔진을 실제로 진행시킨 순서와 맞춘다. 화면 쪽 이야기는 → 5장 클라이언트 엔지니어.

보정이 붙는 값은 읽는 자리를 하나로 모으고, 결과는 로그가 아니라 끝난 상태에서 센다

상태 효과로 바뀔 수 있는 능력치는 단일 접근 함수로만 읽는다. 읽는 자리가 여럿이면 보정이 한 곳에서 빠져도 화면에는 숫자로 드러나지 않고 확률 판정 안에서만 어긋난다. 「몇이 죽었나」처럼 결과를 묻는 값은 사건 로그가 아니라 최종 상태에서 센다. 로그에는 나중에 되돌려진 사건도 남아 있기 때문이다.

사례 · 상태 패널티

무슨 일이 전투에서 쓰러진 캐릭터가 한동안 부상 상태로 남아 능력치가 깎이는 규칙을 들었다. 그런데 무력·지력·통솔 같은 능력치를 읽는 곳이 5개 파일 8군데였다. 한 곳만 빠뜨려도 크리티컬 확률이나 상태이상 저항만 표시 없이 어긋날 판이었다. 쓰러진 캐릭터 수를 로그의 사망 이벤트로 세면, 부활 스킬로 되살아난 캐릭터까지 부상으로 잡혔다. **잡은 방법** 보정을 넣기 전에 읽는 자리를 전부 훑어 접근 함수 하나로 모았다. 쓰러진 수는 전투가 끝난 상태에서 썼다.

분기는 구체적인 갈래부터 하고, 공통 조건은 갈래 밖 맨 앞에서 한 번만 판정한다

분류가 겹치면 더 구체적인 갈래를 먼저 판정하고, 실행과 미리보기가 같은 순서를 쓰게 한다. 갈래를 정하는 기준은 별도 목록이 아니라 효과 데이터에서 읽어야 데이터만 고쳐도 따라온다. 거리·반경 같은 공통 조건이나 「이 상황에서는 못 쓴다」 같은 선결 조건은 갈래마다 따로 두지 않는다. 판정 함수 맨 앞, 분기 밖에서 한 번만 본다. 갈래 안에 두면 그 갈래에 들어가지 않는 경우에는 조건이 한 번도 평가되지 않는다.

사례 · 스킬 분류

무슨 일이 칸에 불이나 성채 같은 지형을 까는 스킬도 계열 분류상으로는 「지원」이었다. 스킬 실행 함수와 성공률 미리보기 함수가 「지형 → 지원 → 상태이상」 순으로 분기하지 않으면 지형 갈래에는 영영 들어가지 않았다. 「지형 스킬인가」는 별도 목록이 아니라 효과 데이터에 지형을 까는 효과가 있는지로 판정했다.

사례 · 사거리 판정

무슨 일이 「주변 8칸 안의 적 1명」에게 쓰는 폭약 아이템이 데이터에 「대상: 적 1명, 반경: 1」로 적혀 있었다. 그런데 타입 정의의 「적 1명」 대상에는 반경 필드가 없었고, 거리 검사도 아군 대상 갈래 안에만 있었다. 그래서 폭약이 판 반대편까지 닿았다. 화면은 반경을 적은 설명문을 그대로 띄우고 있었다. **왜 조용했나** 여태 반경을 쓰는 것이 아군 대상 스킬뿐이라 적 대상 갈래는 그 필드를 받은 적이 없었다. **잡은 방법** 조준 거리를 일부러 깨 보는 확인에서 드러났다. 거리 검사를 갈래 밖으로 뺐다. 그 전에도 같은 꼴이 있었다. 「주변 3×3 안에 지형이 있으면 못 쓴다」를 조준 분기에 엮었더니, 자기 중심 범위 스킬은 조준 사양을 돌려주지 않아 `if (!spec) return ok`에서 그냥 통과했다.

불가능한 상태는 예외로 터뜨리고, 안전장치 상한은 값으로 돌려준다

「아무도 행동할 수 없다」 같은 불가능한 상태를 무승부·0·빈 목록 같은 정상 결과로 내보내지 않는다. 결과 범주에 섞인 비정상은 통계가 삼켜 버린다. 반대로 상한이 정상 흐름을 막으려는 것이 아니라 남용을 막는 안전장치라면, 도달해도 기능이 멈추지 않게 결과 값으로 표현한다. 기준은 하나다. 그 상태가 버그의 증거인가, 설계가 허용한 끝인가.

사례 · 턴 스케줄러

무슨 일이 스케줄러가 시간을 한 번만 진행했다. 그래서 차례를 받을 유닛이 그 구간에서 지속 피해 등으로 죽으면 다음 차례를 받을 유닛이 없어 전투가 멈췄다. 자동 대전 함수는 이 정지를 「무승부」로 보고했다. **왜 조용했나** 5000판 중 3판이 무승부 통계 속에 섞여 정상 결과처럼 보였다. **잡은 방법** 밸런스 측정 도구의 통계에서 무승부 판의 공통점을 봤다. 세 판 모두 특정 캐릭터 하나가 낀 판이었다.

사례 · 가차

무슨 일이 앞의 25,200장 가차 더미에 끝이 있는 것은 정상 흐름의 상한이 아니라 매크로 남용을 막는 장치였다. 다 뽑았을 때 `RangeError`를 던지는 대신 `exhausted: true`를 돌려주게 했다.

한 번 쓰면 안 바뀌는 로그는 깊은 복사하지 말고 공유한다

상태를 불변으로 다루느라 매 행동마다 통째로 복사하면, 덧붙이기만 하는 로그까지 복사되어 비용이 로그 길이에 비례한다. 판 전체로는 제공이 된다. 「한 번 만든 이벤트는 바뀌지 않는다」는 전제 위에서 로그는 얇게 공유한다. 그 성질을 지키는 전용 테스트를 함께 둔다. 테스트가 없으면 누군가 「안전하게」 깊은 복사로 되돌린다.

사례 · 성능

무슨 일이 매 행동마다 `structuredClone`이 전투 상태와 함께 전투 로그까지 복사해, 자동 대전 3판에 152초가 걸렸다. **왜 조용했나** 짧은 테스트 전투에서는 차이가 없었고, 판이 길어질 때만 느려졌다. **잡은 방법** 실행 시간을 켜다. 로그만 얇게 복사하자 1.7초(약 90배)가 됐다. 같은 지뢰가 네트워크로 상태를 보낼 때 다시 나타난 이야기는 → 4장 서버 엔지니어.

3.3 데이터 구동 시스템 — 적힌 것이 읽히는가

스킬·아이템의 효과를 데이터로 적고 엔진은 해석만 하는 구조는 기획 변경을 싸게 만든다. 대신 「데이터에 적혀 있다」와 「엔진이 그것을 읽는다」 사이에 아무 검사도 없는 틈이 생긴다. 데이터 검증 테스트는 값이 올바르게 적혔는지만 보고, 누가 읽는지는 보지 않는다.

데이터에 필드를 더했으면 그것을 읽는 자리와 깎는 자리를 끝까지 따라간다

데이터 스키마에 효과를 더하면 그 필드를 소비하는 코드 경로와, 그 경로로 결과가 실제로 바뀌는지 보는 검사를 함께 만든다. 횡수 제한 같은 필드는 「읽는 자리」뿐 아니라 「깎는 행동」도 찾는다. 버그 하나를 고치면 같은 필드를 쓰는 항목 전체를 훑는다. 비슷한 종류(장비와 아이템)는 통로 하나로 함께 닫는다.

사례 · 장비

무슨 일이 게임 재화로 만들어 캐릭터에게 지급하는 장비가 도입 뒤 3주 동안 화면에만 뜨고 전투에서는 효과가 없었다. 전투로 넘기는 출전 명단 타입에 장비 필드가 없었고, 엔진에는 보호막·크리티컬 피해 같은 장비 효과를 읽는 줄이 하나도 없었다. **왜 조용했나** 데이터 검증은 「값이 맞게 적혔는가」만 봤다. **잡은 방법** 새 소모 아이템의 효과를 엮을 자리를 찾아 코드 길을 따라가다가, 기델 통로가 아예 없다는 것을 알았다.

사례 · 횡수 제한 버프

무슨 일이 「이동 제한을 한 번 무시한다」는 버프에 횡수 1이 적혀 있었는데, 이동이 그 횡수를 깎지 않아 지속시간 내내 무제한이었다. 반대로 그 전에는 소모 함수가 횡수 필드가 없는 지속형 버프까지 지워, 한 번 행동하자 버프가 사라진 일도 있었다. **왜 조용했나** 데이터는 처음부터 맞았고 엔진의 소모 지점만 빠지거나 넘쳤다. **잡은 방법** 기획자가 플레이하다 발견했다. 횡수 필드가 있는 다른 스킬을 전부 훑어 안 깎이는 것이 하나뿐임을 확인했다. 소모 조건은 플레이어의 함정이 되지 않게 정했다. 버프가 없어도 원래 갈 수 있던 칸으로 움직일 때는 깎지 않는다.

데이터와 예외 핸들러를 함께 읽고, 대조 스크립트도 본 파이프라인과 같은 정규화를 거친다

데이터 구동 시스템에도 데이터로 접하지 않는 예외 핸들러가 남는다. 동작이 두 곳에 나뉘어 있으면 한쪽만 읽은 결론도 그럴듯하다. 동작을 판단할 때는 데이터와 핸들러를 함께 읽는다. 검증용 스크립트는 본 추출기와 같은 정규화 단계(이름 정정 표 등)를 공유해야 착시 차이를 만들지 않는다.

사례 · 스킬 데이터

무슨 일이 한 도발 스킬은 데이터만 보면 적에게만 「강제 표적」 상태가 걸려서, 시전자 쪽에 같은 상태를 거는 예외를 새로 만들려 했다. 그런데 그 스킬 전용 스크립트 핸들러가 이미 시전자에게도 같은 상태를 걸고 있었다. 또 다른 스킬의 이름 어긋남은 실제 차이가 아니었다. 대조 스크립트가 본 추출기의 이름 정정 표를 거치지 않아 생긴 착시였다.

여러 표의 열쇠인 열거형에 값을 더하지 말고 새 축으로 가른다

새 변종을 들일 때 기존 열거형이나 목록을 훑는 곳이 몇 군데인지 먼저 센다. 많으면 값을 더하지 말고 새 필드(축)를 만든다. 공용 풀(명단)에 특수 엔티티를 섞으면 그 풀을 훑는 모든 기능이 그것을 뽑는다. 판 크기처럼 변종마다 다른 값은 상수가 아니라 상태에서 읽는다.

사례 · 새 게임 모드

무슨 일이 NPC 적 무리와 싸우는 방어전 모드를 들었다. 판 크기도 대전 판(25×20)과 다른 25×15였다. 판 종류를 기존 대전 모드 열거형(3인전·5인전)에 값 하나로 더하면, 그 열거형을 열쇠로 쓰는 78개 파일의 전투력 계수·보상·전적 표가 흔들렸다. NPC를 플레이어 캐릭터 명단에 넣으면 AI 상대 풀·가차·랭킹이 NPC를 뽑았다. **잡은 방법** 판 종류는 별도의 「시나리오」 필드로, NPC는 별도 목록과 전투 전용 조회 함수로 같았다. 판 안인지를 묻는 자리는 상수가 아니라 판 상태의 크기를 읽게 했다. 그래도 전투 UI 여덟 파일이 명단 전용 조회 함수를 쓰다 죽은 이야기는 → 5장 클라이언트 엔지니어.

텍스트는 그 값이 실제로 달라지는 단위에 붙인다

다국어 문장의 키는 그 값이 실제로 달라지는 단위와 같아야 한다. 같은 문장을 여러 칸에 베끼는 구조는 언젠가 한 칸만 낡는다. 문장에서 조사 하나에 깨지는 방식으로 값을 뽑지 말고, 값의 정보는 구조화된 표에 두고 문장과 대조한다. 수치를 고친 뒤 설명 문장이 따라오지 않는 문제는 → 2장 게임 디자이너에서 다룬다.

사례 · 다국어

무슨 일이 장비의 이름과 해설은 품목마다 달라서 기획 엑셀에 언어별 열로 붙였다. 효과 문장은 품목이 아니라 등급이 정한다. 이것도 엑셀 열로 두면 같은 문장을 행마다 1~3번 베끼게 된다(서로 다른 문장 81개를 135칸에). 한 행만 고치면 조용히 갈라진다. **잡은 방법** 효과 문장은 코드에 이미 있는 「등급별 효과 값 표」 옆에 두었다. 기획자는 「엑셀 열」 안을 골랐는데, 구현자가 이유를 들어 바꾼 결정이다.

기능을 없앴으면 그것만 쓰던 인자·필드·함수도 함께 지운다

남겨 둔 손잡이는 누군가 옛 동작을 조용히 다시 끼워 넣을 자리가 된다. 옛 저장 데이터에 남은 그 값은 마이그레이션이 버리고, 섞여 들어와도 읽지 않는다는 회귀를 둔다.

사례 · 기능 제거

무슨 일이 부대마다 캐릭터의 레벨을 따로 낮춰 출전시키는 기능을 없앴다. 캐릭터를 레벨업해도 부대에는 옛 레벨이 남아, 사용자에게 「레벨업이 반영이 안 된다」는 버그로 읽혔기 때문이다. 그 기능만 쓰던 레벨 상한 인자, 함수 둘, 부대 구성원의 최대 레벨 필드까지 지웠다. 옛 계정에 남은 부대별 레벨은 마이그레이션이 버리고, 전투로 넘기는 변환 함수는 그 값이 섞여 와도 읽지 않는다.

3.4 저장 형식과 마이그레이션

저장 형식의 실수는 가장 늦게 드러나고 가장 오래 남는다. 필드가 사라져도 기본값으로 돌아갈 뿐 오류가 없고, 그 값을 그리는 화면이 생겨야 비로소 보인다. 이 프로젝트는 옛 형식을 버리지 않고, 읽을 때마다 마이그레이션 함수로 새 형식으로 바꿔 쓰는 방식을 택했다. 그런데 그 함수 자체가 손실의 통로가 되기도 했다.

저장에는 결과가 아니라 원인을, 순서가 아니라 식별자를 남기고, 기본값은 「설정 없음」으로 적는다

파생값을 저장하면 공식이 바뀔 때 옛 데이터가 따라오지 못한다. 원인(사건 시각)을 저장하고 결과(완료 시각)는 읽을 때 계산한다. 설정은 배열 순서가 아니라 엔진과 같은 안정된 식별자로 키를 잡는다. 맞지 않으면 예외 대신 `null`로 기본값에 물려나 사람이 고칠 기회를 남긴다. 사용자가 기본값을 그대로 둔 것은 값이 아니라 「설정

없음」이므로 `null`로 적는다. 기본값을 실값으로 굳히면 기본 규칙이 바뀐 날 그 계정만 옛 값에 갱신한다. 쓰지 않는 식별자는 저장 형식에 넣지 않는다. 한 번 넣으면 영원히 남는다.

사례 · 건설 기록

무슨 일이 도시 건설 기록을 저장 형식에 새로 넣으며 「완료 시각」 대신 「건설을 시작한 시각」을 저장했다. 기획 엑셀의 소요 시간이 바뀌면 옛 계정도 새 값을 따라간다. 건물이 놓인 칸 번호는 쓰지 않으므로 저장하지 않고, 칸이 열린 시각 목록만 됐다. 이전 형식의 계정에는 「그 레벨에서 지을 수 있었던 최대치」를 짐작해 대신 지어 주지 않고, 사용자가 직접 고를 기회를 돌려줬다. 마이그레이션은 사용자의 선택을 대신하지 않는다.

사례 · 배치 프리셋

무슨 일이 전투 시작 전 기물 배치를 저장하는 프리셋을 위치 배열로 두면 부대 구성을 고치는 순간 어긋난다. 록을 쿼리로 바꾸면 그 뒤에 있던 비숍의 좌표가 쿼에게 붙는 식이다. 엔진의 유닛 ID가 이미 「진영 + 기물」이라 {기물, x, y}로 저장했다. 두 진영은 좌표계가 달라 따로 저장했다. 이후 배치 편집 화면에서는 기본 배치와 같은 진영을 `null`로 저장하게 했다.

저장 형식에 필드를 더하면 마이그레이션과 왕복 회귀를 함께 고친다

마이그레이션이 필드를 하나씩 옮겨 새 객체를 만드는 구조라면, 새 필드를 빠뜨린 순간 매 읽기마다 그 필드가 지워진다. 영속 데이터에 필드를 더하는 변경에는 마이그레이션 확인을 체크리스트로 묶고, 그때 기존 필드의 누락도 함께 대조한다. 「저장 → 읽기 → 마이그레이션」을 거친 뒤 필드가 남는지 보는 왕복 테스트로 고정한다. 그 값을 처음 그리는 화면이 생기기 전까지는 「동작한다」고 믿지 않는다.

사례 · 필드 누락

무슨 일이 캐릭터 한 명의 데이터를 마이그레이션하는 함수가 돌려주는 객체에 부상 관련 시각 두 개가 빠져 있었다. 서버는 계정을 읽을 때마다 마이그레이션을 거치므로, 약 2주 동안 다친 캐릭터가 다음 읽기에서 조용히 나아 있었다. 치료 건물과 부상 패널티가 사실상 동작하지 않은 것이다. **왜 조용했나** 그 값을 그리는 화면이 없었다. **잡은 방법** 새 부대 현황 화면의 「부상」 표시를 시험하려고 부상을 심었는데 「건강」이 떴다.

사례 · 필드 추가

무슨 일이 도시 이름 변경에 쿨다운을 두려고 「마지막으로 바꾼 시각」 필드를 더했다. 마이그레이션에서 빠뜨리면 쿨다운이 무의미해진다는 것을 미리 짚었다. 기존 필드 목록과 마이그레이션 코드를 한 줄씩 대조하다가, 이미 있던 가차 상태 필드도 마이그레이션을 통과하지 못하고 있음을 발견했다.

마이그레이션한 결과가 원문과 다르면 그 자리에서 다시 저장한다

읽을 때 마이그레이션한 결과를 메모리에만 올리면 저장소에는 옛 형식이 계속 남는다. 마이그레이션이 역등(여러 번 돌려도 결과가 같음)이면 동작은 멀쩡해서 아무도 모른다. 그러나 손상 복구가 굳지 않고, 형식이 또 오르면 아주 옛 버전에서 오는 경로를 언제까지고 살려 뒀야 한다. 결과가 원문과 다를 때만 다시 쓴다. 같으면 쓰지 않는다. 클라이언트 저장 층과 서버의 읽기 자리에 같은 원칙을 적용한다.

사례 · 저장 층

무슨 일이 계정을 읽는 함수가 마이그레이션 결과를 메모리에만 올려서, `localStorage` 에는 첫 버전 형식이 계속 남아 있었다. **왜 조용했나** 읽을 때마다 다시 변환돼서 화면에는 차이가 없었다. **잡은 방법** 저장 층을 검토하다 발견했다. 계정이 서버 DB로 옮겨 간 뒤에는 서버의 계정 읽기 함수에도 같은 규칙을 넣었다(→ 4장 서버 엔지니어).

상대적 변환은 버전에 걸어 한 번만 일어나게 하고, 옛 형식 입력은 손으로 적어 고정한다

「필드를 옮긴다」는 몇 번 지나도 같지만 「값을 덜어 낸다」는 두 번 지나면 두 번 깎는다. 더하기·빼기 같은 상대적 변환은 버전 번호에 걸어 한 번만 일어나게 하고, 빛(음수)을 만들지 않게 바닥을 둔다. 남는 공백은 적어 둔다. 옛 형식의 회귀 입력은 오늘 코드로 생성하지 않는다. 그 형식이 끝난 날의 실제 저장분은 영원히 그때 모양이므로, 리터럴로 적어 「그때 저장이 이랬다」를 고정한다.

사례 · 마이그레이션

무슨 일이 저장 형식을 한 버전 올리면서, 새 계정에 처음 주던 건설 기회 3회를 옛 계정에서 덜어 내야 했다. **잡은 방법** 덜어 내는 처리를 버전 번호에 걸고 0 아래로는 내리지 않게 했으며, 「한 번만 덜어 낸다」는 회귀 둘을 넣었다. 이미 그 기회를 다 써 버린 계정은 결과적으로 3회를 공짜로 받는다. 계정을 약하게 만들지 않는 쪽을 고른, 알고 남긴 공백이다.

사례 · 테스트 입력

무슨 일이 첫 버전 형식의 마이그레이션 회귀가 오늘 데이터로 캐릭터를 최고 레벨까지 키운 뒤, 그것을 첫 버전 모양으로 되돌려 입력을 만들고 있었다. 기획 변경으로 스킬 2종을 삭제하자 그 배열은 더 이상 진짜 첫 버전이 아니었다. 옛 배치 기준으로 읽으니 어긋났고, 처음엔 마이그레이션 버그로 보였다. **왜 조용했나** 데이터가 안 바뀌는 동안에는 생성한 입력과 진짜 옛 저장이 우연히 같았다. **잡은 방법** 손으로 적은 옛 배열로 바꿨다. 마이그레이션이 옛 배열을 읽어 나가는 위치도 지금 데이터의 개수가 아니라 저장 당시의 구조로 민다.

이력은 DB 한 행처럼 적고, 영구 집계는 이력과 따로 쌓는다

저장소를 옮길 계획이 있으면 데이터 모양을 처음부터 목적지에 맞춘다. 이력 한 줄은 평평하고 참조가 없게 두고, 읽는 자리를 함수 하나로 모아 나중에 그 함수만 쿼리로 갈아 끼우게 한다. 보존 기간이 유한한 이력에서 영구 집계를 파생시키지 않는다. 꼬리를 덜어 낼 때 통산이 아무 표시 없이 준다. 한 사건을 여러 단위로 세면 단위마다 별도 칸을 둔다. 시각은 도메인 로직이 아니라 호출하는 쪽이 넣는다.

사례 · 대전 이력

무슨 일이 기획자가 「프로토타입은 브라우저 저장이지만 서비스는 DB로 간다, 옮기기 쉽게 만들라」고 했다. 대전 이력 한 줄을 일련번호가 PK인 평평한 행으로 두고, 읽는 자리는 「최근 이력」과 「전적 합산」 함수 둘로 모았다. 계정 전적은 캐릭터 전적의 합이 아니라 판 수로 따로 썼다. 한 판에 캐릭터가 3~5명 뛰므로, 합을 내면 출전 수가 사람 수만큼 부풀기 때문이다.

사례 · 통산 집계

무슨 일이 부대별 전적을 붙이면서, 대전 이력(최근 200줄만 남기고 꼬리를 던다)에서 계산해 내는 안을 기각했다. 그렇게 하면 오래 쓴 부대일수록 통산이 조용히 준다. 대신 부대에 전적 필드를 따로 두고 결과를 반영할 때 함께 올렸다.

3.5 빌드와 생성 파이프라인

엑셀을 JSON으로, 원본 그림을 웹용 파일로, 소리 원본을 게임용 음원으로 바꾸는 도구는 「잘 됐다」 고만 말하고 「어떻게 됐다」 는 말하지 않는다. 260장 중 1장이 잘못돼도 눈으로는 못 찾는다. 이 절의 원칙은 도구가 조용히 하는 일을 소리 나게 만드는 것이다.

원본과 화면 사이에 변환 지점을 하나 둔다

원본의 이름·형식과 화면이 읽는 이름·형식 사이에 빌드 스크립트 하나를 둔다. 그러면 원본 교체가 화면 코드에 번지지 않는다. 단위 환산 같은 변환은 출력 직전 한 곳에서만 한다.

사례 · 에셋 파이프라인

무슨 일이 캐릭터 초상화는 원본 폴더의 {한글이름}.png → 빌드 스크립트 → 웹 폴더의 {로마자id}.png → 화면 순으로 흘렀다. 원본 폴더를 배경을 제거한 그림으로 통째로 바꿨는데 화면 코드는 한 줄도 안 바뀌었다. 스킬 설명문의 시간 단위를 엔진 단위에서 플레이어용 단위로 바꿀 때도 추출기에서만 바꿨더니 화면 네 곳이 따라왔다.

생성 도구는 말없이 지우지도, 말없이 폴백하지도 않는다

폴백은 허용하되 어느 항목이 폴백됐는지 이름을 출력한다. 자동 삭제는 잘못 판단했을 때 되돌릴 수 없으므로 「알리고 사람이 지운다」 를 기본으로 한다. 정리 기능을 꼭 뒤야 하면 그 도구가 만들 수 있는 이름 패턴만 지운다. 「내 목록에 없으면 지운다」 는 폴더를 혼자 쓸 때만 성립한다. 이름이 곧 id인 구조에서는 개명이 id·경로·파일까지 번지므로, 개명에는 생성물 폴더의 옛 파일 청소가 포함된다. 출시 뒤라면 저장 데이터 마이그레이션도 함께 간다(→ 6장 디자인 엔지니어).

사례 · 이름 변경

무슨 일이 캐릭터 id가 이름을 로마자로 옮긴 문자열이라, 캐릭터 이름 표기를 바로잡자 id·초상화 경로·웹용 파일이 따라 바뀌었다. 개명 뒤 출력 폴더에 옛 이름의 그림이 남아, 캐릭터는 260명인데 그림은 261장이었다. **왜 조용했나** 생성기는 대응하는 캐릭터가 없는 출력을 알리기만 하고 지우지 않게 되어 있었다. **잡은 방법** 출력 파일 수와 명단 수를 대조했다. 나중에 스킬 이름을 정정했을 때도 같은 고아 파일이 남았고, 추출 로그의 「이미지 41장 / 스킬 40종」 대조가 그것을 잡았다.

사례 · 공용 출력 폴더

무슨 일이 배경 그림을 굽는 도구가, UI 그림을 굽는 다른 도구가 같은 폴더에 구운 그림 하나를 「모르는 파일」 로 보고 지웠다. **왜 조용했나** 배경 그림이 404면 화면이 바탕색으로 물러나 눈에 잘 띄지 않는다. **잡은 방법** 처음에는 다른 도구를 다시 돌려 복구만 했다. 열흘쯤 뒤에야 「정리 대상은 자기가 만들 수 있는 이름 패턴으로 좁힌다」 는 규칙을 세웠다.

검증은 지금 다시 돌린 결과만 믿고, 소비자가 생긴 검사는 실패로 올린다

커밋된 빌드 리포트는 그 시점의 사실일 뿐이다. 에셋 파일명이나 원본 데이터를 바꿨으면 검증을 다시 돌린다. 대조는 양방향(이름→파일, 파일→이름)으로 한다. 사람이 쓴 표기로 조인할 때는 정규화 표를 두고, 중복 키를 경고하고, 「연결 n/전체」 를 매번 출력한다. CSV는 정규식이 아니라 진짜 파서로 읽는다. 아직 소비자가 없는 데이터의 불일치는 경고로 두되, 화면이 그 데이터를 쓰기 시작하는 날 빌드 실패로 올린다. 경고는 쌓이면 아무도

안 읽는다. 「없으면 건너된다」 규약이 있다면 건너뛴 문구도 경고로 남겨 사람이 읽는다. 그 규약은 「구조가 바뀌었다」와 「정말 없다」를 구별하지 못한다.

사례 · 빌드 리포트

무슨 일이 초상화 파일명이 바뀌었는데 데이터는 옛 이름을 가리키고 있었다. 그런데 저장소에 커밋된 빌드 리포트는 「문제 없음」이었다. **왜 조용했나** 리포트가 파일명을 바꾸기 전에 만들어진 것이었다. **잡은 방법** 추출을 다시 돌리자 양방향 대조에서 2건이 실패하고 빌드가 실패 코드로 끝났다.

사례 · 데이터 조인

무슨 일이 캐릭터 설명문 파일을 캐릭터 데이터와 한국어 이름으로 이었다. 그랬더니 표기가 다른 8명이 빠졌다. 이름이 같은 두 행 중에서는 나중 행(「위 항목 참조」라는 빈 껍데기)이 먼저 행을 덮었다. 한 사람은 자기 이름 칸만 옛 표기로 남아 있었다. **왜 조용했나** 조인 실패는 그 행만 비울 뿐 오류를 내지 않는다. **잡은 방법** 이름 정정 표를 두고, 추출 로그에 「설명문 n/260명 연결 — 누락 0명」 한 줄을 넣어 상태가 바로 보이게 했다. CSV는 셀 안 싹포 때문에 정규식이 아니라 Python `csv.DictReader` 로 읽었다.

사례 · 선택적 검증

무슨 일이 원본 그림은 저장소에 없고 각자의 컴퓨터에만 있어서, 추출기는 「그림 폴더가 없으면 그림 검사를 건너된다」는 규약을 따랐다. 그런데 라벨 그림이 하위 폴더로 옮겨지자, 추출기의 그림 대조가 「폴더가 비어 있어 건너된다」며 꺼졌다. 규약이 정상 동작한 결과라 실패가 아니었다. **잡은 방법** 리포트의 건너뛴 문구를 읽었다. 그보다 앞서 스킴 연출 그림과 스킴 이름의 대조는 경고로 두었다. 8건 어긋남이 기획자의 정정 뒤 0건이 됐고, 화면이 그 그림을 쓰기 시작한 날 실패로 올랐다.

증분 빌드는 파일 시각이 아니라 내용으로 판단한다

바뀐 것만 다시 만드는 증분 빌드에서 「이미 최신이면 건너된다」를 파일 시각이나 출력의 존재로 판단하면 원본 교체를 못 본다. 내용 해시로 비교하고 같아 끼운 항목의 이름을 출력한다. 출력이 여러 벌이면 전부 있어야 건너된다. 시각 기준으로 남겨 둔 도구는 그 한계를 문서에 적고 강제 재생성 옵션을 둔다. 다만 늘 강제 옵션으로만 돌리면 기본 경로의 버그가 숨으니, 기본 경로도 가끔 돌린다. 확인이 어려운 산출물(소리)일수록 엄격하게 한다.

사례 · 사운드 빌드

무슨 일이 기획자가 배경음악 3번과 4번 곡의 내용을 맞바꿔 저장했는데 파일 시각은 그대로였다. 「출력이 원본보다 새로우면 건너된다」인 도구가 아무것도 안 했고, 게임에서는 옛 곡이 계속 나왔다. **왜 조용했나** 그림은 바뀌면 눈에 띄지만 소리는 그렇지 않고, 도구는 「건너뛴」을 정상으로 보고한다. **잡은 방법** 기획자가 들어 보고 알렸다. 같은 종류로, 초상화 도구는 출력이 있지만 하면 건너뛰어서 같은 이름으로 바로잡은 원본을 반영하지 못했다. 다른 그림 도구에는 출력이 이미 있을 때만 죽는 버그가 있었는데, 늘 강제 옵션으로만 돌려 와서 아무도 못 봤다.

사람이 관리하는 파일은 통째로 다시 쓰지 말고 바꿀 부분만 바꾼다

파일을 파싱했다가 다시 직렬화하는 라이브러리는 「같은 내용」을 저장했다고 여기지만, 사람이 넣은 구조(빈 줄 묶음, 내부 테이블, 줄끝 문자)를 지운다. 사람이 관리하는 파일을 도구로 고칠 때는 있는 키는 그 줄의 값만 바꾸고 새 키는 끝에 붙이는 텍스트 수준 편집을 쓴다. 고친 뒤에는 diff가 의도한 줄뿐인지 본다. 줄끝은

`.gitattributes (eol=lf)`로 고정한다.

사례 · 데이터 파일 편집

무슨 일이 Python openpyxl로 기획 엑셀을 열어 전체를 다시 저장하자, xlsx 안의 공유 문자열 표 (sharedStrings.xml)가 통째로 사라져 직접 만든 xlsx 파서가 깨졌다. 다국어 문구 JSON 파일을 Python json.dump 로 다시 쓰자 사람이 넣어 둔 빈 줄 구분이 사라졌다. **잡은 방법** 둘 다 되돌렸다. 엑셀은 공유 문자열 표의 문자열만 치환하고, JSON은 꼬리에 덧붙이는 방식으로 바꿨다. 그 뒤에도 같은 일이 세 번 더 났다(json.dumps 와 텍스트 모드의 CRLF, JavaScript JSON.stringify 등). 매번 파일 전체가 diff로 뜨는 것을 보고 되돌렸다.

실행 환경이 받아 주는 문법과 모듈 로딩의 한계를 설계 제약으로 적는다

번들 없이 소스를 바로 실행하는 런타임을 쓰면 허용 문법 목록과, 그 때문에 못 쓰는 라이브러리를 규칙 문서에 적는다. 테스트·서버·번들러가 서로 다른 변환기를 쓰면 공용 코드는 가장 엄격한 쪽에 맞춘다 (→ 5장 클라이언트 엔지니어). 순환 import가 가능한 모듈끼리는 최상단 상수를 주고받지 말고, 두 모듈이 같은 원래 출처(설정 JSON)를 각자 읽는다. 선언 전에 변수를 읽어 터지는 문제(TDZ, Temporal Dead Zone)를 피하기 위해서다.

사례 · 런타임 제약

무슨 일이 이 프로젝트는 TypeScript를 빌드 없이 Node의 타입 스트리핑으로 바로 실행한다. 테스트용 가짜 서버 객체를 constructor(readonly initial: State, ...) 로 적자 ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX - parameter property is not supported in strip-only mode 가 났다. 타입 스트리핑은 타입만 지우면 되는 문법만 받아, 데코레이터·enum·namespace 도 막는다. 데코레이터 기반 상태 동기화 라이브러리를 피한 것이 취향이 아니라 물리적 제약이었음이 이때 확인됐다.

사례 · 순환 import

무슨 일이 연구 시스템 모듈이 계정 모듈의 상수를 가져오면, 계정 → 도시 → 연구 → 계정 순환에서 그 상수가 초기화되기 전에 읽힐 위험이 있었다. **잡은 방법** 의존 순서를 따져 보고, 두 모듈이 같은 설정 출처를 각자 읽게 해 미리 피했다.

3.6 측정과 회귀

밸런스를 재는 시뮬레이션과 통계 도구는 언제나 그럴듯한 숫자를 내놓는다. 그 숫자가 신호인지 잡음인지, 「효과 없음」인지 「안 돌았음」인지는 말해 주지 않는다. 테스트도 마찬가지다. 통과는 코드가 맞다는 뜻이 아니라 검사가 그 결함에 둔감하다는 뜻일 수 있다.

회귀 계수는 표준오차와 함께 읽고, 오차 안의 움직임으로 상수를 바꾸지 않는다

통계 도구는 계수·표준오차·z를 항상 함께 출력한다. 설명 변수끼리 서로 거의 같은 정보를 담는 공선(共線)은 계수 값이 아니라 오차에 나온다. 범주형 지시변수의 합이 상수가 되는 구조라면 기준 범주 하나를 뺀다. 긴 측정은 터미널이 아니라 파일로 받는다. 출력 앞부분이 잘려도 계수 표는 멀쩡해 보인다. 운영에 쓰는 계수는 실측을 손으로 옮겨 적은 상수로 고정하고, 변화가 오차 범위를 넘을 때만 의도적으로 갱신한다. 갱신할 때는 항을 골라 섞지 말고 통째로 반영한다. 절편이 모든 항과 함께 적합되었기 때문이다. 문턱식 선택 규칙($|z| < 2$ 면 0)의 결과는 「값이 변했다」와 「문턱에 걸렸다」를 구분해 읽는다.

사례 · 전투력 모형

이 게임은 매칭에 쓰려고 부대의 「전투력」을 정의했다. 수만 판의 AI 대전 결과로 승패를 회귀해, 어떤 구성이 얼마나 이기는지 계수로 켜 값이다. **무슨 일이** 첫 적합에서 기물 종류별 계수가 $+0.08$, -0.20 처럼 그럴듯했지만 표준오차가 6435였다. 진영마다 기물 수가 고정이라 다섯 지시변수의 합이 상수, 즉 완전 공선이었다. **왜 조용했나** 계수 값만 보면 멀쩡했다. **잡은 방법** 표준오차를 출력해 봤다. 기준 범주(폰)를 빼서 각 계수를 「폰 대신 이 기물을 넣으면」으로 읽게 고쳤다.

사례 · 재측정

무슨 일이 전투력을 다시 적합하는 측정(모드당 2만 판, 522초)의 첫 실행에서 출력 앞부분이 잘려, 계수만 보고 판단할 뻔했다. 크게 움직인 MP 항(35~39%)은 z 2.1~2.7로 잡음이었고, 실제로 움직인 것은 스킬 보유 항 하나(z 4.7)였다. 이후 기물 규칙을 개편한 뒤 2만 판 $\times$ 2로 다시 켜 때는, 계수가 전부 표준오차 1배 안팎에서만 움직여서 상수를 옮겨 적지 않았다. 한 계수가 0.60에서 0이 된 것은 $|z|$ 1.3이 문턱에 걸린 결과였다. 200판짜리 빠른 측정은 기물당 표본이 약 155회라 $\pm 4\%p(48.2 \rightarrow 52.3\%)$ 씩 흔들려서 판단에 쓰지 않았다.

모형을 관측 범위 밖으로 외삽하지 않고, 테스트에 박은 직관도 실측으로 검증한다

회귀는 관측된 범위 안에서는 잘 맞고, 관측 범위 밖으로 값을 늘려 추정하는 외삽에서 틀린다. 입력값이 표본에 없는 범위에 있으면 연속값 대신 관측된 범주별로 켜다. 테스트에 박아 둔 가정이 실측과 어긋나면 먼저 가정이 맞는지 의심하고, 틀렸으면 테스트를 사실 쪽으로 고친다.

사례 · 전투력 모형

무슨 일이 캐릭터 고유 스킬을 「보유 여부 + 스킬 비용」 연속값으로 모형에 넣었다. 그런데 표본의 비용은 4·5·6뿐이었고 단 한 캐릭터만 7이었다. 모형이 그 7을 외삽하자, 실측 최약체(승률 35.8%)가 최하 등급 캐릭터보다 세게 나왔다. 거의 공선인 두 항이 서로를 밀어내기도 했다($-52.6 / +14.3$). 같은 주에 테스트에 박아 둔 직관 「5인전 전투력 > 3인전 전투력」, 「하급 셋 < 중급 셋 < ...」도 실측에 두 번 깨졌다. **잡은 방법** 개별 캐릭터의 예측값을 실측 승률과 대조했다. 연속값 대신 등급 더미 변수 셋으로 바꿔, 관측된 값만 각각 재게 했다. 대가로 적합도(로그우도)가 101 떨어졌다.

「변화 없음」이 나오면 먼저 측정 실패를 의심하고, 새 행동은 분포부터 찍는다

소수점까지 같은 수치는 효과가 없다는 뜻이 아니라 기능이 실행되지 않았다는 신호로 본다. 새 행동(AI의 스킬 사용 등)을 붙이면 승률보다 사용 횟수와 대상 분포를 먼저 확인한다. 기준선 AI의 낭비가 기능 전체의 값어치를 낮게 보이게 할 수 있다. 경제 시뮬레이션이 이상한 숫자를 내면 규칙보다 「플레이어 전략」 모델부터 의심하고, 「돈을 더 쓰면 더 빠르다」처럼 단조여야 할 것이 단조인지 본다($\rightarrow$ 2장 게임 디자이너).

사례 · 밸런스 시뮬레이션

무슨 일이 레벨업 때 능력치를 MP에 몰아 준 캐릭터가 레벨 1 캐릭터와 소수점까지 같은 승률을 냈다. AI가 MP를 쓰는 스킬을 아예 쓰지 않고 있었다. AI에 스킬 사용을 붙이자 이번에는 300판 동안 체력이 가득 찬 아군에게 큰 회복 스킬을 2595회 쓰는 낭비가, 기능의 값어치를 낮게 보이게 했다. 체력이 가득 찬 대상을 거르자 912회로 줄었다. **잡은 방법** 소수점까지 같은 값을 이상하게 여겼다.

수치를 맞추는 함수의 회귀는 전 구간을 훑고, 계수가 바뀌면 그 가정을 다시 낸다

대표점 몇 개만 재는 회귀는 범위의 끝을 놓친다. 입력 전 구간을 촘촘히 훑는다. 테스트가 느려지면 성글게 되돌리지 말고 규칙 쪽의 낭비를 먼저 찾는다. 회귀를 촘촘히 하는 일과 코드를 빠르게 하는 일이 같은 작업일 때가 있다. 계수나 분포가 바뀌면 값에 기대던 가정이 깨진다. 계산된 실수에 「정확히 같다」 단언을 쓰지 않고, 경계를 훑는 범위를 다시 재고, 무작위성을 검사하는 입력이 우연히 결정론적인 점에 놓이지 않았는지 본다.

사례 · AI 상대 생성

무슨 일이 플레이어 부대의 전투력에 맞춰 비슷한 세기의 AI 상대 부대를 만드는 함수가 있었다. 이 함수가 목표 여섯 점에서는 통과했는데, 전 구간을 훑자 바닥 근처(목표 250)에서 31이 벗어났다(허용 ± 20). 탐욕적으로 슬롯을 채우다 보니 오차가 마지막 슬롯에 몰린 탓이다. 훑기로 바꾸자 `npm test` 가 0.8초에서 14초가 됐다. **왜 조용했나** 고른 점이 모두 범위의 가운데였다. **잡은 방법** 느린 원인(매번 후보를 다시 만들고 전부 정렬)을 상위 k개만 고르는 방식으로 고쳐 0.42초가 됐다. 보정 뒤 최대 오차는 11·20이고 허용 범위를 벗어난 점은 0건이다.

사례 · 계수 교체

무슨 일이 새로 낸 전투력 계수를 바꿔 끼우자 회귀 셋이 깨졌다. 첫째, 덧셈 누적 순서 때문에 바닥값 20이 19.999999999999997 이 됐다. 둘째, 매칭 회귀가 훑는 범위의 상한이 새 계수로 도달할 수 있는 범위를 넘었다. 셋째, 특정 목표 전투력을 맞추는 조합이 새 계수에서 사실상 하나뿐이라, 「시드가 다르면 다른 상대가 나온다」 검사에서 두 시드가 같은 부대로 수렴했다. **왜 조용했나** 옛 계수에서는 세 가정이 모두 우연히 참이었다. **잡은 방법** 단언을 허용오차 $1e-9$ 비교로 바꾸고, 실제 스캔으로 안전 범위를 다시 재고, 조합이 여럿인 목표값으로 옮겼다.

검사는 구현의 키가 아니라 결과를 재고, 일부러 깨서 실패하는 것을 본다

재는 축을 잘못 고르면 검사가 거짓 경보를 낸다. 실패를 데이터 문제로 보기 전에 검사가 무엇을 재고 있는지 다시 본다. 효과 키 단위 비교 대신 기대 데미지처럼 사용자가 겪는 결과를 넓은 입력 범위로 훑는다. 검사가 걸린 기존 성질은 지우지 말고 주석으로 남긴다. 회귀를 넣으면 대상 코드를 일부러 깨서 실패하는지 본다. 안 깨지면 입력이 그 결함에 둔감한 것이니, 작은 차이가 결과를 실제로 뒤집는 경계 입력을 넣는다. 이 원칙의 일반형은
→ 1장 모든 직무 공통에서 다룬다.

사례 · 장비 등급

무슨 일이 장비의 「상위 등급은 하위 등급의 효과를 모두 포함한다」를 효과 키별 비교로 적었다. 그러자 한 등급을 실패로 잡았다. 그 등급은 「일반 공격 +1」을 「크리티컬 +2」로 바꾸면서 하위 등급의 「크리티컬 피해 +1」을 흡수했는데, 키가 사라졌으니 「포함하지 않는다」로 본 것이다. 「일반 공격 피해가 0이 되지 않는다」 검사는 장비와 상관없는 기존 성질(피해 반감과 공포 상태가 겹치면 0)에 걸렸다. **잡은 방법** 키 비교를 기대 데미지 비교로 바꾸고 공격력 7중 × 크리티컬 확률 5종을 훑었다. 기존 성질은 지우지 않고 주석으로 남겼다.

사례 · 큐레이션 점검

무슨 일이 차례 순서를 예보하는 함수에 회귀를 넣고, 예보 코드의 네 군데(시간 경과, 대기 시간 보정, 동점 순번, 상태 처리 순서)를 일부러 깨 봤다. 그중 둘(시간 경과, 보정)이 안 잡혔다. **왜 조용했나** 작은 보정은 차례의 순서를 바꾸지 않았고, 검사는 순서만 비교했다. 틀린 코드도 같은 답을 냈다. **잡은 방법** 작은 차이가 순서를 실제로 뒤집는 경계 입력을 만들어 넣었다.

작업 전후 점검표

- ☐ 새로 쓴 규칙 함수가 `Date.now()` 나 `Math.random()` 을 직접 부르지 않는가? 시각과 시드를 인자로 받는가?
- ☐ 시간으로 차는 값에 자투리 보존·상한에서의 시각 전진·뒤로 간 시계 무시·0/누락값 처리가 모두 있는가?
요율이 바뀌는 함수가 먼저 정산하는가?
- ☐ 나중에 일어날 일을 예약했다면, 그 시각을 앞당기거나 미루는 수단이 게임에 있는가?
- ☐ 화면이 보여 주는 판정·예보·버튼 상태가 엔진 함수에서 오는가? 그 회귀의 정답은 엔진을 실제로 돌린 결과인가?
- ☐ 보정이 붙을 수 있는 능력치를 읽는 자리가 접근 함수 하나뿐인가?
- ☐ 거리·반경·사용 불가 조건이 분기 밖 맨 앞에서 판정되는가? 그 갈래에 들어가지 않는 스کیل에서도 평가되는가?
- ☐ 「아무도 못 움직인다」 같은 불가능한 상태가 무승부·0·빈 목록으로 새고 있지 않은가?
- ☐ 데이터에 새 필드를 적었다면 그것을 읽는 줄과 짝는 줄을 코드에서 실제로 찾았는가? 타입에 그 필드가 있는가?
- ☐ 열거형에 값을 더하기 전에 그 열거형을 훑는 곳이 몇 군데인지 켜는가?
- ☐ 기능을 없앴다면 그것만 쓰던 인자·필드·함수까지 지웠는가?
- ☐ 저장 형식에 필드를 더했다면 마이그레이션이 그 필드를 통과시키는지 「저장 → 읽기 → 마이그레이션」 왕복 테스트로 확인했는가?
- ☐ 마이그레이션에 덜어 내기·더하기 같은 상대적 변환이 있다면 버전에 걸려 있고 바닥이 있는가? 옛 형식 테스트 입력이 손으로 적은 리터럴인가?
- ☐ 원본을 바꾼 뒤 검증 파이프라인을 다시 돌렸는가? 생성 도구가 폴백·건너뛴·고아 파일을 이름으로 찍었고, 그 출력을 읽었는가?
- ☐ 사람이 관리하는 파일을 도구로 고친 뒤 diff가 의도한 줄뿐인가?
- ☐ 통계 결과를 계수만이 아니라 표준오차·z와 함께 봤는가? 관측 범위 밖의 입력은 없는가? 새 회귀를 일부러 깨서 실패하는 것을 봤는가?

4 장

서버 엔지니어

누가 무엇을 믿을지 정하고, 그 믿음을 지키는 사람

이 장은 온라인 게임의 서버·계정 API·DB를 맡는 사람을 위해 썼다. 사례는 서버가 판정하는 온라인 대전과 AI 대전, 가차와 캐릭터 육성, 시간이 지나면 차오르는 자원과 도시 건설 같은 메타 경제가 있는 턴제 전술 게임에서 나왔다. 이 자리에서 가장 자주 조용히 틀리는 것은 **버려지는 값**이다. 서버가 클라이언트 값을 무시하기 시작하면 오류는 하나도 안 나고 효과만 사라진다. 그다음은 **동시에 도착하는 요청**이다. 두 요청이 모두 200을 받았는데 한쪽 결과가 없어진다. 마지막은 **서버 밖의 것**이다. 브라우저, 클라우드 대시보드, 라이브러리 버전은 타입 검사도 단위 테스트도 볼 수 없다. 셋 다 로그에 아무것도 남기지 않는다는 점이 같다.

이 장의 원칙

1. 필드를 서버 소유로 돌리면, 그 필드를 바꾸던 사건마다 서버 경로를 함께 만든다
2. 신뢰 경계가 다른 저장 함수는 이름부터 가른다
3. 통짜 저장에서 「왜 바뀌었나」를 역추적하지 말고, 행위마다 전용 경로를 만든다
4. 권한이 서버로 옮겨 오면 「통과시키던」 회귀를 뒤집는다
5. 권위 규칙은 DB 없이 회귀가 도는 순수 모듈에 둔다
6. 결정적 롤 엔진이 있으면 판정 주체를 상주시키지 말고 끝난 뒤 재생으로 검증한다
7. 재생에 들어가는 시각은 함께 보내고, 받는 타입에 필드로 적고, 범위로 묶는다
8. 재생 입력은 관측 시점과 무관한 결정적 값으로 만든다
9. 「가끔 값이 되돌아간다」면 잠금 없는 읽기-고치기-쓰기를 먼저 의심한다
10. 고유성은 미리 조회해서 막지 말고 DB 고유 인덱스로 보장한다
11. 여러 개를 한 번에 사는 동작은 원자적 요청 하나로 만든다
12. 순서가 뜻을 갖는 자료구조에 우선권을 넣을 때는 나머지 항목의 자리까지 정한다
13. 동점이 가능한 「최고값 고르기」에는 명시적인 2차 정렬 키를 둔다
14. 실시간 전송에는 로그를 뺀 스냅샷과 이벤트를 싣고, 마감은 남은 시간으로 보낸다
15. 메시지 리스너는 요청을 보내기 전에 건다
16. 네트워크 라이브러리의 버전 짝은 진짜 소켓을 여는 스모크로 확인한다
17. 기존 경로를 그대로 탄다고 가정한 새 입력도 끝에서 끝까지 흘러 본다
18. 404를 「서버에 못 닿음」으로 읽지 않는다
19. 폴백은 그 수에서 읽는 쪽이 없을 때만 한다
20. 오류 코드의 뜻을 확인하기 전에는 설정을 바꾸지 않는다
21. 개발용 치트 통로는 기본이 닫혀 있고, 닫혀 있을 때는 이유를 말한다
22. 사람용 오류 문장은 그대로 두고, 번역용 코드와 매개변수를 덧붙인다

23. 클라우드 설정 값은 「형식이 맞는가」가 아니라 「실제로 통하는가」로 확인한다
24. 외부 서비스의 설정은 코드 주석이 아니라 실제 응답 모양으로 방어한다
25. CORS 허용 메서드는 명시하고, 라우트를 더할 때마다 다시 본다
26. 신원과 계정 데이터를 같은 저장소에 두어 신뢰 경계를 없앤다
27. 비밀번호는 앱이 저장하지 않는다
28. 서버가 반영했다는 검사는 서버 값을, 부수 효과 없는 경로로 읽는다
29. 서버를 고쳤으면 떠 있는 프로세스가 그 코드인지부터 확인한다
30. 패키지 진입점과 프로세스 부팅을 분리한다

4.1 서버 권위의 경계

서버 권위(server authority)란 게임 상태의 정답을 서버가 정하고, 클라이언트의 주장은 검증한 뒤에만 받아들이는 구조다. 유료 가차와 랭킹이 있는 게임에서 클라이언트가 보낸 값을 그대로 저장하면 그것이 곧 치팅 경로다. 그런데 「클라이언트 값을 버린다」는 결정은 그 자체로 새 버그를 만든다. 버려진 값은 오류 없이 사라지기 때문이다.

필드를 서버 소유로 돌리면, 그 필드를 바꾸던 사건마다 서버 경로를 함께 만든다

서버가 필드를 보호하기 시작하면 그 필드를 로컬로 고치던 모든 경로가 효과를 잃는다. 버려진 값은 다음 왕복에서 서버 값으로 덮일 뿐이라 화면은 잠깐 맞아 보인다. 그래서 「버린다」를 넣기 전에 그 값을 늘리거나 줄이던 사건의 목록부터 만들고, 사건마다 서버 쪽 반영 경로를 만든다.

사례 · 서버 권위 전환

계정 전체를 한 덩어리로 올리는 저장 API(`put /profile` 같은 형태)가 있었다. 여기서 대전 입장에 드는 소모 자원을 클라이언트가 마음대로 늘리지 못하게, 서버가 그 값을 무시하고 자기 값으로 되쓰도록 바꿨다. **무슨 일이** 클라이언트가 로컬로 반영하던 무승부 보상(자원을 고르는 선택지)이 사라질 뻔했다. 무승부 정산 전용 경로를 새로 만들었다. 2주 뒤에는 AI 대전 입장료가 클라이언트에서만 빠지고 있었다. AI 대전에는 서버 쪽 방이 없어 입장료를 걷을 주체가 없었던 것이다. 화면에서는 줄었다가 새로고침하면 되살아났다. **왜 조용했나** 그 장면을 볼 수 있던 스모크 테스트(실제 화면을 띄워 끝까지 눌러 보는 자동 검사)는 앞 단계에서 이미 죽어 있었다. **잡은 방법** 이후 도시 건설 관련 필드를 서버 소유로 돌리며 「건설·입장료·재료 구매, 경로 셋을 함께 만든다」를 세우는 순간 같은 구멍이 보였다.

신뢰 경계가 다른 저장 함수는 이름부터 가른다

「클라이언트가 보낸 것을 저장한다」와 「서버가 계산한 것을 저장한다」는 같은 테이블에 쓰지만 보호 규칙이 정반대다. 둘이 같은 함수에 플래그 하나로 갈리면 잘못 고르기 쉽다. 이름을 갈라 두고, 깨지기 쉬운 자리로 문서에 적는다.

사례 · 저장 API 설계

무슨 일이 서버의 계정 저장 함수가 돌아 됐다. 하나는 클라이언트의 저장 요청용으로 서버 소유 필드를 보호한다. 다른 하나는 서버가 직접 계산한 보상을 쓰는 용도로 보호하지 않는다. 서버가 계산한 보상을 앞쪽에 넣으면 보상이 사라지고, 클라이언트 입력을 뒤쪽에 넣으면 보호가 통째로 뚫린다. **왜 조용했나** 보호되는 필드는 조용히 무시되므로 오류 없이 값만 원래대로 돌아간다. **잡은 방법** 두 함수의 차이를 이름에 담고(뒤쪽에 「신뢰」를 뜻하는 접미사를 붙였다), 프로젝트 지침에 「이름이 비슷해서 잘못 골라 쓰면 다시 뚫린다」고 적었다.

통짜 저장에서 「왜 바뀌었나」를 역추적하지 말고, 행위마다 전용 경로를 만든다

전체 상태를 한 번에 받는 저장은 「무엇이 바뀌었나」만 알고 「왜 바뀌었나」는 모른다. 차이를 보고 원인을 추론하는 검증은 여러 행위가 섞이면 조합이 폭발한다. 필드를 서버 소유로 돌리고 행위 단위로 서버가 계산하는 것이 가장 단순하다. 난수가 필요하면 시드도 서버가 낸다.

사례 · 설계 비교

캐릭터 레벨업과 카드 정리(남는 육성 카드를 교환하는 기능)를 서버에서 검증해야 했다. **무슨 일이** 안 넷을 비교했다. A 행위마다 전용 POST, B 통짜 저장의 전후 차이 검증, C 행위 기록을 받아 재생, D 「이 값은 이 범위를 넘을 수 없다」 같은 불변식으로 잘라 내기. **판단** B는 한 번의 저장에 여러 행위가 섞이면 조합이 폭발한다. C는 순서·오프라인 큐·부분 실패를 새로 설계해야 한다. D는 값의 출처를 못 봐 반쪽이다. **결과** A를 단계적으로 채택했다. 유료 재화와 가치를 먼저, 육성 카드와 캐릭터 명단을 그다음에 옮겼다.

권한이 서버로 옮겨 오면 「통과시키던」 회귀를 뒤집는다

회귀 테스트(한 번 고친 동작이 다시 깨지지 않게 고정하는 자동 검사)가 과도기의 타협을 정상 동작으로 고정하고 있으면, 그 줄이 곧 보안 구멍이고 늘 초록이다. 행위가 서버로 옮겨 오면 해당 필드를 통째로 서버 소유로 돌리고 회귀를 「통짜 저장으로 온 값은 저장되지 않는다」로 뒤집는다. 그 뒤 그 필드를 통짜 저장으로 심던 테스트가 깨지는 것은 검사가 틀린 게 아니라 통로가 막힌 것이다. 신뢰 경로로 심게 고친다.

사례 · 권한 회귀

무슨 일이 「레벨업을 통과시키고 부상만 지킨다」는 권한 회귀가 있었다. 레벨업을 클라이언트가 계산하던 시절의 타협이었다. 그 틈으로 캐릭터의 레벨·육성 카드·명단을 저장 API로 마음대로 적을 수 있었다. **왜 조용했나** 회귀가 바로 그 허용을 검사하고 있었다. **잡은 방법** 레벨업을 서버 전용 경로로 옮기며 「클라이언트가 바꿀 수 있는 값이 아직 남는가」를 다시 훑었다.

권위 규칙은 DB 없이 회귀가 도는 순수 모듈에 둔다

「어느 필드가 서버 소유인가」같은 보안 규칙도 I/O 없는 순수 모듈로 빼야 단위 테스트가 선다. 서버는 그 모듈을 불러 쓰기만 한다. 규칙을 고정하는 픽스처(테스트용 고정 데이터)가 있다면, 새 서버 소유 필드를 더할 때마다 픽스처에도 값을 넣어야 검사가 헛거워지지 않는다.

사례 · 모듈 경계

무슨 일이 서버 소유 필드 목록을 계정 API 서버 패키지가 아니라, 클라이언트와 서버가 함께 쓰는 도메인 로직 패키지에 뒀다. 서버 패키지 쪽에 두면 DB 없이는 회귀를 돌릴 수 없다. 나중에 제작 아이템을 서버 소유로 더할 때는 같은 픽스처에 그 필드 값을 함께 넣었다. 비워 두면 「이 필드는 저장되지 않는다」를 확인하는 검사가 빈 값을 비교하며 통과해 버린다.

4.2 결정적 재생으로 얻는 권위

서버 권위를 얻는 방법이 「서버에 판을 상주시킨다」 하나만은 아니다. 룰 엔진이 결정적(deterministic, 같은 입력이면 언제 어디서 돌려도 같은 결과)이면 판이 끝난 뒤 입력만 받아 다시 계산할 수 있다. 다만 그 입력에 시각이 섞이면 결정성이 조용히 깨진다.

결정적 룰 엔진이 있으면 판정 주체를 상주시키지 말고 끝난 뒤 재생으로 검증한다

엔진에서 `Date.now()` 와 `Math.random()` 을 몰아내고 난수는 시드 PRNG(시드에서 정해진 순서로 난수를 내는 생성기)만 쓰게 해 두면, 클라이언트가 사람의 의도 순서만 보내도 서버가 같은 시드로 상대 AI의 수까지 처음부터 다시 계산할 수 있다. 판마다 방을 붙들지 않는 「한 번 계산하고 끝나는 요청」이라 부하가 판 길이와 동시 접속자 수에 얽매이지 않는다. 능력치·레벨 같은 입력은 클라이언트가 자칭하지 못하게 서버가 계정에서 다시 만든다. 판정 주체가 이미 서버라면(온라인 대전 방) 재생도 필요 없다. 그 방이 결과를 결정한 순간 방을 감싼 서버 코드가 내부 API를 직접 부르고, 그 호출이 실패해도 판은 멈추지 않게 한다.

사례 · 서버 부하

무슨 일이 AI 대전을 서버 권위로 옮기는 첫 안은 실시간 대전 방에 봇 자리를 앉히는 것이었다. 온라인 대전과 똑같은 방·시계를 쓰니 구현은 가장 쉬워 보였다. 기획자가 지적했다. AI 대전은 온라인 상대가 없을 때의 기본 경로라 훨씬 자주 열리는데, 봇 방은 판마다 살아 있는 방과 250ms 시계를 판이 끝날 때까지 붙든다. **결과** 판이 끝난 뒤 의도 기록을 받아 처음부터 다시 돌리는 재생 함수로 바꿨다. 재생 회귀를 쓰다 로그가 어긋났는데, 실제 로컬 대전을 AI가 먼저 자동으로 준비를 마치고 테스트는 사람이 먼저였다. 엔진이 아니라 테스트 쪽 버그였다.

재생에 들어가는 시각은 함께 보내고, 받는 타입에 필드로 적고, 범위로 묶는다

시간이 지나면 변하는 상태(부상 회복, 연구 완료)는 「지금」으로 재생하면 실제 판과 다른 결과가 나온다. 판이 시작된 시각을 재생 입력에 넣는다. 클라이언트가 주장하는 시각은 허용 범위로 눌러 받고, 속여서 얻는 것이 무엇인지 따져 본다. 그리고 그 값이 받는 함수의 타입에 필드로 선언돼 있는지 확인한다. TypeScript에서 스프레드로 넣은 필드는 초과 속성 검사를 받지 않아, 타입에 없으면 조용히 버려진다.

사례 · 재생 검증

이 게임의 캐릭터는 전투에서 다치고, 실제 시간이 지나면 낫는다. 다친 채로 나가면 약하게 싸운다. **무슨 일이** AI 대전 재생이 판이 끝난 뒤의 「지금」으로 부상을 계산해, 정상 플레이를 거부할 뻔했다. 판 시작 시각을 함께 보내고 서버가 [지금 - 2시간, 지금]으로 눌러 받게 했다. 속여서 얻는 것은 「나을 때까지 미룬다」뿐인데, 그 시각이면 실제로 나온 것이다. 18일 뒤, 서버 라우트가 그 시각을 `...{ startedAt }` 식의 스프레드로 요청 객체에 넣는데 받는 타입에 그 필드가 없어 정산 함수가 **한 번도 읽지 않았다**는 것이 드러났다. 그동안 서버는 부상을 안고 싸운 판을 부상 없이 재생하고 있었다. **왜 조용했나** 타입 검사도 회귀도 통과했다. **잡은 방법** 다른 값을 같은 길로 태우려고 코드를 열어 보다 발견했다.

재생 입력은 관측 시점과 무관한 결정적 값으로 만든다

서버가 판을 다시 계산해 검증한다면, 그 판에 영향을 주는 모든 입력은 「언제 들었나」에 따라 달라지면 안 된다. 「언제 정산했나」가 아니라 「언제 끝났나」를 저장한다. 시각이 아니라 상태인 입력은 이 방법이 통하지 않으니 공백으로 적어 둔다.

사례 · 재생 검증

스킬을 강화하는 연구는 시작하고 1시간 뒤에 끝난다. 끝난 연구는 다음 판부터 강화된 스킬로 싸운다. **무슨 일이** 연구가 전투 도중에 끝나면 클라이언트는 원본 스킬로, 서버는 강화형으로 재생해 정상 플레이가 거부될 수 있었다. 완료 시각을 「서버가 정산한 시각」으로 찍으면 사람이 언제 들었느냐에 따라 값이 달라진다. **잡은 방법** 완료 시각을 「시작 + 1시간」으로 고정하고, 판의 시작 시각까지 끝나 있던 연구만 실게 했다. 장착 장비는 시각이 아니라 상태라 이 방법이 안 통해 공백으로 적어 뒀다.

서버가 검증할 수 없는 값을 나르는 대신 그 값이 필요 없는 규칙으로 바꾸는 것, 불리한 이벤트를 피할 동기가 없게 보상 표를 짜는 것은 기획과 함께 푸는 문제다. → 2장 게임 디자인어

4.3 동시성 · 순서 · 원자성

서버는 두 요청 모두에 200을 돌려줬는데 한쪽 결과가 사라지는 일이 있다. 순서와 동점은 작은 표본에서는 우연히 맞는다. 이 절의 버그는 전부 「가끔」 나타났고, 「가끔」은 재현하기 전까지 다른 원인으로 기록되기 쉽다.

「가끔 값이 되돌아간다」면 잠금 없는 읽기-고치기-쓰기를 먼저 의심한다

한 계정에 두 쓰기가 동시에 오면, 잠금 없는 「읽고 → 고치고 → 쓰기」는 늦게 끝난 쪽이 자기가 읽은 옛 값으로 앞의 결과를 덮는다. 계정 쓰기를 전부 행 잠금(`SELECT ... FOR UPDATE`, 트랜잭션이 끝날 때까지 다른 쓰기가 그 행을 못 읽게 막는 것)을 지나는 단일 함수로 모은다. 트랜잭션만으로는 막히지 않는다. 잠금을 빼고 다시 재서 잠금이 실제로 일을 한다는 것까지 확인한다.

사례 · 동시 쓰기

온라인 매칭에서 상대를 거절하고 [다시 찾기]를 누르면 입장 자원을 1 깎는 벌칙이 있다. 이 벌칙은 매칭 서버가 계정 서버를 직접 불러 깎는다. **무슨 일이** [다시 찾기] 한 번에 화면의 통짜 저장 요청과 매칭 서버의 내부 차감 요청이 동시에 나갔다. 통짜 저장이 자기가 읽어 둔 값(4)으로 되어서 거절 차감을 삼켰다. 12일 동안 「거절 경로의 별도 문제」로 기록돼 있었다. **왜 조용했나** 두 요청 모두 200이었다. 서버 소유 필드 보호는 「자기가 읽은 값」만 지켜서, 읽고 쓰는 사이에 끼어든 다른 쓰기를 몰랐다. **잡은 방법** 같은 계정에 두 호출을 동시에 쏘는 스크립트로 썼다. 고치기 전 15번 중 1번 유실, 행 잠금 후 40번 중 0번, 트랜잭션은 두고 FOR UPDATE 만 빼면 40번 중 37번 유실이었다.

고유성은 미리 조회해서 막지 말고 DB 고유 인덱스로 보장한다

「조회 → 없으면 쓰기」는 그 사이에 남이 끼면 뚫린다. 제약은 DB가 지키고, 애플리케이션은 위반 코드를 사용자 오류로 번역한다. 그 필드를 통짜 저장으로 바꿀 수 있으면 겹치는 값 하나가 그 행의 저장 전부를 막으므로, 서버 소유로 돌리고 전용 경로로만 바꾼다. 테스트가 공용 이름 공간에 만드는 데이터에는 무작위 꼬리를 붙인다.

사례 · DB 제약

무슨 일이 플레이어마다 도시 이름이 하나 있고, 랭킹에서는 그 이름이 유일한 공개 식별자다. 이 이름을 대소문자 무시로 고유하게 했다. 정본은 DB 고유 인덱스이고, Postgres의 위반 코드(23505)를 409로 돌린다. 이름은 서버 소유로 돌리고 첫 저장과 이름 변경 경로로만 바꾸게 했다. 검증 삼아 서버 소유 목록에서 이름을 빼 보니 「통짜 저장으로는 이름이 안 바뀐다」 검사가 깨졌다. 검사가 살아 있다는 확인이다. 같은 날, 스모크 테스트가 매번 같은 고정 이름으로 만든 계정 45개가 랭킹을 덮고 있는 것을 보고 테스트 계정 203개를 지웠다. 그 뒤로 테스트 데이터 이름에는 무작위 꼬리를 붙인다.

여러 개를 한 번에 사는 동작은 원자적 요청 하나로 만든다

장바구니를 한 개짜리 구매 여러 번으로 만들면 가운데서 막혔을 때 반만 산 채로 남는다. 잔액이 모자라거나 네트워크가 끊기는 것은 드문 일이 아니다. 묶음 결제는 전부 사거나 하나도 안 사는 서버 요청 하나로 처리한다.

사례 · 결제 설계

무슨 일이 게임 내 상점에 장바구니를 붙이며, 이미 있던 단건 구매 API를 반복해 부르는 안을 버리고 묶음 요청 하나로 정했다. 구현이 가장 쉬운 쪽이 가장 위험한 쪽이었다.

순서가 뜻을 갖는 자료구조에 우선권을 넣을 때는 나머지 항목의 자리까지 정한다

대기열·랭킹을 고칠 때 넣는 항목 하나만 보면, 나머지가 어디로 밀리는지가 빠진다. 모든 항목의 상대 순서를 테스트로 고정한다. 순서 검증에는 셋 이상의 표본이 필요하다.

사례 · 매칭 대기열

매칭이 성사돼도 한쪽이 상대를 거절할 수 있다. 거절당한 사람이 억울하지 않게 대기열 맨 앞으로 돌려보내기로 했다. **무슨 일이** 구현하면서 거절한 쪽도 같은 우선 표시로 대기열에 다시 넣었다. 「우선」 항목은 배열 맨 앞에 끼우는 방식이라, 나중에 넣은 거절한 쪽이 더 앞자리를 차지했다. 의도와 정반대였다. **왜 조용했나** 단위 테스트의 표본으로는 누가 먼저 집혔는지가 드러나지 않았다. **잡은 방법** 실제 서버로 도는 온라인 스모크에 세 번째 참가자를 넣자, 그가 거절당한 사람이 아니라 거절한 사람과 매칭됐다. 그 테스트가 순서를 명시적으로 검증하게 남겼다.

동점이 가능한 「최고값 고르기」에는 명시적인 2차 정렬 키를 둔다

저장소는 순서를 지켜 주지 않는다. 같은 코드가 입력 순서만 다른 자료에서 다른 답을 낸다. 동점은 id 순 같은 결정적 기준으로 깨고, 입력 순서를 뒤집어도 같은 답이 나오는 회귀로 고정한다.

사례 · 랭킹

랭킹 요약 화면은 계정의 캐릭터 중 점수가 가장 높은 캐릭터를 하나 골라 보여 준다. 무슨 일이 새 계정은 캐릭터가 전부 0점인데, Postgres jsonb 가 객체의 키를 다시 늘어놓아 저장 전 화면과 서버가 서로 다른 「최고 캐릭터」를 골랐다. 순위가 「—」로 빠졌다. 왜 조용했나 동점이 아닌 계정에서는 드러나지 않는다. 잡은 방법 랭킹 요약 화면을 갓 만든 계정으로 확인하다 발견했다.

4.4 실시간 방과 전선

실시간 대전 서버는 방·전송·재접속을 맡고, 판정은 클라이언트와 함께 쓰는 룰 엔진이 한다. 이 경계에서 틀리는 것은 메시지 크기, 시계, 리스너 시점, 라이브러리 버전이다. 모두 브라우저 둘을 띄워야 보인다.

실시간 전송에는 로그를 뺀 스냅샷과 이벤트를 싣고, 마감은 남은 시간으로 보낸다

누적 로그를 싣으면 메시지가 판 길이에 비례해 커진다. 이벤트만 보내면 받는 쪽에 상태 리듀서, 즉 엔진 적용 함수의 두 번째 구현이 생긴다. 상태 표현은 엔진 하나만 두고, 전선에서는 로그를 떼고 받는 쪽이 이벤트를 이어 붙여 복원한다. 떼는 자리와 붙이는 자리는 각각 하나다. 절대시각 마감은 양쪽 시계가 어긋나면 틀리므로 「남은 ms」로 싣고, 단계가 바뀔 때만 다시 잔다.

사례 · 네트워크 전송

수치 로그를 뺀 상태 스냅샷은 3대3 판 2,408B, 5대5 판 3,705B였고, 로그를 싣으면 27,743B, 55,089B였다. 로그 쪽은 판이 길어질수록 계속 커진다. 잡은 방법 일부러 깨 봤다. 로그 떼기를 끄자 회귀가 「스냅샷이 판 길이를 따라 커진다」로 실패했고, 마감을 메시지마다 다시 재계 하자 「배치 중 보낸 의도가 마감을 되살렸다」로 실패했다. 양쪽 시계를 100억 ms 어긋나게 해도 같은 마감이 나오는지 확인했다.

메시지 리스너는 요청을 보내기 전에 건다

응답이 같은 틱에 도착하면, 늦게 건 리스너는 그 메시지를 영원히 못 받는다. send() 보다 먼저 onMessage() 를 걸고, 나중에 올 응답을 기다리는 리스너는 객체를 만들 때 건다. 멈추는 버그는 긴 전체 스모크 안에서 반복하지 말고 가장 작은 재현 스크립트로 떼어 좁힌다.

사례 · 테스트 코드

무슨 일이 테스트 코드가 두 참가자를 대기열에 넣는 호출(Promise.all)을 기다린 뒤에야 「매칭 완료」 리스너를 걸었다. 두 번째 참가자가 들어오는 순간 서버는 곧바로 매칭 완료를 보낼 수 있고, 그 메시지는 리스너 없이 지나간다. 증상은 「타임아웃도 없이 멈춘다」였다. 실제 클라이언트 구현에서 이미 피한 경주를 테스트 코드에서 새로 짰다. 왜 조용했나 대개는 메시지가 조금 늦게 와서 통과한다. 같은 틱에 올 때만 걸린다. 잡은 방법 포트 충돌을 의심하다가, 매칭만 떼어 낸 디버그 스크립트로 「받을 준비가 늦었다」임을 알았다. 120초 넘는 전체 스모크 안에서 반복했으면 훨씬 오래 걸렸다.

네트워크 라이브러리의 버전 작은 진짜 소켓을 여는 스모크로 확인한다

클라이언트와 서버의 메이저 버전이 어긋나면 타입 검사와 단위 테스트는 통과하고 실제 접속에서만 죽는다. 메이저를 맞춰 정확한 버전으로 고정하고, 올릴 때는 진짜 서버를 띄우는 스모크로 확인한다. 쓰는 기능만 담은 하위 패키지를 골라 의존성을 좁힌다.

사례 · 의존성 버전

무슨 일이 실시간 서버 프레임워크 Colyseus를 설치하자 서버는 0.17.10이 왔는데 JS 클라이언트의 최신은 0.16.22였다. 방 좌석을 예약하는 순간 `Cannot read properties of undefined (reading 'name')` 로 죽었다. 서버를 0.16으로 내려 맞추려 하자, 배포된 `package.json` 에 모노레포 내부 표기(`workspace:^`)가 남은 버전, redis·인증까지 전부 import하는 메타 패키지가 차례로 막았다. **잡은 방법** 테스트 안에서 서버를 직접 띄워 한 판을 끝까지 도는 온라인 스모크가, 만들어지자마자 처음 한 일이 이 불일치를 찾은 것이다. 메타 패키지를 버리고 필요한 하위 패키지 둘만 정확한 버전으로 고정했다.

기존 경로를 그대로 탄다고 가정한 새 입력도 끝에서 끝까지 흘러 본다

「서버는 한 줄도 안 고쳤으니 되겠지」인 입력에도 방 왕복 회귀와 재생 일치 회귀를 따로 둔다. 방이 한 입력에 메시지를 여러 번 내보내면 마지막 것만 보지 않는다. 데모에서 당기 어려운 상태는 검사와 함께 진입 통로를 만든다.

사례 · 새 입력

무슨 일이 전투 중 아이템을 쓰는 새 행동을 더했다. 구조상 서버의 방 로직과 재생 함수를 한 줄도 고치지 않고 통과하게 돼 있었다. 그래도 아이템 든 캐릭터 차례까지 턴을 넘겨 실제로 보내고, 양쪽이 받은 모든 메시지에서 「재생한 로그가 글자까지 같은가」를 확인했다. 방은 행동을 적용한 직후 시간을 밀어 메시지를 여러 개 내보내므로 마지막 것만 보면 놓친다. 일반 데모 계정으로는 그 단추가 영영 뜨지 않아, 아이템을 줘여 주는 개발용 URL 매개변수를 검사와 함께 만들었다.

시간 제한 여럿의 관계를 불변식으로 묶는 일은 → 2장 게임 디자이너, 전송 구현체 둘의 동작 계약과 연결 소유권은 → 5장 클라이언트 엔지니어에서 다룬다.

4.5 실패를 읽는 법

오류 코드를 잘못 읽으면 폴백(실패했을 때 물러나는 대체 경로)이 돌고, 폴백은 성공처럼 보이는 화면을 그린다. 이 절은 실패의 뜻을 가르고, 실패를 정확히 말하게 하는 원칙이다.

404를 「서버에 못 닿음」으로 읽지 않는다

404는 서버가 살아 있는데 그 길을 모른다는 뜻이다. 개발 중에는 대개 새 경로를 붙이고 서버를 다시 안 띄운 것이다. 이때 로컬 계산으로 물러나면 화면에서만 성사되고 서버에는 안 남는다. 404는 말하고 멈춘다.

사례 · 오류 해석

클라이언트는 계정 서버가 꺼져 있어도 게임이 돌도록, 못 닿으면 로컬 계산으로 물려나게 만들어져 있었다. **무슨 일이** 계정 서버가 건설 재료 구매 경로를 붙이기 전의 프로세스로 떠 있었다. 클라이언트가 404를 「못 닿음」으로 읽고 로컬로 물려났다. 재료가 화면에서만 늘었다가, 다음 통짜 저장에서 서버가 자기 값으로 되써 사라졌다. **왜 조용했나** 사람에게는 「샀는데 증축이 거부된다(0/10)」로만 보였다. **잡은 방법** 임시 서버를 다른 포트로 띄워 401과 404 경로를 직접 확인했다.

폴백은 그 수에서 잃는 쪽이 없을 때만 한다

폴백을 만들기 전에 그 거래에서 내는 것과 받는 것의 소유자를 본다. 둘 다 다음 왕복에 정정해도 아무도 잃지 않으면 폴백해도 된다. 내는 쪽이 클라이언트 소유(저장이 남는다)이고 받는 쪽이 서버 소유(저장이 버려진다)면, 폴백하는 순간 내는 것만 사라진다. 그 수는 서버가 판정하고, 실패하면 알린다.

사례 · 폴백 설계

무슨 일이 건설 재료 구매는 금화(클라이언트가 저장하는 값)를 내고 재료(서버 소유)를 받는다. 장비 제작도 금화를 내고 아이템(서버 소유)을 받는다. 둘 다 폴백하면 다음 동기화에서 받은 것은 버려지고 낸 금화만 사라진다. **결과** 두 수만 폴백을 하지 않는다. 제작 주문과 취소는 서버가 판정하게 했고, 아이템 총량이 늘지 않는 장착·해제만 클라이언트 계산 + 통짜 저장으로 남겼다.

오류 코드의 뜻을 확인하기 전에는 설정을 바꾸지 않는다

DNS의 `EAI_AGAIN`은 일시적 실패이고, 존재하지 않는 호스트(`NXDOMAIN`)가 아니다. 설정을 고치지 말고 다시 시도한다. 뜻을 모르는 오류에 설정을 고치면 원래 문제 위에 새 문제를 엮는다.

사례 · 인프라

무슨 일이 DB 연결 풀러(pooler) 호스트의 DNS 조회가 간헐적으로 `EAI_AGAIN`을 냈다. 호스트 이름이 바뀐 줄 알고 이름의 앞자리를 고쳤다가 `tenant not found`를 받고 되돌렸다. 원래 설정이 맞았다.

개발용 치트 통로는 기본이 닫혀 있고, 닫혀 있을 때는 이유를 말한다

개발용 재화 지급 경로를 배포에 켜 두면 그 자체가 치팅 경로다. 환경 변수로 오픈할 때만 연다. 닫힌 응답은 404가 아니라 이유가 담긴 400으로 준다. 404는 「서버를 안 다시 띄웠다」와 헛갈린다. 화면이 이 설정에 따라 달라져야 하면 추측하지 말고 묻는 경로를 둔다.

사례 · 개발 도구

무슨 일이 테스트용으로 금화를 넣어 주거나 캐릭터를 다치게 하는 경로를 환경 변수 하나가 켜져 있을 때만 열었다. 배포에 켜 두면 금화를 찍어 내는 통로가 된다. 그 뒤 화면의 [개발용 부상시키기] 단추가 스위치를 끈 뒤에도 늘 그려져 있다가, 누르면 「꺼져 있다」만 받는 것이 드러났다. 개발자는 「스위치」를 서버의 환경 변수로, 기획자는 화면의 그 단추로 읽고 있었다. 서버에 개발 설정을 묻는 경로를 두고, 켜져 있을 때만 단추를 그리게 했다.

→ 1장 모든 직무 공통

사람용 오류 문장은 그대로 두고, 번역용 코드와 매개변수를 덧붙인다

서버의 실패 이유 문장은 400 응답·로그·API 소비자가 읽는다. 다국어 화면을 위해 그 문장을 바꾸지 말고 기계용 `code` 와 `params` 를 더한다. 코드와 번역 키의 완전성은 회귀로 검사한다.

사례 · 다국어

게임은 열 개 언어를 지원하는데, 서버가 돌려주는 실패 이유는 한국어 문장 하나였다. 무슨 일이 「보유 캐릭터가 모자라 도시를 증축할 수 없다」 같은 규칙 실패에 한국어 문장(`reason`)은 그대로 두고 `code` · `params` 를 더했다. 회귀가 「실패마다 정해진 목록 안의 코드가 있고, `params` 의 숫자가 원문 문장에도 있다」 와 「코드마다 열 언어 문구가 다 있다」 를 고정한다. 영어 번역 키 하나를 지워 그 자리에서 실패하는 것을 확인했다.

4.6 외부 서비스 · 계정 · 비밀

클라우드 대시보드의 토글, 브라우저의 CORS, 템플릿의 자리표시자는 코드 밖에 있다. 형식 검사와 서버 대 서버 호출로는 재현되지 않는다. 확인은 언제나 실제 호출로 한다.

클라우드 설정 값은 「형식이 맞는가」가 아니라 「실제로 통하는가」로 확인한다

URL 파싱과 문자 클래스 검사는 「맞는 형식」과 「유효한 값」을 가르치 못한다. 템플릿을 손으로 채울 때는 자리표시자를 감싼 기호까지 지운다. 연결 문자열은 구조적으로 뜯어 보고, 최종 확인은 실제 호출로 한다. 비밀 값은 로그에 다시 찍지 않고 메타데이터만 본다.

사례 · 클라우드 설정

무슨 일이 Supabase(Postgres 호스팅 + 인증)를 붙이며 세 번 막혔다. ① Direct connection이 기본 IPv6 전용이라, IPv6 아웃바운드가 없는 환경에서 DNS는 정상인데 `ENOTFOUND` 가 났다. Transaction Pooler 주소로 바꿨다. ② 연결 문자열 템플릿의 `[YOUR-PASSWORD]` 에서 대괄호까지 비밀번호로 남겨, 비밀번호를 리셋해도 계속 실패했다. ③ 서버용 비밀 키가 레거시 JWT와 새 `sb_secret_` 두 형식이었다. 왜 조용했나 URL 파싱은 대괄호를 정상 문자로 통과시켰고, 형식 검사로는 맞는 키와 유효한 키를 가릴 수 없었다. 잡은 방법 `nslookup` 으로 돌아오는 주소 종류를 봤다. 비밀번호는 URL 디코드 뒤 첫 문자와 끝 문자의 문자 코드를 찍어 `[(91)·](93)` 를 찾았다. 키는 값을 화면에 다시 찍지 않고 JWT payload의 `role` 만 본 뒤, 실제 관리자 API를 불러 확인했다.

외부 서비스의 설정은 코드 주석이 아니라 실제 응답 모양으로 방어한다

대시보드 토글에 따라 응답 모양이 바뀌는 문제는 타입 검사와 로컬 회귀가 재현하지 못한다. 한 모양만 가정하지 말고 받은 응답을 보고 가른다. 운영 설정을 바꿔도 코드가 안 깨지는 쪽을 고른다.

사례 · 외부 서비스

무슨 일이 회원가입 코드는 주석에 「확인 메일 꺼짐」이라고 적고, 그 전제로 응답의 `user.id` 를 읽었다. 실제 대시보드에서는 확인 메일이 켜져 있었고, 그때는 세션 없이 유저 필드가 감싸지 않은 채 최상위에 온다. 실사용자의 가입이 `Cannot read properties of undefined` 로 죽었다. 잡은 방법 실사용자의 스크린샷 보고, 대시보드 설정을 끄는 안(코드 무변경)도 있었지만, 코드가 두 모양을 다 견디고 「확인 메일을 보냈는가」를 돌려주게 했다. 확인 메일이 갔으면 로그인시키지 않고 메일함을 확인하라고 안내한다.

CORS 허용 메서드는 명시하고, 라우트를 더할 때마다 다시 본다

라이브러리 기본값이 쓰는 메서드를 다 열어 준다고 믿지 않는다. 빠지면 브라우저는 프리플라이트(본 요청 전에 허용 여부를 묻는 `OPTIONS` 요청)를 통과시킨 뒤 진짜 요청을 보내지도 않고 막아, 서버 로그에 흔적이 없다.

`curl` 과 `Node fetch` 는 CORS를 검사하지 않으므로 검증은 진짜 브라우저로 한다. 파괴적인 요청은 화면이 결과를 확인한 뒤에야 다음 단계로 넘어간다.

사례 · CORS

무슨 일이 @fastify/cors 에 `origin: true` 만 주자 허용 메서드가 `GET, HEAD, POST` 로 잡혀 `PUT` 이 빠졌다. 서버 로그에는 `OPTIONS` 와 `GET` 만 남았고, `curl` 로는 200이 왔다. Playwright로 진짜 로그인 폼 채우고 네트워크 탭을 보고서야 찾았다. 2주 뒤 계정 삭제 라우트를 붙이며 같은 함정을 `DELETE` 로 다시 밟았다. 화면은 삭제 결과를 안 보고 새 계정 만들기로 넘어갔고, 새 도시가 남아 있던 옛 행 위에 얹혀 「지웠는데 그대로네」가 됐다. `DELETE` 를 열고, 실패하면 화면이 말하게 했다.

신원과 계정 데이터를 같은 저장소에 두어 신뢰 경계를 없앤다

인증과 데이터가 두 시스템으로 갈리면 「이 사용자 id가 실제로 있는가」를 매번 믿어야 하는 경계가 생긴다. 계정 행이 인증 사용자 테이블을 FK로 참조하면 존재 검증과 삭제 정리가 DB 제약 하나로 끝난다. 대상 시장에 맞지 않는 로그인 수단은 기본에서 뺀다.

사례 · 인증 설계

무슨 일이 첫 설계안은 Firebase Auth + 한국 메신저 로그인이었다. 글로벌 배포가 목표라 한국 전용 로그인을 기본에서 빼자, Firebase의 유일한 우위였던 「익명 계정 → 정식 계정 승격」 흐름의 값이 줄었다. DB로 이미 Postgres를 골라 둔 상태였고, Supabase Auth는 그 Postgres 안에 산다. 그쪽을 택했다. 검증 계정 스모크가 진짜 토큰으로 요청하고, 인증 계정을 지우면 FK cascade로 계정 데이터 행도 사라지는지 확인한다. 가짜 토큰으로 돌리면 토큰 검증 경로 자체가 시험되지 않는다.

비밀번호는 앱이 저장하지 않는다

평문 비밀번호를 `localStorage` 에 두면 같은 출처의 모든 스크립트가 읽는다. 비밀번호 관리자는 OS 잠금 뒤에 있고 출처까지 검증한다. 앱이 기억할 것은 이메일뿐이다. 사용자의 기능 요청은 그대로 구현하지 말고 이미 있는 안전한 장치로 쪼개 맞춘다.

사례 · 계정 보안

무슨 일이 「아이디·비밀번호 자동 저장」 요청을 셋으로 갈랐다. 자동 로그인은 이미 있는 세션 유지로 해결했다. 이메일은 `localStorage` 에 저장하고 로그아웃해도 지우지 않았다. 로그아웃은 「이 계정에서 나간다」이지 「이 기기를 남에게 넘긴다」가 아니다. 비밀번호는 입력란을 `<form>` 으로 감싸고 `autocomplete` 를 달아 브라우저·OS의 비밀번호 관리자에게 넘겼다.

비밀 파일을 「위험한 낱말 제외」로 걸러 출력하다 연결 문자열이 생긴 사례는 → 1장 모든 직무 공통에서 다룬다.

4.7 서버를 검증하는 법

서버 권위를 시험하는 검사가 화면을 보거나, 옛 프로세스에 붙거나, 읽기라고 믿은 쓰기를 부르면 아무것도 확인하지 못한다. 셋 다 검사는 초록이다.

서버가 반영했다는 검사는 서버 값을, 부수 효과 없는 경로로 읽는다

클라이언트가 낙관적으로 같은 숫자를 계산해 화면에 내면, 화면 값 검사는 서버 호출을 지워도 통과한다.

「서버가 반영했다」를 주장하는 검사는 서버에서 직접 읽는다. 그 읽기는 부수 효과가 없어야 한다. 읽으면서 되쓰는 GET으로 폴링하면 검사 도구가 경주를 만든다.

사례 · 테스트 검증

무슨 일이 스모크의 「입장료를 내면 자원이 10 → 7」 검사는 클라이언트의 React 상태를 읽고 있었다.

클라이언트가 여전히 로컬로 같은 계산을 해 화면에 내기 때문에, 서버 호출을 통째로 지워도 통과했을 검사였다.

서버에서 값을 읽어 확인하는 검사를 따로 더했다. 그런데 계정 조회 GET은 저장 형식 마이그레이션(옛 형식의 데이터를 새 형식으로 고쳐 읽는 것)이나 시간 정산 결과가 저장값과 다르면 그 자리에서 되쓰도록 설계돼 있었다.

이것으로 폴링하자 동시에 들어온 쓰기를 덮어, 거절 벌칙이 반영됐다가 다음 폴링에 되돌아갔다. **잡은 방법** 서버 값 확인을 DB 행 직접 읽기로 바꿨다. → 1장 모든 직무 공통

서버를 고쳤으면 떠 있는 프로세스가 그 코드인지부터 확인한다

옛 서버도 정상 응답을 하므로 검사는 통과하거나 엉뚱한 오류를 낸다. 스모크가 이미 잡힌 포트를 그대로 쓰면 고친 코드를 한 번도 거치지 않는다. 재시작 후 검증하고, 다른 사람이 쓰는 서버를 건드리지 않으려면 별도 포트에 둘째 인스턴스를 띄운다.

사례 · 개발 서버

무슨 일이 위의 쓰기 경합을 고친 뒤에도, 개발 중 떠 있던 계정 서버는 옛 코드였다. 스모크는 포트가 이미 잡혀 있으면 그 서버를 그대로 쓰게 돼 있어, 고친 코드를 한 번도 안 거칠 뻔했다. 다른 날에도 새로 붙인 경로들과 이름 중복 409가 재시작 전에는 안 보였다. 그 사이 겹치는 도시 이름은 고유 인덱스가 500을 내, 화면에 「서버에 닿지 못했다」로 떴다. **잡은 방법** 재시작 후 다시 돌렸고, 다른 포트에 둘째 인스턴스를 띄워 확인할 요청만 그쪽으로 보냈다.

패키지 진입점과 프로세스 부팅을 분리한다

라이브러리 진입점(여러 모듈을 모아 다시 내보내는 배럴 파일)은 내보내기만 하고, 부팅은 별도 파일에 둔다. 테스트가 부작용 없이 import할 수 있어야 한다.

사례 · 패키지 구조

무슨 일이 실시간 서버 패키지의 `index.ts`가 `listen()` 까지 해서, 스모크가 방 클래스만 빌리려고 import했는데 포트를 잡았다. 부팅을 `main.ts`로 옮기고 `index.ts`는 내보내기만 하게 했다.

작업 전후 점검표

- ☐ 서버가 무시하기 시작한 필드가 있는가? 그 필드를 바꾸던 사건 목록과 사건마다의 서버 경로가 있는가?
- ☐ 새 저장 호출이 신뢰 저장 함수와 비신뢰 저장 함수 중 맞는 쪽을 부르는가?
- ☐ 행위가 서버로 옮겨 왔는데 예전 허용을 「정상」으로 고정한 회귀가 남아 있지 않은가?
- ☐ 재생 검증에 들어가는 시각이 요청 타입에 필드로 선언돼 있고, 서버가 범위로 눌러 받는가?
- ☐ 재생 입력 중 「언제 정산했나」에 따라 달라지는 값이 있는가?
- ☐ 계정 쓰기가 전부 행 잠금을 지나는 단일 함수로 모였는가? 잠금을 빼면 검사가 깨지는가?
- ☐ 고유성을 조회 후 쓰기가 아니라 DB 제약으로 지키는가? 위반 코드를 사용자 오류로 번역하는가?
- ☐ 정렬·최고값 선택에 동점을 깨는 결정적 2차 키가 있는가? 순서 검사 표본이 셋 이상인가?
- ☐ 리스너를 요청 전에 거는가? 한 입력에 여러 메시지가 나가면 전부 확인하는가?
- ☐ 클라이언트·서버 네트워크 라이브러리의 메이저가 같고 정확한 버전으로 고정됐는가?
- ☐ 이 폴백에서 내는 것과 받는 것의 소유자가 같은가? 404와 「못 닿음」을 가르는가?
- ☐ 새 HTTP 동사를 쓰는 라우트를 붙였는가? CORS `methods`에 넣고 진짜 브라우저로 확인했는가?
- ☐ 외부 서비스 응답의 모양을 하나만 가정하지 않는가? 설정 값은 실제 호출로 확인했는가?
- ☐ 개발용 지급 통로가 기본으로 닫혀 있고, 닫힌 응답이 404와 구별되는가?
- ☐ 「서버가 반영했다」검사가 부수 효과 없는 경로로 서버 값을 읽는가? 지금 떠 있는 서버가 고친 코드인가?

5 장

클라이언트 엔지니어

판정이 낸 결과를 사람이 보고 누를 수 있게 만드는 사람

이 장은 화면·연출·입력·다국어·브라우저 검사를 맡는 사람을 위해 썼다. 사례의 무대는 턴제 전술 게임이다. 캐릭터는 체스 기물처럼 움직이고, 행동 순서는 각자의 대기 시간으로 정해진다. 온라인 대전은 서버가 판정하고, 열 개 언어를 지원한다. 이런 구조에서 클라이언트는 판정을 하지 않는다. 판정은 룰 엔진과 서버가 하고, 클라이언트는 그 결과를 시간에 맞춰 보여 주고 사람의 의도를 받아 보낸다. 그래서 클라이언트의 버그는 대부분 판정을 틀리지 않는다. 화면이 거짓말을 하거나, 누를 길이 없거나, 소리가 늦거나, 한 언어에서만 글자가 잘린다. 타입 검사와 엔진 테스트는 전부 통과하고, 사람이 한 판을 끝까지 해 보거나 몽골어로 바꿔 봐야 드러난다. 주니어가 가장 자주 하는 착각은 「테스트가 초록이니 화면도 맞다」다. 이 장의 원칙 서른 개는 그 「조용한 어긋남」을 미리 막는 방법이다.

이 장의 원칙

1. 연출의 모든 구간은 공용 타임라인 커서 하나 위에 놓고, 커서를 미는 호출을 맨 먼저 부른다
2. 화면 상태는 이미 적용이 끝난 결과다 — 연출은 옛 값도 결과도 스스로 만들지 않는다
3. 화면이 읽는 단계값은 지금 실제로 일어나는 일과 같아야 한다 — 연출 중에는 단계를 굳힌다
4. 소리는 이벤트 도착 시각이 아니라 그림의 시각표에 심고, 연출 게이트가 소리까지 얼리지 않는지 본다
5. 렌더 계기 밖에서 오는 변화는 그 자리에서 직접 다시 그린다
6. 열린 화면은 데이터의 스냅샷이 아니라 id를 들고 렌더마다 다시 찾는다
7. 상태의 가짓수를 코드 모양에 드러낸다 — 셋 이상은 하나씩, 하나만 뜯 것은 값 하나로, 걸음은 key 로
8. 동기 초기화를 비동기로 바꾸면 「마운트 때 한 번」 도는 effect를 전부 다시 본다
9. 프레임워크의 관리 밖에 둔 것은 프레임워크가 지켜 주지 않는다 — 만들지 않거나, 한 방식으로 끝까지 책임진다
10. 넘겨주는 자원은 「지금 누가 담는가」를 상태로 들고, 닫기·재생은 여러 번 불러도 안전하게 만든다
11. 같은 인터페이스를 구현한다는 것은 동작 계약까지 같다는 뜻이다
12. 기다리지 않는 저장도 같은 자원에는 순서를 지키고, 실패를 숨기지 않는다
13. 실행 환경이 거부하는 것은 오류로 나타나지 않는다 — 가장 엄격한 경로에 맞추고 거부를 전제로 짤다
14. 엔진이 허용하는 모든 수에는 화면의 통로가 있어야 하고, 판정으로 잠그는 것은 의도를 내는 단추뿐이다
15. 칠한 칸은 화면 안에서 누를 수 있어야 하고, 위에 띄운 것은 그 아래를 막지 않아야 한다
16. 입력은 조용히 버리지도, 두 번 받지도 않는다
17. 화면의 길은 보이고 정직해야 한다 — 모드 표시와 탈출, 진입 경로에 맞춘 라벨, 섬이 된 화면
18. 화면이 미리 보여 주는 숫자는 판정 엔진의 함수가 내고, 미리보기는 실제 기능을 그대로 돌린다
19. 가정값과 확정값은 다른 칸, 다른 모양으로 둔다
20. 서버 권위 결과는 짐작하지 말고 받은 값에서 읽는다

21. 값을 고르는 함수를 갈랐거나 자리를 비워 뒀으면, 소비하는 자리가 무엇을 부르는지 전부 훑는다
22. 문장은 키 하나에 통째로 두고, 엔진은 표시 언어를 모르게 둔다
23. 「없으면 원어로 물러난다」는 누락을 가린다 — 「원어와 다른가」로 기계 검사한다
24. 표시용 이름은 고르는 함수 하나만 거치게 하고, 번역 소스가 붙는 날 읽는 자리를 전부 훑는다
25. 배치는 가장 긴 번역으로 확인하고, 한 언어의 배치를 다른 언어에 물려주지 않는다
26. 테스트는 순서·모양·화면 글자가 아니라 뜻 속성(data-*)으로 요소를 찾는다
27. 시간은 단위 테스트가 지키고, 브라우저 검사는 실제 상태에 도달한 뒤 존재와 위치만 본다
28. 「있는가」「제자리에 있는가」「몇 줄인가」는 각각 따로 좌표로 켜다
29. 검사의 재료가 거짓이면 검사도 거짓이다 — 기준표는 원본을, 픽처는 실제 같은 값을, 셋업은 적용 확인을
30. 검사가 없는 자리는 고치기 전에 현재 출력을 먼저 못 박고, 새 모드에는 끝까지 도는 스모크를 붙인다

5.1 연출의 시간축 — 화면은 결과를 시간에 맞춰 풀어 놓는다

판정 엔진은 공격 한 번을 즉시 적용하고 「무슨 일이 일어났는가」를 이벤트 목록으로 돌려준다. 걷고, 때리고, 맞고, 쓰러지는 몇 초는 전부 클라이언트가 만든다. 최종 상태는 맞는데 도중의 순서가 틀리는 버그는 스크린샷 한 장에 찍히지 않는다.

연출의 모든 구간은 공용 타임라인 커서 하나 위에 놓고, 커서를 미는 호출을 맨 먼저 부른다

유닛마다 연출 시계를 0에서 시작하면 앞선 이동만큼 밀린 유닛과 안 밀린 유닛이 엇갈린다. 이벤트를 훑으며 커서 하나를 밀고, 자세·카메라·점멸·소리를 모두 그 위에 놓는다. 카메라 이동처럼 커서를 움직이는 호출은 모든 분기의 첫 줄에 둔다. 순서 자체가 계약이 되어야 다음 사람이 뒤집지 않는다.

사례 · 연출 타이밍

무슨 일이 공격자는 다가가는 이동 0.6초만큼 늦게 때렸는데 피격자의 타임라인은 0에서 시작해, 대상이 아직 걸어오는 공격자보다 먼저 아파했다. 공용 커서로 고친 뒤 「카메라가 먼저 그 자리로 간다」를 넣자 카메라 함수가 커서를 밀게 됐는데, 이를 자세 재생보다 늦게 부른 분기에서 「줄인이 도는 동안 이미 때리고 있는」 어긋남이 돌아왔다. **왜 조용했나** 스크린샷에는 시간 순서가 안 찍힌다. **잡은 방법** 실제 화면에서 20ms 간격으로 유닛별 자세 번호를 시계열로 기록해 사양과 대조했다.

화면 상태는 이미 적용이 끝난 결과다 — 연출은 옛 값도 결과도 스스로 만들지 않는다

엔진이 넘겨준 상태는 「다 끝난 뒤」의 모습이다. 연출 도중에 필요한 옛 값(출발 좌표, 맞기 전 HP)은 이벤트에 「어디서(from)」 「언제(at)」로 실려 와야 하고, 연출 층은 그것만 쓴다. 화면이 이전 상태를 스스로 기억하면 재접속처럼 상태를 통째로 받는 순간 무너진다. 같은 이유로 엔진이 시간을 한 번에 점프하면, 화면은 화면 시계가 따라잡지 못한 만큼 되돌린 값(엔진값 + 엔진 시각 - 화면 시각)을 그린다. 주사위처럼 추첨으로 보이는 연출도 결과를 정하지 않는다. 결과는 권위 상태에 이미 있고, 연출은 그것을 굴리는 모양으로 보여 줄 뿐이다.

사례 · 상태와 연출의 분리

무슨 일이 죽으면 다른 칸에서 되살아나는 캐릭터가 맞자마자 부활 자리에서 점멸했다. 엔진이 부활 이벤트와 함께 좌표를 이미 갈아 끼웠기 때문이다. 카메라가 먼저 가는 0.6초를 넣자 기물이 도착지에 한 번 뒹다가 출발점부터 다시 걸기도 했다. **왜 조용했나** 실전 데이터에서는 연출이 상태에서 되읽은 좌표가 우연히 맞는 값이었다. **잡은 방법** 좌표를 고정한 테스트 픽스처(검사용으로 미리 만들어 둔 입력 데이터)와 「인접하지 않은 칸으로 건너뛰면 실패」하는 순간이동 탐지기가 잡았다. 이벤트에 옛 좌표를 싣고 연출은 그것만 읽게 고쳤다.

사례 · 추첨 연출

무슨 일이 대기 시간이 같은 유닛끼리 순번을 정하는 주사위 연출을 넣었다. 주사위는 엔진이 판을 만들 때 이미 뽑아 둔 순번을 굴리는 모양으로 보여 주기만 하고, 판을 멈추지 않으며 클릭도 통과시킨다. **결과** 연출 층이 난수를 쓰지 않으므로 온라인의 두 사람이 같은 결과를 본다.

화면이 읽는 단계값은 지금 실제로 일어나는 일과 같아야 한다 — 연출 중에는 단계를 굳힌다

상태 기계의 단계값(phase)은 「지금 입력을 받아도 되나」를 다 말하지 않는다. 연출은 상태가 아니라 시간이라서, 턴이 연출 뒤에 넘어가면 그 몇 초 동안 단계값은 계속 「입력 대기」다. UI 표시 조건과 테스트의 「안정됨」 판정에는 단계값과 함께 연출 진행 여부(busy)를 묶는다. 상태 기계를 옮겨 적을 때는 「무엇을 하나」뿐 아니라 「언제 하나」까지 옮긴다.

사례 · 리팩터링

무슨 일이 AI 대전과 온라인 대전이 같은 재생기를 쓰도록 전송 층을 떼어 내면서, 새 메시지 처리 함수가 메시지를 받자마자 다음 단계를 정해 버렸다. 때리는 연출 2.6초 동안 화면이 읽는 단계값이 이미 「시간 진행 중」이었다. 옛 코드의 「연출이 끝난 뒤 진행」을 옮기면서 진행만 미루고 단계 결정은 미루지 않은 것이다. **왜 조용했나** 판정 결과가 똑같아서 단위 회귀 테스트(한 번 고친 동작이 다시 깨지지 않는지 지키는 자동 검사)는 모두 통과했다. **잡은 방법** UI 스모크 테스트(실제 브라우저에 화면을 띄워 끝까지 눌러 보는 자동 검사)가 X 대기 시간 게이지가 시간이 흘러도 줄지 않는다 (151.0 → 151.0)로 잡았다. 그 뒤 「연출 중에 단계가 바뀌었다」를 단위 회귀로 내려 고정했다.

소리는 이벤트 도착 시각이 아니라 그림의 시각표에 심고, 연출 게이트가 소리까지 얼리지 않는지 본다

이벤트를 받는 순간 소리를 들면 카메라 이동 시간만큼 소리가 그림보다 앞선다. 소리·자막처럼 연출과 맞아야 하는 것은 연출 시각표의 큐로 둔다. 반대로 「연출 중에는 프레임 갱신을 건너뛴다」같은 게이트를 두면 같은 루프에 매달린 소리·타이머도 함께 언다. 게이트를 둘 때는 그 루프에 걸린 다른 시간축을 같이 보고, 게이트에 걸리는 연출의 소리는 연출을 시작하는 자리에서 직접 튜다. 에셋을 새로 연결했다면 오디오 굽기(원본을 배포용 파일로 변환하는) 단계를 돌렸는지도 확인한다 — 없는 파일은 설계대로 조용히 무음이 된다.

사례 · 사운드 동기화

무슨 일이 적의 공격·마법·처치에서 소리가 카메라 이동 시간만큼 먼저 났다. 내 턴에는 카메라가 이미 그 자리라 지연이 안 붙어 안 보였다. **왜 조용했나** 회귀 테스트용 가짜 상태가 유닛 좌표를 비워 두어 카메라가 늘 「같은 자리」로 판단했고, 지연이 붙는 경로가 테스트에서 한 번도 돌지 않았다. **잡은 방법** 사용자 플레이 보고를 따라가, 실제 좌표를 채운 픽스처를 따로 만들어 재현 회귀 6건을 세웠다.

사례 · 사운드 동기화

무슨 일이 캐릭터 필살기 배너가 도는 동안 게임 씬의 프레임 갱신이 「연출 중」 플래그로 통째로 건너뛰어지자, 같은 시각표에 있던 성우 대사가 배너 6초 내내 얼었다가 끝난 뒤에 나왔다. 코드 머리말에는 「배너 1단이 시작하는 바로 그 프레임에 튜다」고 적혀 있었다. **왜 조용했나** 소리는 결국 나서 오류가 없고, 테스트는 소리 타이밍을 보지 않았다. **잡은 방법** 사용자 보고. 배너를 띄우는 함수에서 소리를 직접 틀게 고쳤다.

5.2 다시 그리기와 화면 상태

화면이 낡는 버그는 대부분 「언제 다시 그리나」와 「무엇을 들고 있나」에서 나온다. 렌더를 상태 변화에만 걸어 두면 그 밖의 변화는 오류 없이 무시되고, 값을 복사해 들고 있으면 바꾸는 경로 하나를 빠뜨리는 순간 조용히 낡는다.

렌더 계기 밖에서 오는 변화는 그 자리에서 직접 다시 그린다

렌더 트리거가 「엔진 상태 변경」뿐이면, 엔진에 아무것도 안 보내는 화면 전용 조작(확인창 열기·취소)과 시간 경과로 일어나는 전이는 그림을 바꾸지 못한다. 이런 변화마다 명시적으로 다시 그리기를 부른다. 갱신 루프가 멈추는 구간(연출·일시정지)에는 「다음 갱신 때 사라지겠지」가 통하지 않는다. 상태 전이를 일으키는 그 순간에 관련 UI를 직접 닫는다.

사례 · 렌더 트리거

무슨 일이 필살기 연출이 2초간 판을 덮는 동안 화면 갱신이 돌지 않아, 직전에 띄운 질문 창이 그대로 남아 있었다. 시간이 흘러 저절로 얻은 차례에는 이동 가능 칸 하이라이트를 다시 그릴 계기가 없었다. 열흘 뒤에는 공격 확인창 열기·취소가 엔진에 아무것도 보내지 않아, 클릭도 먹고 화면 상태도 켜는데 그림만 그대로였다. 사용자에게는 「눌러도 반응이 없다」로 보였다. **왜 조용했나** 상태 밖 변화는 아무 오류 없이 무시됐다. **잡은 방법** 두 버그가 같은 자리임을 확인하고, 화면 전용 조작과 시간 경과를 다시 그리기 계기로 등록했다.

열린 화면은 데이터의 스냅샷이 아니라 id를 들고 렌더마다 다시 찾는다

열릴 때의 행을 state에 복사하면 그 값을 바꾸는 경로마다 「이 화면도 갱신해 달라」를 메워야 하고, 하나를 빠뜨리면 조용히 낡는다. 열린 화면은 참조(id)만 들고, 그리는 순간 원본에서 다시 찾는다.

사례 · 화면 상태

무슨 일이 캐릭터 상세 카드가 열릴 때의 데이터를 state에 복사해 두어, 레벨업 뒤에도 카드가 옛 값을 보였다. **왜 조용했나** 카드를 닫았다 다시 열면 맞는 값이 나왔다. **잡은 방법** 갱신 경로마다 카드 갱신을 부르는 대신 id만 들게 바꿨다. 육성 초기화·전적 반영까지 한꺼번에 실시간이 됐다.

상태의 가짓수를 코드 모양에 드러낸다 — 셋 이상은 하나씩, 하나만 뜯 것은 값 하나로, 걸음은 key 로

상태가 둘일 때 웬 A ? ... : ... 는 하나가 더해지는 순간 틀리는데, 타입 검사도 테스트도 말이 없다. 상태가 셋 이상이면 빠뜨리면 컴파일 오류가 나는 switch로 하나씩 적는다. 동시에 하나만 떼야 하는 팝업은 불리언 여러 개가 아니라 열거형 값 하나로 두어 「둘 다 떼 있다」를 표현할 수 없게 만든다. 같은 컴포넌트로 걸음만 바꾸는 품은 걸음 식별자를 key 로 주어 걸음마다 새 인스턴스를 만든다. React는 같은 자리의 같은 컴포넌트를 재사용해

앞 걸음의 내부 상태를 이어 붙인다. 폼 안의 제출용이 아닌 단추에는 `type="button"` 을 적는다 — 기본값이 submit이다.

사례 · 다단계 화면

무슨 일이 육성 화면은 「관리 · 레벨업 · 초기화 후 다시 키우기」 세 모드인데, 이를 「관리가 아니면 다시 키우기」로 접은 탓에 평범한 레벨업에 「다시 키우기 — Lv2 고르는 중 (1/0)」이 떴다. 같은 날, 다시 키우기 마법사가 다음 레벨로 넘어가도 선택기 인스턴스가 남아 Lv2에서 고른 선택지가 Lv3에서도 눌러 있었다. **왜 조용했나** 둘 다 타입 · 테스트를 통과했다. **잡은 방법** 700px 폭 스크린샷을 눈으로 검사할 때만 보였다.

사례 · 로그인 폼

무슨 일이 비밀번호 관리자가 알아보도록 로그인 팝업을 `<form onSubmit>` 으로 바꿨더니, `type="button"` 이 없던 [비밀번호 초기화]·[계정 생성]·[닫기]가 셋 다 로그인을 시도했다.

동기 초기화를 비동기로 바꾸면 「마운트 때 한 번」 도는 effect를 전부 다시 본다

데이터가 마운트 시점에 이미 있다고 가정한 effect는, 로딩이 비동기가 되는 순간 빈 값으로 한 번 헛돌고 다음 기회까지 아무것도 안 한다. 로딩 방식을 바꾸는 변경에서는 그 데이터를 읽는 effect를 모두 찾아 「데이터가 도착하는 순간」을 의존성에 넣는다.

사례 · 비동기 전환

무슨 일이 시간이 지나면 차오르는 자원(대전 입장에 드는 소모 자원)의 충전 tick이 `useEffect(..., [])` 로 마운트 때 한 번 돌았는데, 계정 로딩이 로컬 저장소 읽기에서 서버 호출로 바뀌자 그 순간 계정 값이 `null` 이라 헛돌았고 다음 기회는 1분 뒤였다. 로그인 직후에는 「자원이 안 찬다」로 보였다. **잡은 방법** 스모크의 「세 시간 뒤에 셋 찾는다」 검사가 잡았다. 의존성에 「계정이 있는가」를 더해 상태가 처음 채워지는 순간에도 한 번 더 돌게 했다.

5.3 프레임워크 밖의 자원, 연결, 실행 환경

캔버스 엔진, 전역 리스너, 소켓, 백그라운드 저장, 브라우저 정책은 화면 프레임워크의 수명 관리 밖에 있다. 여기서 틀리면 흔적이 다른 화면에, 다른 사람에게, 혹은 아무 데도 남지 않는다.

프레임워크의 관리 밖에 둔 것은 프레임워크가 지켜 주지 않는다 — 만들지 않거나, 한 방식으로 끝까지 책임진다

자원을 직접 만드는 명령형 엔진(Phaser 같은 캔버스 게임 엔진)은 React StrictMode의 effect 이중 실행에 캔버스를 둘 만든다. 정리 코드를 두거나 StrictMode에서 뺀다. `window` 에 직접 건 리스너는 엔진의 `destroy()` 가 치워 주지 않는다. 화면 수명보다 오래 사는 전역 리스너는 두지 않는 것이 최선이고, 꼭 필요하면 등록과 해제를 한 자리에서 만든다. 화면 전체를 듣는 위임 리스너(효과음·분석 로그)는 캡처 단계에 건다 — 버블 단계는 안쪽의 `stopPropagation()` 에 막힌다(팝업 8곳의 확인음이 이 이유로 전부 안 났고 `{ capture: true }` 한 줄로 고쳤다). 프레임워크가 관리하는 값과 직접 DOM 쓰기도 섞지 않는다. 측정과 적용은 처음부터 끝까지 한 방식으로 한다.

사례 · 리스너 누수

무슨 일이 전투 화면이 window에 건 keydown 리스너를 Phaser.Game.destroy()가 치우지 않아, 전투를 한 번 치른 뒤 죽은 전투 화면이 Q·E 단축키를 계속 가로챘다. 다른 화면의 이름 입력창에 Q·E가 안 써졌다. **왜 조용했나** 전투를 거친 뒤 다른 화면의 입력창에서만 드러났다. **잡은 방법** 처음에는 입력창 가드와 AbortController 해제를 만들었다가, 단축키 자체를 없애는 쪽으로 전부 건너뛴다. 누수를 막는 코드보다 원인이 된 기능을 들어내는 쪽이 낫다.

사례 · 간헐 버그

무슨 일이 설정 화면의 칩 폭을 React state로 들고 style prop으로 내리면서, 측정하려고 el.style.width = ''로 DOM을 직접 지웠다. 다시 랜 값이 직전과 같으면 React의 diff가 「바뀐 게 없다」며 DOM을 안 건드려 지운 상태가 남았다. **왜 조용했나** 폰트 로딩 콜백 둘이 겹쳐 풀릴 때만 나는 간헐 버그였고, 진단을 두 번 틀렸다. **잡은 방법** MutationObserver로 style 변화를 밀리초·콜스택 단위로 추적하고, 고친 뒤 자동화로 언어 전환을 10번 연속 반복해 재현이 사라진 것을 확정했다.

넘겨주는 자원은 「지금 누가 닫는가」를 상태로 들고, 닫기·재생은 여러 번 불러도 안전하게 만든다

화면 사이에 연결·방을 넘기면 닫을 책임도 함께 옮겨 간다. 그 사실을 플래그로 들고 다니지 않으면 넘긴 것을 닫거나, 넘기지 않은 것을 안 닫는다. 「닫는다」를 부르는 자리가 둘 이상이면 하나의 멍등한(여러 번 불러도 한번 부른 것과 같은) 진입점으로 모은다. 연출 재생 함수도 같다. play()는 첫 줄에서 이전 재생을 끝낸다.

「동시에 두 번 터질 수 있는가」는 실전의 동시 발동 조합으로 따져 본다 — 두루마리 연출을 연달아 부르자 칸이 겹쳐 그려졌다.

사례 · 연결 소유권

무슨 일이 매칭 화면이 사라질 때 게임 서버의 방을 닫게 했더니, [전투 준비]로 전투 화면에 넘긴 방까지 닫혀 상대에게는 「사라진 사람」이 됐다. 닫지 않으면 [뒤로 가기]만 눌러도 상대가 재접속 유예 60초를 기다린다. 다음 날에는 검색 타임아웃과 화면 언마운트가 각각 대기열 나가기를 불러 WebSocket is already in CLOSING or CLOSED state.가 콘솔에 남았다. **잡은 방법** 넘겼는지를 「넘김」 플래그로 표시했고, 나가는 「이미 나감」 플래그로 감싼 함수 하나로 모았다. 스모크는 콘솔 오류를 실패로 본다.

같은 인터페이스를 구현한다는 것은 동작 계약까지 같다는 뜻이다

타입은 메서드의 모양만 보고 「언제 무엇을 내보내는가」는 보지 않는다. 인터페이스에는 시그니처와 함께 호출 순서·첫 이벤트 같은 동작 계약을 주석으로 적고, 구현체가 둘 이상이면 같은 계약 테스트를 양쪽에 돌린다.

나중에 올 응답을 기다리는 리스너는 요청을 보내기 전, 가능하면 객체 생성 시점에 건다 → 4장 서버 엔지니어.

사례 · 전송 계층

무슨 일이 온라인 대전용 전송 객체가 열기·보내기·준비·닫기 메서드를 다 갖췄는데 화면이 아무것도 그리지 않았다. AI 대전용 전송 객체는 열 때 첫 상태를 바로 내보내는데 이쪽은 내보내지 않았고, 재생기는 변경 알림이 한 번은 와야 화면을 맞춘다. **왜 조용했나** 타입 검사는 통과했다. **잡은 방법** 온라인 스모크가 「첫 상태가 안 왔다」로 잡았다.

기다리지 않는 저장도 같은 자원에는 순서를 지키고, 실패를 숨기지 않는다

화면을 막지 않으려고 응답을 기다리지 않는 저장(fire-and-forget)을 쓰면 요청이 겹친다. 네트워크는 보낸 순서대로 도착한다고 보장하지 않으므로, 늦게 보낸 것이 먼저 반영되면 최신 상태를 옛 값이 덮는다. 같은 자원에 대한 쓰기는 앞 응답을 기다린 뒤 다음을 보내는 쓰기 큐로 직렬화한다. 간헐적인 「값이 한 단계 옛것」 증상은 요청 겹침을 먼저 의심한다. 그리고 백그라운드 저장의 실패를 로그로만 삼키지 않는다. 「저장이 안 된다」 제보는 ① 요청이 나갔는가 ② 사람이 저장 단추에 닿을 수 있었나 순으로 가른다.

사례 · 서버 저장

무슨 일이 「육성 초기화 → 다시 키우기」 스모크가 간헐적으로 「Lv2로 올렸는데 새로고침하면 Lv1」로 실패했고, 재현할 때마다 지점이 달랐다. 로컬 왕복이 400ms~2.8s라 계정 저장 put 이 겹쳤다. **왜 조용했나** 대부분은 순서대로 도착해서 가끔만 틀렸다. **잡은 방법** 쓰기 큐를 넣었다. 큐는 순서만 보장하고 완료를 기다려 주지는 않으므로, 스모크가 새로고침 전에 큐를 비우는 함수를 부르게 했다(앱이 아니라 스모크를 고쳤다).

사례 · 버그 제보

무슨 일이 「편성이 저장 안 된다」는 재현되지 않았다. 저장 요청에 붙인 순번이 한 번도 서버에 도착한 적 없다는 사실이 「다른 저장이 덮어썼나」 가설을 지웠고, 기획자가 [등록 완료] 단추가 화면 밖으로 밀려 저장하지 않는 [목록으로]를 눌렀음을 찾았다. 조사 중 진짜 구멍도 드러났다. 저장 함수가 실패를 `console.warn` 으로 삼키고 어떤 화면도 오프라인 표시를 읽지 않아, 로그인 토큰 갱신이 실패한 뒤의 모든 저장 실패가 화면에 안 보인다(미결로 남겼다).

실행 환경이 거부하는 것은 오류로 나타나지 않는다 — 가장 엄격한 경로에 맞추고 거부를 전제로 짤다

브라우저의 자동재생 거부 정책이라 콘솔에도 안 남는다. 오디오는 거부를 전제로 「재생 대기 곡」 상태를 두고 첫 사용자 입력에서 다시 튜트. 헤드리스 검사는 「어느 곡을 틀기로 했나」까지만 보장한다는 것을 적어 둔다. 테스트·서버·번들러가 서로 다른 변환기를 쓰면 공용 코드는 가장 제한적인 쪽 문법에 맞춘다. 투명 영상처럼 플랫폼마다 지원이 갈리는 형식은 데스크톱 크롬에서 보이는 것으로 판단하지 않는다(사파리가 알파 채널 WebM을 못 들어 알파 WebP 스프라이트로 구웠다) → 6장 디자인 엔지니어.

사례 · 브라우저 정책

무슨 일이 첫 화면에서는 아직 아무도 누르지 않아 배경음악 `play()` 가 거부되어 「첫 화면만 소리가 안 난다」가 됐다. **왜 조용했나** 원인이 코드 어디에도 없다. **잡은 방법** 조작 전에는 「메인 곡 대기 중」, 클릭 뒤에는 「대기 없음」인 것을 실측으로 확인하고 첫 입력에서 재시도하게 했다.

사례 · 빌드 도구 차이

무슨 일이 카메라 모듈에 TypeScript 생성자 파라미터 프로퍼티(`constructor(private readonly cues)`)를 썼더니 Node의 타입 스트리핑(타입만 지우고 그대로 실행하는 방식)으로 도는 테스트만 깨졌다. **왜 조용했나** Vite(esbuild)는 그 문법을 받아 주어 타입 검사도 브라우저도 멀쩡했다.

5.4 화면의 통로 — 엔진이 허용하는 모든 수에 손이 닿는가

엔진 테스트는 「이 의도가 허용되는가」를 본다. 「그 의도를 사람이 화면에서 낼 수 있는가」는 아무도 보지 않는다. 길이 막힌 화면은 오류를 내지 않고 그냥 멈춘다.

엔진이 허용하는 모든 수에는 화면의 통로가 있어야 하고, 판정으로 잠그는 것은 의도를 내는 단추뿐이다

엔진이 허용하는 의도 집합의 각 경우, 특히 원소가 하나뿐인 경우마다 화면에 그 의도를 낼 길이 있는지 본다. 기획 문구를 글자대로 옮기기 전에 「엔진이 허용하는 수 중 화면에서 닿을 수 없게 되는 것이 있나」를 따진다. 화면 전용 상태를 만들면 화면만으로 되돌리는 길도 함께 만든다. 단추를 엔진 판정으로 잠그는 것은 의도를 내는 단추에만 건다. 사거리 보기처럼 보기를 바꾸는 단추까지 잠그면 정보를 감추게 된다. 화면 작업 뒤에는 실제로 한 판을 끝까지 해 보는 확인을 요청한다.

사례 · 교착

무슨 일이 이동만 하고 공격할 대상이 없고 MP가 가득이면 낼 수 있는 의도가 「턴 종료」 하나뿐인데, 그 단추를 다음 작업 단계로 미뤄 게임이 멈췄다. **왜 조용했나** 헤드리스 스크린샷과 단위 테스트는 한 턴을 끝까지 진행해 봐야 드러나는 교착을 못 본다. **잡은 방법** 기획자가 직접 플레이하다 발견했다.

사례 · 기획 해석

무슨 일이 이동 뒤 메뉴에 대한 「이동을 제외하고 보여준다」를 글자대로 받아 이동 단추를 통째로 빼면, 실수로 제자리를 눌렀을 때 엔진은 다시 이동을 허용하는데 화면에 통로가 없는 함정이 됐다. 같은 날 공격 단추를 대상이 있을 때만 열자 「내 사거리가 어디까지인가」를 볼 방법이 없어졌고, 스모크도 대상 없는 판에서 잠긴 단추 앞에 멈췄다. **잡은 방법** 구현 중 함정을 기획자에게 알리고 경우를 갈라 확인받았다. 공격 단추는 늘 열고, 실제로 칠 수 있는 대상만 엔진이 고르게 했다.

칠한 칸은 화면 안에서 누를 수 있어야 하고, 위에 띄운 것은 그 아래를 막지 않아야 한다

「화면에 칠해진 칸 = 엔진이 허용한 칸」은 화면 쪽에서만 깨질 수 있는 계약이다. 연출용 카메라 규칙(보여 주기)과 조작용 카메라 규칙(고르기)은 다르다. 기획서가 보여 주기만 정했으면 고르기 상황은 구현이 채운다. 규칙의 조건은 증상(「고르는 중」)이 아니라 이유(「후보가 화면 밖일 수 있다」)로 적어야 해당하지 않는 경우까지 걸리지 않는다. 판 위에 띄우는 오버레이 컨테이너는 `pointer-events: none`, 조작하는 요소만 `auto` 로 둔다. 겹침은 오류 없이 클릭만 삼킨다.

사례 · 카메라와 조작

무슨 일이 기획서의 「내 차례 = 200% 확대」를 그대로 넣자, 멀리 떨어진 적을 고르는 광역 마법의 대상 두 칸이 화면 밖($y = -38$, $y = 91$)에 있었다. 「고르는 중엔 100%」로 고치자 대상이 늘 인접한 근접 공격에도 걸려 포커스가 물러났고, 「후보가 멀리 퍼지는가」로 다시 적었다. 줌을 고친 뒤에도 스모크가 같은 자리에서 멈췄는데, 대상 칸 위에 명령 패널의 「취소」가 떠 있었다. **잡은 방법** 스모크가 「마법을 썼는데 MP가 그대로다」로 멈췄고, `elementFromPoint` 로 클릭 지점의 맨 위 요소를 찍어 겹침을 찾았다.

입력은 조용히 버리지도, 두 번 받지도 않는다

입력을 무시하는 분기는 적어도 개발용 로고를 남긴다. 놀리는데 아무 일도 없는 단추는 사용자에게 고장으로 읽히고, 테스트에서는 「기능이 없는 것」처럼 보인다. 반대로 한 번만 나가야 하는 요청은 state가 아니라 동기 ref로 잠근다. 바깥 state는 다음 렌더에야 단추를 막는다. 비동기 응답 처리에서는 누른 순간의 화면 값이 아니라 지금 화면을 ref로 다시 묻고, 금지된 화면에서는 아예 그리지 않는다. 드래그·제스처 판정에는 반드시 문턱을 둔다 — 문턱이 없으면 클릭 중 1px 흔들림이 수동 카메라 모드를 세워 자동 카메라가 영영 꺼진다(이 경우 문턱은 8px로 잡았다).

사례 · 버려지는 입력

무슨 일이 스모크가 「항복 → 결과 화면」을 20초 기다려도 화면이 안 났고 콘솔 오류도 없었다. 재생기가 입력 대기 상태가 아닐 때(AI 차례·연출 중) 들어온 의도를 아무 말 없이 버리는데, 화면의 [항복]은 그때도 눌렀다. **잡은 방법** 스모크가 내 차례를 기다린 뒤 누르게 고쳤다. 사람에게 안내가 없다는 점은 미결로 넘겼다.

사례 · 중복 요청

무슨 일이 [지금 전투]를 두 번 누르자 요청이 두 번 나가, 둘째의 「이미 시작됐다」 거부 알림이 이미 넘어간 전투 화면 위에 났다. **왜 조용했나** 바깥 state는 늦게 막았고, 실패 처리 클로저는 누른 순간의 화면을 기억했으며, 알림 금지 플래그는 「떨울지」만 막고 이미 켜진 알림의 그리기는 막지 않았다. **잡은 방법** 기획자가 실제로 보고 알렸다.

화면의 길은 보이고 정직해야 한다 — 모드 표시와 탈출, 진입 경로에 맞춘 라벨, 섬이 된 화면

모드를 만들었으면 「지금 그 모드다」라는 표시와 터치로 누를 수 있는 탈출을 화면에 둔다. 숨은 단축키는 「길이 있다」가 아니라 「아는 사람만 쓴다」다. 휴대폰에는 키보드가 없다(수동 카메라에서 자동으로 돌아가는 길이 키보드 단축키 하나뿐이었다). 화면으로 들어오는 길이 둘이면 뒤로 단추의 행선지와 함께 라벨도 같은 값(진입 경로)에서 파생시킨다. 내비게이션의 입구를 옮기면 그 화면으로 들어오는 다른 길이 남는지 센다. 들어갈 수 없는 화면은 오류도 내지 않는다. 기능의 입구를 걷을 때는 그 입구가 만든 저장 상태가 여전히 효과를 내는지 본다 → 2장 게임 디자이너.

사례 · 내비게이션

무슨 일이 메인 화면에서 랭킹에 들어와도 뒤로 단추가 「← 도시 관리로」라고 적고 실제로는 메인으로 갔다. 스크린샷을 보고서야 알았다. 다른 때에는 안내 화면의 이동 단추를 새 흐름으로 바꾸자, 옛 전면 화면 셋이 밖에서 들어오는 길을 잃고 셋끼리만 서로 이은 섬이 됐다. **규칙** 섬이 된 화면은 지우거나 남긴 이유를 적는다.

5.5 화면이 말하는 숫자의 출처

화면이 공식을 다시 적으면 공식이 바뀌는 날 표시만 어긋나고 아무 테스트도 깨지지 않는다. 화면의 숫자는 그럴듯해 보이기 때문에 틀려도 사람이 의심하지 않는다.

화면이 미리 보여 주는 숫자는 판정 엔진의 함수가 내고, 미리보기는 실제 기능을 그대로 돌린다

발동 확률·예상 피해·레벨업 증분 같은 미리보기는 엔진이 계산해 준 값을 보여 주기만 하고, 「미리보기 = 실제 결과」 회귀로 고정한다. UI를 접어도 그 계약(함수와 회귀)은 남긴다. 같은 정보는 한 자리에서만 그린다.

미리보기는 난수를 쓰지 않는다 — 결정적(같은 입력이면 언제나 같은 결과를 내는) 엔진에서 미리보기가 난수를 소비하면 실제 판의 결과가 바뀐다. 연출 미리보기·튜토리얼·갤러리도 따로 만들지 않고 본 기능의 컴포넌트를 그대로 돌린다 — 따로 만들면 미리보기만 옛 모습으로 남는다. 엔진 쪽에서 판정 함수를 어떻게 모으는지는 → 3장 시스템 엔지니어.

사례 · 미리보기 수치

무슨 일이 마법 발동 확률, 공격력 범위, 예상 피해를 전부 엔진 함수로 냈다. 공격 확인창 UI는 판을 가린다는 이유로 뺐지만 예상 피해 함수와 「미리 잔 값 = 실제 피해」 회귀 4건은 남겼다. **막은 것** 화면이 「20 + 지력 차」 같은 공식이나 내림 규칙을 다시 적었다면 공식이 바뀌는 날 표시만 조용히 어긋났을 것이다. 실제로 「공격력 +1」 이 화면 글자에만 남고 데이터와 엔진은 0.5였던 일이 있었다. 이후 캐릭터 상세의 [발동 영상 보기]도 전투의 필살기 연출 컴포넌트를 그대로 돌리게 만들었다.

가정값과 확정값은 다른 칸, 다른 모양으로 둔다

「고르면 오를 양」과 「실제로 적용될 값」은 뜻이 다르다. 한 칸에 섞으면 화면만 보고는 어느 쪽인지 알 수 없다. 가정(what-if) 값은 별도 칸에 다른 스타일로, 확정 값은 제 칸에 두고, 「선택하지 않은 항목의 결과는 현재값과 같다」를 검사로 고정한다.

사례 · 레벨업 화면

무슨 일이 레벨업 때 능력치 하나를 고르는 화면에서, 안 고른 줄에도 증분을 보여 달라는 요청에 줄마다 미리보기 함수를 썼더니 결과 칸까지 가정치가 됐다(MP 5 +2 → 7 인데 실제로는 안 오른다). **왜 조용했나** 그럴듯한 숫자였다. **잡은 방법** 「고르면 오를 양」(회색) 칸과 「실제 적용값」 칸을 나누고, 스모크에 「안 고른 줄의 결과는 지금 값 그대로, 고른 줄은 늘어 있다」를 넣었다.

서버 권위 결과는 짐작하지 말고 받은 값에서 읽는다

서버 권위(결과의 정본을 서버가 정하고 클라이언트는 따르는 구조)에서 클라이언트가 결과를 예측해 팝업을 그리면, 서버가 다른 판정을 한 날 팝업만 거짓말을 한다. 결과 표시는 응답에 실려 온 값에서 읽는다. 받은 값이 이름과 숫자라면 문장으로 조립하지 말고 라벨-값 구조로 각자 제 줄에 둔다(다국어 이유는 5.6). 비슷해 보이는 문장 가족도 뜻이 다르면 함수를 나눈다.

사례 · 서버 결과 표시

무슨 일이 건물 증축 완료 팝업의 레벨을 「지금 레벨 +1」로 짐작하던 자리를, 서버가 돌려준 계정 값에서 레벨을 읽게 고쳤다. 같은 작업에서 지도의 건물 소개 문장과 관리 화면의 현황 문장을 한 함수로 묶었다가, 뜻이 달라 갈라 한 파일에 나란히 됐다.

값을 고르는 함수를 갈랐거나 자리를 비워 뒀으면, 소비하는 자리가 무엇을 부르는지 전부 훑는다

기본값 계산과 실효값을 나눴으면 소비처는 실효값만 부르게 하고, 기본값 함수의 용도(테스트·초기값)를 이름과 주석으로 못박는다. 기능이 붙기 전에 임시 `null`로 채운 `prop`은 기능이 붙는 날 전부 찾아 바꾼다. 옵셔널 `prop`의 `null`은 타입 에러 없이 여러 화면에 퍼진다. 주석에 「화면에서 쓴다」고 적힌 함수에 호출처가 없으면 배선 누락을 의심한다.

사례 · 배선 누락

무슨 일이 화면 13개 전부가 공용 화면 틀에 `account={null}` 을 넘겨, 로그인해도 「로그인 안 함」으로 표시됐다. 세션 조회 함수 주석에는 「화면에 보여 줄 때 쓴다」고 적혀 있었지만 부르는 화면이 없었다. 같은 날, 음성 더빙 언어를 사람이 직접 고르는 기능(고르면 자동 선택을 이긴다)을 넣었는데 재생 함수가 여전히 「UI 언어로 자동 선택」 함수를 부르고 있었다. 그대로 뒀으면 사람이 골라도 재생에 하나도 반영되지 않았다. **잡은 방법** 소비처를 「실제로 쓸 언어」 함수로 바꾸고 자동 선택 함수의 용도를 이름과 주석으로 못박았다.

5.6 다국어 — 열 언어에서 같은 말을 하게 하기

사례의 게임은 열 언어를 지원한다. 번역 함수는 번역이 없으면 원어(한국어)로 물러나도록 짜여 있어 화면은 절대 깨지지 않는다. 그래서 다국어 버그는 대부분 「다른 언어 사용자에게만 한국어가 보인다」「몽골어에서만 글자가 넘친다」 같은, 개발자가 안 보는 화면에서 난다.

문장은 키 하나에 통째로 두고, 엔진은 표시 언어를 모르게 둔다

문장 조각을 이어 붙이면 원어의 어순이 모든 언어에 굳는다. 「시장이 / 훈련소가」처럼 앞말의 받침에 따라 갈리는 한국어 조사는 다른 언어로 번역할 수 없다. 문장은 키 하나에 통째로 두고, 한국어 조사는 한국어일 때만 값에 붙인다. 문장 안의 일부만 강조해야 하면 앞뒤로 쪼개지 말고 `{to}` 같은 자리표시자로 가른다. 서버도 같은 룰 엔진을 돌리므로 엔진은 표시 언어를 알면 안 된다. 엔진은 id만 내고, 번역은 화면 층의 조회 표가 한다. 문구 변경은 다국어 이관과 섞지 않는다.

사례 · 다국어 이관

무슨 일이 전투 UI를 다국어로 옮기면서 엔진의 상태 이상·지형 이름 표는 그대로 두고, 화면이 id로 다국어 이름을 다시 고르게 하는 조회 모듈을 따로 뒀다. 번역문에서 숫자 하나만 색을 칠해야 하는 자리는 자리표시자로 한 문장을 유지한 채 가르고, 「갈라 놓은 조각을 이어 붙이면 원문이 그대로 돌아온다」를 문장 15종 × 열 언어로 회귀에 고정했다.

「없으면 원어로 물러난다」는 누락을 가린다 — 「원어와 다른가」로 기계 검사한다

우아한 물러나기를 둔 자리에는 누락을 세는 검사를 짝으로 둔다. 「키가 있는가」로 보면 번역이 빠진 것과 물러난 것을 구별할 수 없다. 기준 언어와 키 집합이 일치하는지, 값이 원어와 다른지를 스크립트로 확인한다. 언어는 실행 중에 바뀐다는 것도 잊지 않는다. 렌더 캐시의 키에 현재 언어를 넣고, 번역 문자열로 채우는 표는 모듈 상수가 아니라 함수로 만든다. 기능을 잠시 빼더라도 그 번역 키는 바로 지우지 않고, 기계 번역으로 채운 언어는 「검토 필요」로 표시해 정본과 구분한다.

사례 · 번역 누락

무슨 일이 랭킹 화면의 키 49개가 한국어 파일에만 있었고, 메인 화면 키들은 나머지 아홉 언어 파일에 아예 없었다. **왜 조용했나** 번역 함수가 한국어로 물러나 오류도 빈 칸도 없었다. 다른 언어 사용자에게는 랭킹 전체가 한국어로 보였다. **잡은 방법** 아홉 개 파일이 한국어 원본과 키 개수·이름까지 일치하는지 스크립트로 검증했다.

사례 · 언어 전환

무슨 일이 화면 갱신 캐시의 키에 언어가 빠져 있어 언어를 바꿔도 DOM이 그대로였고, 모듈 최상단에서 번역 함수로 채운 표는 로드 시점의 언어로 굳어 있었다. **잡은 방법** 검사를 「원어와 다른가」로 바꾸고, 일본어 한 줄을 일부러 빼서 그 검사가 실제로 잡는지 확인했다.

표시용 이름은 고르는 함수 하나만 거치게 하고, 번역 소스가 붙는 날 읽는 자리를 전부 훑는다

스킬 이름·진영 이름·승패 라벨처럼 화면에 찍히는 이름은 언어에 맞춰 고르는 함수 하나만 지나게 한다. 데이터의 원문 필드를 직접 읽는 자리가 하나라도 있으면 한국어 화면에서는 결과가 같아 안 보인다. 「나중에 채울 자리」를 만들었으면 채우는 날 그 데이터를 직접 읽는 호출부를 전부 검색한다. 한 영역에서 이런 사고를 고쳤으면 같은 패턴이 다른 영역에 남았는지 훑는다. 같은 사건을 두 자리에서 적어야 한다면 주석으로 서로를 가리키게 한다.

사례 · 번역 배선

무슨 일이 마법 이름 다국어는 배선만 되어 있었고 이름을 고르는 함수는 한국어만 냈다. 번역 소스를 붙인 뒤에도 원문 필드를 직접 찍던 자리가 네 곳, 캐릭터 카드의 마법 칩도 한국어를 찍고 있었다. 일주일 뒤 필살기 쪽에서 원문 이름 필드를 직접 읽는 자리가 여섯 군데 나왔다. 3주 전 마법 쪽에서 겪은 사고가 그대로 남아 있었다. **잡은 방법** 다국어 이관 중 코드에 남은 한국어 리터럴을 세다가 발견했다.

배치는 가장 긴 번역으로 확인하고, 한 언어의 배치를 다른 언어에 물려주지 않는다

한국어 스크린샷에서 멀쩡한 화면은 아무것도 증명하지 않는다. 확인 도구에 언어 인자를 두고 가장 긴 언어로 완주시킨다. 특정 언어 전용 배치는 그 언어로 분기하고 나머지는 자동 줄바꿈으로 물려나게 한다. 해법은 반복해서 같다. 언어와 무관한 아이콘, 가장 긴 번역에 맞춘 고정 폭, 끝이 중요한 문장은 말줄임 대신 줄바꿈, 그리고 구운 그림 안에 글자를 넣지 않는 것이다 → 6장 디자인 엔지니어.

사례 · 긴 번역

무슨 일이 몽골어에서만 다섯 군데가 깨졌다. 능력치 머리 글자 겹침, 「부상」을 뜻하는 몽골어 「Шархтай」가 상태 칸을 넘음, 팝업 표 머리 겹침, 병원 팝업 이름 칸 폭 0, 치료 중 줄의 남은 시간 잘림이다. 한국어 화면에서는 전부 멀쩡했다. **잡은 방법** 화면 확인 도구에 언어 인자를 넣어 몽골어로 돌렸다. 그 전에는 한국어 붓글씨 제목의 두 줄 어긋난 배치를 다른 언어에 물려주었다가, 영어 제목이 줄마다 다시 접혀 겹친 일도 있었다.

5.7 화면 검사 — 브라우저 스모크를 거짓말하지 않게 만들기

앞의 여섯 절에서 잡은 버그 대부분이 스모크로 잡혔다. 실제 브라우저에서 화면을 끝까지 돌며 단언하는 검사다. 그런데 스모크 자신도 조용히 틀린다. 아무것도 못 보는 검사는 언제나 초록이다. 검사 범위를 넓혔으면 넓은 부분에서 일부러 한 번 깨 본다. 스모크 설계 일반론은 → 1장 모든 직무 공통.

테스트는 순서·모양·화면 글자가 아니라 뜻 속성(`data-*`)으로 요소를 찾는다

`nth-child`나 모양 클래스, 화면 글자로 요소를 찾으면 UI가 바뀌는 순간 엉뚱한 요소를 누르거나 대상이 0개가 되어 조용히 통과한다. 테스트용 선택자는 `data-action`·`data-side` 같은 뜻 속성에 걸고, 「검사한 대상 수」를

로그로 남겨 0이 되면 의심한다. 마크업을 리팩터링할 때는 상태 속성이 어느 노드로 옮겨 가는지 확인하고 검사도 같은 커밋에서 옮긴다.

사례 · 선택자

무슨 일이 단추가 하나 늘자 `nth-child` 로 단추를 찾던 스모크가 엉뚱한 단추를 눌렀다. **왜 조용했나** 다른 단추도 눌리기는 하므로 오류 없이 다른 경로로 진행했다. 열흘 뒤에는 「적의 마법 목록은 가린다」 검사가 「적군」이라는 화면 글자로 대상을 고르다, 디자인 정리로 그 글자를 빼자 조건이 늘 거짓이 되어 무엇을 노출해도 통과했다. 로그의 「마법 가림」이 「마법 0종」으로 바뀐 것을 보고 알았다.

시간은 단위 테스트가 지키고, 브라우저 검사는 실제 상태에 도달한 뒤 존재와 위치만 본다

연출 시간표는 시각을 인자로 받는 순수 함수로 빼서 단위 테스트로 고정한다. 브라우저 검사가 「몇 칸째인가」 같은 타이밍을 보면 기다리는 사이에 지나가 버린다. 스모크에는 연출 길이를 다시 적지 않는다. 고정 대기 대신 DOM 속성·서버 상태 같은 관측 가능한 조건을 상한 시간과 함께 기다린다. 측정은 스크린샷 같은 느린 동작보다 먼저 한다. 수백 ms짜리 짧은 상태는 밖에서 폴링하지 말고 페이지 안에서 `requestAnimationFrame` 으로 표본을 모아 한꺼번에 돌려받는다. 외부 폴링 간격보다 짧은 상태는 「없는 것」처럼 보인다.

사례 · 고정 대기

무슨 일이 UI 스모크의 필살기 배너 대기는 1300ms였는데 배너 연출을 늘리면서 실제 길이가 2000ms가 되어, 그날부터 산술적으로 통과할 수 없었다. 스모크는 첫 실패에서 멈추므로 그 뒤의 검사 20여 건이 2주 동안 한 번도 안 돌았다. **왜 조용했나** 뒤의 검사는 실패가 아니라 「실행 안 됨」이 됐고, 사람들은 「스모크는 원래 깨져 있다」고 받아들였다. **잡은 방법** 고정 대기를 Playwright의 `waitForFunction` 으로 실제 연출 단계를 기다리게 바꾸고, 끝까지 완주한 것을 보고 나서 통과라고 적었다.

사례 · 타이밍 검사

무슨 일이 스모크가 「두루마리 연출이 1칸에서 시작하는가」를 봤다가, 클릭 뒤 기다리는 0.2초 사이에 이미 3칸이라 실패했다. **잡은 방법** 칸 순서와 11.7초 시간표는 순수 함수 단위 테스트가 고정하고(시간 상수를 일부러 깨면 5건 중 4건이 실패함을 확인), 스모크는 「한 칸이 보인다」와 「설명 불투명도가 1에 닿은 뒤 라벨 위치」만 보게 갈랐다.

「있는가」「제자리에 있는가」「몇 줄인가」는 각각 따로 좌표로 쟀다

잘림·치우침·겹침은 레이아웃을 깨뜨리지 않아 존재 검사(`textContent`)를 통과한다. `data-x="right"` 같은 속성도 코드의 의도를 말할 뿐 실제 렌더를 말하지 않는다. 배치가 중요한 요소는 `getBoundingClientRect()` 로 위치·크기를, `elementFromPoint` 로 보이는 맨 위인지를, `scrollWidth/clientWidth` 로 넘침을, 줄 수와 중심 차로 정렬을 각각 쟀다. 이미지는 실제로 로드됐는지(폭 > 0) 본다. 「그렸는데 안 보인다」는 데이터 → 그리기 명령 → 깊이(z-order) 순으로 좁힌다(마법 대상 하이라이트가 유닛 초상화보다 낮은 깊이에 그려져 덮여 있었다). 레이아웃이 안 맞으면 CSS 값을 더 넣기 전에 `position` 기준 조상부터 쟀다. 고치기 전에 스크린샷으로 실제로 렌더하는 컴포넌트를 특정한다 — 이름이 비슷한 「상세 화면」을 고쳤는데 실제로 뜨는 것은 「상세 카드 모달」이라 바뀐 것이 없었던 적이 있다.

사례 · 레이아웃 측정

무슨 일이 지금 표의 이름 칸은 폭이 0px인데 글자는 DOM에 멀쩡히 있었다. 카드 이름표는 nowrap + 말줄임이라 넘치지 않고 잘린 채였고, 두 줄로 고친 뒤에는 잘림 검사는 통과했지만 글자가 위로 치우쳐 있었다. 포르투갈어 「Não atribuída」(미배정)는 단추 위에 겹쳤다. **잡은 방법** 언어별로 돌리는 측정 스크립트로 좌표, 넘침, 세로 중심차, 줄 수를 따로 잴다. 폭이 나뉜 목록은 첫 칸과 마지막 칸을 다 열었다.

사례 · 중첩 모달

무슨 일이 스킴 설명 모달이 카드(position: relative) 안에서 열려, 배경막의 inset: 0 기준이 카드만 한 상자가 됐다. max-width: 21rem을 넣어도 272px였다. **왜 조용했나** 정적 목업에는 그 조상이 없어 재현되지 않았다. **잡은 방법** 진짜 컴포넌트를 가짜 데이터로 마운트하는 임시 페이지에서 잴다. 모달을 바깥 컴포넌트로 올려 형제 모달과 같은 기준 조상을 공유하게 하자 폭과 세로 위치가 저절로 맞았다.

검사의 재료가 거짓이면 검사도 거짓이다 — 기준표는 원본을, 픽스처는 실제 같은 값을, 셋업은 적용 확인을

테스트에 기준표를 복사해 두면 원본이 바뀌어도 테스트는 옛 표 기준으로 통과한다. 제품 코드의 원본을 노출해 읽는다. 픽스처가 빈 값·같은 값 같은 기본값에 기대면 그 값이 끄는 분기는 시험되지 않는다(5.1의 좌표 없는 가짜 상태). 실제처럼 채운 픽스처를 따로 둔다. 셋업은 헬퍼의 반환값을 가정해 가공하지 말고, 셋업이 실제로 적용됐는지 서버 값으로 먼저 확인한다. 「없는 예시」는 시간이 지나면 생긴다. 일부러 만든 가짜 키를 쓰고, 언어·시계 같은 전역 상태를 바꾸는 테스트는 try/finally로 되돌린다. 한 기계에서 두 사용자를 흉내 낼 때는 계정 키와 기기 식별자를 둘 다 가르다 — 브라우저 탭 둘은 localStorage를 공유하고, 계정만 가르면 서버가 둘째 접속을 같은 사람의 재접속으로 처리한다.

사례 · 테스트 셋업

무슨 일이 스모크가 시드 데이터의 연구 항목 id를 「헬퍼가 준 id + -plus」로 만들었는데, 그 헬퍼가 이미 개량형 id를 돌려줘 ...-plus-plus가 됐다. **왜 조용했나** 서버가 모르는 id를 오류 없이 버렸다. **잡은 방법** 화면이 「되돌릴 연구가 없다」고 말해서 잡았다.

사례 · 테스트 예시

무슨 일이 「번역이 없으면 한국어로 몰러난다」 테스트의 예시 키가 하필 실제 키였고, 번역을 채우자 깨졌다. 뒤따라 다른 테스트까지 깨졌는데, 앞 테스트가 언어 설정을 되돌리기 전에 던졌기 때문이다.

검사가 없는 자리는 고치기 전에 현재 출력을 먼저 못 박고, 새 모드에는 끝까지 도는 스모크를 붙인다

이관이나 리팩터링 전에 현재 출력을 통째로 회귀로 고정한다(골든 출력 테스트). 그 배열이 곧 사양이고, 고칠 때마다 「의도한 변경인가」를 묻게 된다. 엔진 회귀만으로는 화면이 안 잡힌다. 기존 엔티티 집합 밖의 것(명단에 없는 NPC 등)이 들어오는 새 모드에는 그 모드를 처음부터 끝까지 도는 브라우저 스모크를 함께 만든다. 스모크로 잡은 버그는 가능하면 단위 회귀로 내려 고정한다.

사례 · 골든 출력

무슨 일이 전투 로그 문장을 만드는 함수를 부르는 테스트가 0건이었다. 다국어 이관 전에 이벤트 한 벌로 로그 28줄을 통째로 고정하자, 묵은 버그 돌이 바로 나왔다. 받침 없는 스킬 이름 뒤에 조사를 손으로 적어 둔 「「○○」이 무산됐다」와, 진영 내부 id가 그대로 노출된 「P1 승리」다. **왜 조용했나** 검사가 없는 자리라 몇 달 동안 아무도 출력을 대조하지 않았다.

사례 · 새 게임 모드

무슨 일이 NPC 무리를 막는 방어전 모드를 넣자, 플레이어 캐릭터 명단 밖의 NPC가 판에 서는 순간 전투 UI 여덟 파일이 명단만 보는 조회 함수로 캐릭터를 찾다 `undefined`의 속성을 읽으며 죽었다. **왜 조용했나** 회귀는 엔진만 돌렸고 UI는 명단 밖 유닛을 만난 적이 없었다. **잡은 방법** 그 모드용으로 새로 만든 스모크가 진짜 계정으로 한 판을 끝까지 돌며 잡았다. 조회는 「전투 참가자 전체」를 보는 함수 하나로 모았다.

작업 전후 점검표

- ☐ 새 연출의 모든 구간이 공용 커서 위에 있고, 커서를 미는 호출(카메라)이 각 분기의 첫 줄인가?
- ☐ 연출 도중에 필요한 옛 값을 화면이 기억하고 있지 않은가 — 이벤트에 실려 오는가?
- ☐ 연출이 도는 동안 화면이 읽는 단계값과 단추 표시가 「입력 대기」라고 거짓말하지 않는가?
- ☐ 소리가 이벤트 도착이 아니라 그림의 시각표에 붙어 있고, 갱신을 멈추는 게이트에 얼지 않는가?
- ☐ 엔진에 아무것도 안 보내는 조작과 시간 경과가 다시 그리기를 부르는가?
- ☐ 열린 화면이 데이터 사본이 아니라 id를 들고 있는가? 상태가 셋 이상인 분기가 하나씩 적혀 있는가?
- ☐ 로딩을 비동기로 바꿨다면, 「마운트 때 한 번」 도는 effect가 데이터 도착 순간에도 도는가?
- ☐ 직접 만든 전역 리스너·외부 엔진 자원·넘겨준 연결에 해제 경로와 책임자가 있고, 단기가 멎듯한가?
- ☐ 기다리지 않는 저장에 쓰기 큐를 지나고, 실패가 사용자에게 보이는가?
- ☐ 엔진이 허용하는 수(특히 하나뿐인 경우)마다 화면에서 그것을 낼 단추가 있고, 칠한 칸이 화면 안에서 실제로 놀리는가?
- ☐ 무시되는 입력이 로그를 남기고, 한 번만 나가야 할 요청이 동기 ref로 잠겨 있는가?
- ☐ 화면이 보여 주는 숫자가 엔진 함수나 서버 응답에서 오는가 — 화면이 공식을 다시 적고 있지 않은가?
- ☐ 새 문구가 키 하나에 통째로 들어갔고, 원문 필드를 직접 읽는 자리가 남아 있지 않은가?
- ☐ 가장 긴 번역(몽골어 등)으로 화면을 끝까지 돌려 넘침·겹침·0쪽을 좌표로 잰는가?
- ☐ 스모크가 뜻 속성으로 요소를 찾고, 고정 대기 대신 실제 상태를 기다리며, 끝까지 완주했는가 — 새 검사는 일부터 깨서 실패하는 것을 봤는가?

6장

디자인 엔지니어

그림을 화면에 올리고, 화면이 그림대로 보이게 만드는 사람

이 장은 에셋 파이프라인(원본 그림을 게임이 읽는 파일로 굽는 도구)과 화면 스타일(CSS)을 맡는 사람을 위한 것이다. 이 자리에서 나는 실수는 거의 다 오류 메시지가 없다. CSS는 규칙이 지거나 아예 안 걸려도 경고하지 않는다. 굽기 도구는 「몇 장 만들었다」만 말하고, 잘 만들었는지는 말하지 않는다. 이미지 처리는 수치가 정상이어도 그림이 깨질 수 있다. 그래서 주니어가 가장 자주 조용히 틀리는 것은 「봤으니 됐다」고 믿는 일이다. 이 장의 원칙은 대부분 「무엇을 재고, 어디서 보고, 무엇을 구조로 막을 것인가」다. 사례는 웹·모바일로 내는 턴제 전술 게임(체스 기물처럼 움직이는 캐릭터 260명, 가차와 캐릭터 육성, 열 개 언어)을 두 달 동안 만들며 실제로 밟은 것들이다.

이 장의 원칙

1. 새 화면의 클래스는 처음부터 화면 접두사로 시작한다
2. 덮어쓸 규칙의 세기는 `:not()` 안까지 세고, 이겼는지는 계산된 값으로 확인한다
3. 단축 속성이 무엇을 초기화하는지 기억한다
4. 화풍을 그림 테두리로 바꾸면 상태 표시를 처음부터 다시 만든다
5. 쌓임 맥락과 CSS 애니메이션은 다른 효과를 오류 없이 지운다
6. 잘리면 안 되는 것은 flex 비율에 맡기지 말고 구조로 지킨다
7. 트랙 폭은 내용이 아니라 틀에 못 박고, 「이 칸만 넓다」면 `min-width: auto` 부터 의심한다
8. 오버레이는 컨테이너 밖에 그리고, 크기가 이상하면 기준 조상부터 센다
9. 빈 요소와 띄운 판은 클릭을 통과시킨다
10. 단추 이름은 접하지 않게 하고, 긴 글자는 가장 썸 쪽에서 흡수한다
11. 크기 지정은 눈대중으로 환산하지 말고 실측한 상자에 곱한다
12. 스프라이트의 위치값은 프레임 경계에서 계산하고 찍어서 확인한다
13. 줄여서 맞춘 값과 이유가 걸린 값은 이유가 사라지면 함께 지운다
14. 고치기 전과 후를 재서, 수정이 정말 무언가를 바꿨는지 확인한다
15. 정보가 가장 많은 원본을 받고, 화면이 이상하면 파이프라인을 거꾸로 짚는다
16. 저장 형식은 런타임이 소비하는 방식과 대상 플랫폼으로 고른다
17. 표시 조정과 정규화는 런타임이 아니라 굽는 단계에서 끝낸다
18. 파라미터를 고치면 전체를 다시 굽고, 건너뛰기는 존재가 아니라 내용으로 판단한다
19. 굽기 도구는 자기 이름만 지우고, 경고는 측정값으로 내고, 짝이 될 경고는 나란히 찍는다
20. 그림 시트는 등분하지 말고 픽셀로 경계를 찾고, 찾은 수를 단언한다
21. 표본은 대표값 하나로 접지 말고, 임계값은 정상값의 최댓값을 재고 정한다

22. 한 축에서 상충이 풀리지 않으면 다른 축을 찾고, 틀림의 무늬를 본다
23. 자기 출력을 다시 넣는 단계와 정답 표본은 둘 다 의심하고 잔다
24. 자동 처리는 다수에 맞추고, 예외는 표시와 수동 통로로 처리한다
25. 다른 바탕에 올릴 소재는 가장자리와 그림자를 다시 만든다
26. 투명 이미지는 모든 바탕 위에서, 크게, 극단 표본으로 보고 판단은 픽셀 통계로 한다
27. 짝 채우는 그림은 가장 큰 프레임에서, 에셋이 다 온 뒤에 본다
28. 「안 보인다」 「폰트가 다르다」 는 굵기·색보다 좌표계를 먼저 본다
29. 장식 효과는 결정적으로 만들고, 불변 성질은 좌표로 검사한다
30. 목업은 결함을 찾는 데 쓰고, 느낌은 실제 화면에서 판단한다

6.1 CSS는 틀려도 말하지 않는다

CSS에는 컴파일 오류가 없다. 이름이 겹쳐도, 세기(specificity, 어느 규칙이 이기는지 정하는 점수)에서 져도, 단축 속성이 다른 값을 지워도 브라우저는 다른 규칙을 조용히 적용한다. 타입 검사와 테스트는 전부 통과하고, 화면을 직접 봐야만 안다. 우리는 같은 종류의 사고를 한 주에 다섯 번씩 냈다. 막는 방법은 셋이다. 이름 규약, 세기 계산, 그리고 계산된 값(`getComputedStyle`)을 읽는 검사.

새 화면의 클래스는 처음부터 화면 접두사로 시작한다

전역 CSS에 `.panel` · `.card` · `.bar` · `.head` 같은 흔한 이름을 새로 쓰면, 어딘가에 이미 있는 같은 이름의 규칙이 끼어든다. 새 화면은 `.shop-` · `.inv-` 처럼 화면 접두사로 시작하고, 접두사 뒤에도 `head` · `row` 같은 흔한 낱말은 피한다. 접두사를 붙여도 같은 파일 안에서 겹칠 수 있고, 뜻이 같다고 같은 이름을 쓰면 그냥 겹친다. 여러 화면이 함께 쓰는 범용 클래스(팝업 `.modal`, 선택지 `.opt` 등)에 새 화풍을 입힐 때는 `.modal[data-modal="settings"]` 처럼 속성 선택자로 좁힌다.

사례 · 이름 충돌

무슨 일이 새 메뉴 화면의 판때기에 `.panel` 을 붙였더니, 전투 화면의 플로팅 명령 패널 규칙(`position: absolute`, 폭은 판 크기의 3/8)이 먹어 판때기가 오른쪽 아래에 좁게 붙었다. 사흘 전에도 같은 이름이 다른 옛 규칙(`position: relative`)과 부딪혀 패널이 좌상단에 붙은 적이 있었다. 같은 이름이 두 번 사고를 냈다. **왜 조용했나** 이름이 겹쳐도 CSS는 오류가 없고 테스트도 통과한다. **잡은 방법** 화면을 직접 보고 찾았다. 이후 실제 좌표를 재는 스모크 테스트(실제 화면을 띄워 끝까지 눌러 보는 자동 검사)를 넣었다.

사례 · 접두사 안의 충돌

무슨 일이 캐릭터 목록 화면에서 접두사를 붙인 `-head` 클래스 하나를 표 머리(grid)와 상세 머리(flex)에 함께 써서 한쪽이 무너졌다. 같은 주에 도시 관리 화면을 만들며, 메인 화면에 이미 있는 「도시」 클래스와 뜻이 같다고 같은 이름을 쓰려다 멈췄다. **잡은 방법** 표 머리와 상세 머리에 서로 다른 이름을 주고, 도시 관리 화면에는 별도 접두사를 붙였다.

덮어쓸 규칙의 세기는 `:not()` 안까지 세고, 이겼는지는 계산된 값으로 확인한다

화면 단위로 넓은 화풍 규칙을 깔면, 그 뒤에 쓰는 예외는 세기를 넘겨야 이긴다. `:not()` 안의 클래스도 세기에 들어간다. `button.x { font: inherit }` 같은 리셋 규칙(요소+클래스)은 클래스 하나짜리 글자 크기 규칙을

이긴다. 세기가 같으면 파일 순서로 갈린다. 예외를 쓴 뒤에는 「덮었다」고 믿지 말고 `getComputedStyle` 로 실제 값을 읽는다. 일부 요소만 멀쩡한 화면은 에셋 문제처럼 보이지만 대개 우선순위 문제다. 쪽 넘김 단추 같은 공용 부품은 넓은 규칙에서 처음부터 `:not()` 으로 빼 둔다.

사례 · 화풍 교체

무슨 일이 목록 화면의 단추에 목판 그림을 입히는 규칙은 `.screen-x .btn:not(.ghost):not(.primary)`, 클래스가 넷이다. 이것을 클래스 셋짜리 규칙으로 덮으려다 실패했다. 속성 선택자 한 칸이 더 붙어 우연히 이긴 [처음]·[마지막] 단추만 멀쩡했다. **왜 조용했나** 일부만 정상이라 「그림 하나가 빠졌다」로 읽혔다. **잡은 방법** 정상인 단추와 아닌 단추를 나란히 놓고 선택자 세기를 손으로 썼다.

사례 · 같은 주에 다섯 번

무슨 일이 팀 편집 화면에서 같은 사고가 한 주에 다섯 번 났다. 붉은 삭제 단추 `.btn.del (0,3,0)`이 화면 단위 규칙 `.screen-y .btn:not(.ghost):not(.primary) (0,4,0)`에 파일 순서와 상관없이 졌다. 그런데 주석에는 「순서로 갈린다」고 틀리게 적혀 있었다. 쪽 넘김 단추도 `.btn` 이라 참나무 액자 그림에 갇혔고, 단추 요소의 `font: inherit (0,1,1)`가 목록 글자 크기(0,1,0)를 이겼다. **잡은 방법** 공용 부품을 넓은 규칙에서 `:not()` 으로 빼고, 예외를 쓸 때마다 계산된 값으로 승패를 확인했다.

단축 속성이 무엇을 초기화하는지 기억한다

border: 단축 속성은 `border-image` 를 초기화한다. 9분할 이미지(그림을 아홉 조각으로 나눠 모서리는 그대로 두고 변만 늘리는 테두리, CSS `border-image`)를 입힌 클래스가 `border:` 를 쓰는 클래스와 같은 세기로 겹치면, 파일에서 아래에 있는 쪽이 이겨 그림이 통째로 사라진다. 장식 그림을 입힌 요소는 선택자를 한 칸 세게 적어 (`.modal.done` 처럼) 겹침을 끊는다. 크기를 줄힐 때는 `padding` 을 조절하고, 장식이 든 `border-width` 는 건드리지 않는다. 요점이 되는 선언(`display: inline-flex` 등)에는 「이것이 빠지면 무엇이 무너진다」는 경고 주석을 단다.

사례 · 사라진 액자

무슨 일이 건물 증축 완료를 축하하는 팝업의 액자 그림이 통째로 사라졌다. 액자를 입힌 클래스 하나가 파일 아래쪽 범용 `.modal` 의 `border:` 에 졌다. 앞서 기술 설명 팝업에서 한 번 났을 자리였다. 같은 주에는 이미 `*`로 닫힌 주석 뒤에 설명 세 줄을 덧붙였더니, 파서가 그 글과 바로 뒤의 규칙 하나를 한 덩어리로 삼켜 버렸다. 그 규칙의 `inline-flex` 가 날아가 「글씨는 커졌는데 아이콘만 사라졌다」. **왜 조용했나** 두 경우 모두 브라우저는 경고 없이 다른 규칙을 적용한다. **잡은 방법** 스크린샷 눈검사로 증상을 보고 거꾸로 추적했다. 액자는 선택자를 두 클래스로 세게 적어 고쳤다.

화풍을 그림 테두리로 바꾸면 상태 표시를 처음부터 다시 만든다

테두리가 그림이면 `border-color` 와 `inset box-shadow` 는 아무 일도 안 한다. 9분할 그림의 `fill` (가운데 칸 채우기)이 안쪽 그림자도 덮는다. 리스킨(화풍 교체)을 하면 「선택됨」「눌림」같은 기존 상태 표시를 전부 그림 교체나 필터 기반으로 다시 만들고, 눈 대신 계산된 값으로 확인한다. 스타일을 다른 자리로 옮길 때는 모양과 배색 변수를 한 묶음으로 옮긴다. 펼친 상태까지 화풍이 필요한 컨트롤은 네이티브 `<select>` 가 아니라 버튼 + 뜨는 패널로 처음부터 만든다. 닫힌 상자만 CSS로 꾸며지고 펼친 목록은 OS가 그린다.

사례 · 리스킨

무슨 일이 팀 편성 화면을 그림 테두리로 리스킨하자 「고른 칸」을 표시하던 상태 규칙 넷이 전부 죽었고, 참나무 목판 위의 `inset box-shadow`도 보이지 않았다. **왜 조용했나** 효과가 없어도 오류가 없다. 「눌렀는데 아무 표시가 없다」로만 남는다. **잡은 방법** `getComputedStyle`의 `borderImageSource · filter · backgroundColor`를 읽어 「고른 칩이 옥색인가」를 검사했다. 랭킹 화면의 필터 `<select>`도 같은 이유로 펼친 목록만 OS 모양이라 커스텀 드롭다운으로 바꿨다.

쌓임 맥락과 CSS 애니메이션은 다른 효과를 오류 없이 지운다

`container-type · transform · isolation`은 쌓임 맥락(stacking context, 자식들이 바깥과 따로 합성되는 묶음)을 만들고, 그 안의 `mix-blend-mode`는 바깥과 섞이지 못한다. 블렌드가 필요한 요소의 조상에는 이것들을 두지 않는다. CSS 애니메이션은 인라인 스타일을 이기므로, JS 시간표가 주는 속성(`opacity` 등)에는 CSS 애니메이션을 걸지 않는다. 마스크는 그림이 아니라 상자에 걸리므로, 마스크를 걸 상자는 그림 비율에 맞춘다.

사례 · 연출 합성

무슨 일이 두루마리가 펼쳐지는 연출에서 종이 요소에 건 `container-type` 때문에 장면의 블렌드가 두루마리가 아니라 투명한 바닥하고만 섞였다. 설명·라벨에 남아 있던 먹 번짐 CSS 애니메이션은 시간표 코드가 인라인으로 주는 `opacity`를 덮어 겹침 순서를 깨뜨렸다. `object-fit: contain`으로 생긴 여백에서는 마스크 경계가 딱딱하게 남았다. **잡은 방법** 세 효과를 하나씩 끄고 켜며 원인을 갈랐다.

6.2 레이아웃 — 누가 줄고 누가 늘어나는가

`flex`와 `grid`는 공간이 모자라거나 남을 때 누구를 줄이고 누구를 늘릴지 기본값으로 정한다. 그 기본값(`flex-shrink: 1`, `min-width: auto`, 트랙 `auto`)은 여유 있는 화면에서는 아무 문제가 없다가, 내용이 넘치는 데이터·좁은 해상도·다른 인원 구성에서만 드러난다. 증상은 「글자가 없다」 「그림이 크다」처럼 다른 곳의 문제로 보인다.

잘리면 안 되는 것은 flex 비율에 맡기지 말고 구조로 지킨다

세로 `flex` 안의 줄은 기본이 `flex-shrink: 1`이라, 영역이 얇아지면 필요한 높이보다 작게 늘린다. 내용이 잘리면 안 되는 자식에는 `flex: none`을 준다. 저장 단추처럼 절대 줄어들면 안 되는 것은 `flex` 비율 조정으로 지키지 말고, 스크롤 묶음과 바닥 판을 갈라 구조로 빼낸다. 비율을 만지면 잘리는 자리만 옮겨 다닌다. 검증은 여유 있는 기본 크기가 아니라 영역을 일부러 극단적으로 좁히거나 내용이 넘치는 데이터로 한다.

사례 · 사라진 글자

무슨 일이 전투 화면의 시스템 메시지 창에서 상자만 그려지고 글자가 사라졌다. 줄 높이가 필요한 19px보다 작은 15.5px로 눌러 있었다. **왜 조용했나** 디버그 함수로 보면 줄은 다 들어 있었다. 상단 정보 표시(HUD)가 두꺼운 5대5 대전에서만 늘 냈고 3대3은 멀쩡해서 「가끔 안 보인다」로만 보였다. **잡은 방법** 기획자가 플레이 중에 발견했고, 해상도 셋에서 줄 높이를 실측했다. 스모크 테스트가 영역을 20px로 좁혀 그 성질을 지키게 했다.

사례 · 밀려난 저장 단추

무슨 일이 팀 편성 화면의 [등록 완료] 단추가 화면 밖으로 밀렸다. 첫 원인은 아무 규칙에도 안 걸린 클래스 낱말 (grow) 하나였다. 고치자 리스킨 테두리가 다시 밀었고, 테두리를 줄이자 캐릭터 표가 0줄이 됐고, min-height 를 옮기자 잘린 자리만 바뀌었다. **잡은 방법** 캐릭터 44명을 가진 시험 계정을 심어 일부러 넘치는 상태를 만들어 재현하고, 바닥 판을 스크롤 묶음 밖으로 뺐다.

트랙 폭은 내용이 아니라 틀에 못 박고, 「이 칸만 넓다」면 min-width: auto 부터 의심한다

grid의 줄·열을 auto 나 max-content 로 두면 내용이 틀을 정한다. 줄마다 제 grid를 갖는 목록은 줄마다 폭이 달라지고, 구성이 다른 화면은 한 구성에서만 넘친다. 줄은 minmax(0, 1fr) , 열은 고정 폭으로 못 박는다. grid·flex 아이템의 기본값 min-width: auto 는 nowrap 인 긴 글자가 칸을 넓히게 만든다. 안쪽 요소에 min-width: 0 을 줘도 조상 아이템이 auto 면 소용없으므로, 아이템 자체에 준다. 증상을 에셋 탭으로 돌리기 전에 레이아웃 계산부터 본다.

사례 · 구성마다 다른 화면

무슨 일이 전투 화면 아래 캐릭터 카드 띠의 grid 줄이 auto 라, 3대3에서만 카드가 띠를 넘어 HP·대기 시간·고유 스킬 줄이 잘렸다. 5대5는 우연히 높이가 맞았다. **왜 조용했나** 5대5만 확인했다. 한 장면만 보면 통과한다. **잡은 방법** 3대3을 열어 보고 「카드 높이 > 칸 높이」 스모크 검사를 넣은 뒤, 수정을 일부러 되돌려 검사가 실제로 실패하는 것까지 확인했다.

사례 · 커 보이는 그림

무슨 일이 장비 화면에서 방어구 그림이 커 보여서, 종류별 축소와 품목별 축소까지 두 라운드를 돌렸다. 진짜 원인은 grid 아이템의 min-width: auto 였다. nowrap 인 긴 이름이 그 열을 넓혔고, 같은 열을 쓰는 다른 줄까지 함께 넓어졌다. 줄마다 독립 grid인 다른 표에서는 단추 칸을 auto 로 두자 단추가 없는 머리줄만 이름 칸이 넓어졌다. **잡은 방법** 기획자가 「액자 자체가 옆 칸보다 넓다」고 짚었다.

오버레이는 컨테이너 밖에 그리고, 크기가 이상하면 기준 조상부터 센다

팝업을 position: relative 가 걸린 판 안에 두면 판 크기만 한 팝업이 된다. 「자식 전부에 position: relative 」 같은 상위 규칙(.screen > *)에 걸리면 화면 가운데가 아니라 내용 맨 아래에 흘러 붙는다. 오버레이는 조각(fragment)이나 포털로 컨테이너 밖, 화면 최상위 바로 밑에 그린다. 크기가 안 맞으면 CSS 숫자를 더 넣기 전에 inset: 0 의 기준이 되는 조상이 누구인지 센다. 이름과 단추를 함께 가운데 두는 줄은 flex가 아니라 1fr auto 1fr 격자로 둔다. 검사는 「떠 있는가」와 「제자리에 있는가」를 따로 둔다.

사례 · 흘러내린 팝업

무슨 일이 배경 그림을 깬 화면 위의 팝업이 2주 전부터 화면 한가운데가 아니라 내용 맨 아래에 붙어 있었다. 배경 화면의 「모든 자식에 position: relative 」 규칙이 팝업 뒷막의 absolute 를 덮었다. **왜 조용했나** 팝업 스모크 테스트는 「떠 있는가」만 봤다. **잡은 방법** 가장 큰 프레임(700px) 눈검사로 찾고, 팝업이 프레임을 실제로 덮는지 좌표로 확인하는 검사를 더했다.

사례 · 판 안의 팝업

무슨 일이 캐릭터 육성 화면의 [이름 변경] 팝업을 판 안에 두자 판 크기만 해줬다. flex로 이름과 단추를 가운데 두자 단추가 이름을 52px 옆으로 밀었다. **잡은 방법** 확인 도구가 좌표로 「제자리에 있는가」를 봤다. 팝업 안에서 또 팝업을 여는 중첩 모달의 기준 조상 문제는 → 5장 클라이언트 엔지니어에서 다룬다.

빈 요소와 띄운 판은 클릭을 통과시킨다

자리만 지키는 빈 요소나 판 위에 띄운 패널은 투명해도 그 아래 클릭을 삼킨다. 빈 자리 요소와 오버레이 컨테이너에는 `pointer-events: none`을 주고, 조작하는 요소만 `auto`로 둔다. 그림에 붙어야 하는 클릭 영역은 그림과 같은 층에 둔다. 브라우저 자동화의 실제 클릭·호버는 가로채기를 알려 주므로 좌표 검사보다 낫다.

사례 · 투명한 벽

무슨 일이 메인 화면의 빈 지도 자리 `<div>` 하나가 「배경 위 내용은 z-index 2」 규칙에 걸려 떠올랐다. 그 결과 z-index 0인 그림 층 안에 있는 건물 클릭 영역이 전부 막혔다. **왜 조용했나** 투명한 빈 요소라 눈에 안 보이고, CSS는 오류를 내지 않는다. **잡은 방법** Playwright의 `hover()`가 「이 div가 포인터 이벤트를 가로챈다(intercepts pointer events)」고 알려 줬다.

단추 이름은 접하지 않게 하고, 긴 글자는 가장 썸 쪽에서 흡수한다

단추 라벨에는 `white-space: nowrap`을 기본으로 걸고, 길이는 설명문이 접하며 감당하게 한다. 안내문은 그것이 설명하는 단추 바로 밑에 둔다. 긴 번역 같은 극단값 하나 때문에 레이아웃 전체를 바꾸지 않는다. 레이아웃 조정, 콘텐츠(이름) 조정, 지정값의 미세 조정 중 가장 썸 쪽을 고르고, 지정값을 바꾸면 그 이유(몇 px 모자랐는지)를 남긴다. 다국어 화면 검사 방법은 → 5장 클라이언트 엔지니어를 본다.

사례 · 꺾인 단추 이름

무슨 일이 단추 라벨에 `nowrap`이 없어 「튜토리얼 시나리 / 오」로 꺾였다. 화면 끝에 몰아 둔 안내문은 어느 단추 이야기인지 알 수 없었다. **왜 조용했나** 둘 다 오류가 없고, 가장 큰 프레임 눈검사에서만 보였다.

사례 · 다국어

무슨 일이 열 개 언어 중 긴 번역 이름 때문에 캐릭터 카드를 표로 바꿀지 고민했다. 결국 두 줄 + 글자 크기 조정으로 카드를 지켰고, 그래도 세 줄이 되는 두 언어(이탈리아어·몽골어)는 번역 이름을 줄였다. 기획자가 지정한 패널 폭 $\times 1.1(391\text{px})$ 은 한국어 최장 줄 392px에 1px 모자라 $\times 1.114$ 로 밀고 그 이유를 적었다. 모든 언어를 한 줄에 넣으려면 $\times 1.34$ 가 필요해 화면을 잡아먹었다.

6.3 숫자는 재서 정하고, 이유와 함께 지운다

스타일 값은 눈대중으로 정하기 쉽고, 한 번 들어가면 이유 없이 남는다. 정할 때는 실측한 값에서 출발하고, 고친 뒤에는 실제로 바뀌었는지 재고, 이유가 사라지면 값도 지운다. 화면·문서에 뜨는 숫자와 색은 손으로 옮겨 적지 않고 출처(상수·계산 함수·원본 그림)에서 부른다 → 2장 게임 디자이너.

크기 지정은 눈대중으로 환산하지 말고 실측한 상자에 곱한다

「1.3배 키워 달라」는 요청을 받으면 지금 렌더된 상자의 치수를 재고 거기에 곱한다. 스크린샷을 보고 rem으로 환산하면 틀린다. 「줄 높이 × 줄 수」처럼 CSS 값을 손으로 합산해도 실측과 어긋난다. 높이는 가장 짝 찬 상태를 재서 고정하고, 스모크 테스트가 그 상자를 켜다. 치환 요소()에는 퍼센트로 상자를 준다. 절대 배치된 의 `width: auto`는 원본 폭으로 풀린다.

사례 · 배수 요청

무슨 일이 「폭 1.3배, 높이 1.2배」를 스크린샷 눈대중으로 환산했다가 오히려 더 작게(74×28) 만들었다. 「줄 높이 × 쪽 크기」를 손으로 계산한 목록 표는 쪽마다 3px씩 줄었다. 절대 배치된 무기 그림은 원본 폭으로 풀려 화면 밖으로 사라졌다. **잡은 방법** 렌더된 상자를 재고 곱하는 방식으로 다시 했고, 스모크 테스트가 상자 치수를 단언하게 했다.

스프라이트의 위치값은 프레임 경계에서 계산하고 찍어서 확인한다

`background-position` 퍼센트로 N칸 가로 스프라이트(여러 프레임을 한 장에 나란히 붙인 그림)를 `steps(N)`으로 넘길 때, 종료 위치는 100%가 아니라 $100 \times N / (N-1)$ %다. 퍼센트 위치는 「상자와 그림의 크기 차이」에 대한 비율이라 직관과 다르다. 프레임 경계를 직접 계산해 넣고, 계산된 값이 그 경계에 정확히 찍히는지 확인한다.

사례 · 스프라이트 애니메이션

무슨 일이 레벨업 도장 연출(3프레임)의 종료 위치를 100%로 두자 단계값(0/33.3/66.7%)이 실제 프레임 경계(0/50/100%)와 어긋나 두 그림이 겹쳐 보였다. **잡은 방법** 150%로 고치고, `getComputedStyle().backgroundPosition`이 정확히 0% / 50% / 100%로 찍히는지, 세 프레임이 겹침 없이 갈라지는지를 확인했다.

줄여서 맞춘 값과 이유가 걸린 값은 이유가 사라지면 함께 지운다

「글자를 줄여 맞춘다」는 대개 다른 무언가가 자리를 잡아먹고 있다는 신호다. 그 원인을 치운 뒤 줄였던 값을 되돌리지 않으면 「왜 이 글자만 작지」가 근거 없이 남는다. 간격·폭 규칙도 마찬가지다. 내용을 빼거나 요소를 옮길 때는 그 요소에 걸린 규칙과, 그 이유를 적은 주석을 함께 찾아 지운다. 남기면 주석이 이미 없는 이유를 설명하게 된다.

사례 · 남은 축소값

무슨 일이 캐릭터 명패가 좁아서(레벨 배지가 폭을 차지했다) 글자를 .85rem에서 .72rem으로 줄였다. 나중에 배지를 빼자 줄인 이유가 사라졌다. **잡은 방법** 실측으로 원래 값을 복원했다.

사례 · 자리를 옮긴 요소

무슨 일이 단추 판의 아래 여백은 「마지막 줄이 안내문」이라서 있었는데, 안내문이 빠지자 판이 기울어 보였다. 삭제 단추의 폭 규칙(`width: 3.3rem`)은 목록 줄 끝의 작은 단추용이었는데, 그 단추가 편성 화면의 폭 전체를 쓰는 단추로 옮겨 갔다. 규칙을 남겼으면 새 자리에서 짜부라졌을 것이다.

고치기 전과 후를 재서, 수정이 정말 무언가를 바꿨는지 확인한다

전후 차이가 0이면 그 수정은 지금 환경에서 아무 일도 안 한 것이다. CSS에서는 부모의 `max-width` 가 `width` 수정을 조용히 누르고, 빌려 쓴 클래스의 규칙이 다른 컨테이너 안에만 있어 안 걸리기도 한다. 폭을 바꿀 때는 `max-width / min-width` 도 함께 보고, 다른 화면의 클래스를 빌리지 말고 제 이름을 만든다. 회전·확대 장식은 외접 상자 기준으로 공간을 잡는다. 라이브러리 버전에 의존하는 방어 코드는 남길 수 있지만, 확인한 사실과 추측을 구분해 적는다.

사례 · 하지 않아도 됐던 수정

무슨 일이 초상화를 축소할 때 투명 픽셀의 색이 번질 것을 걱정해 프리멀티플라이(색에 알파를 미리 곱해 두는 방식) 경유를 넣었는데, 출력이 바이트 단위로 같았다. 쓰던 이미지 라이브러리(Pillow 12)는 원래 번지지 않았다. **왜 조용했나** 재 보지 않았으면 「이렇게 안 하면 번진다」는 주석과 함께 없는 문제를 해결했다고 남길 뻔했다. **잡은 방법** 전후 출력을 비교하고, 합성 시험 이미지(왼쪽 불투명 흰색 | 오른쪽 투명한 빨강)로 따로 켜다.

사례 · 안 걸린 수정 넷

무슨 일이 한 주에 넷이 났다. 부모의 `max-width: 19rem` 이 `width` 수정을 눌렀다. 다른 화면에서 빌려 쓴 클래스가 그 화면의 컨테이너 안에서만 걸려, 경고문이 본문보다 큰 글자로 났다. 45° 회전한 네모의 외접 상자가 $\sqrt{2}$ 배라 가로 스크롤이 생겼다. `margin-top: auto` 둘이 빈자리를 나눠 가져 목록이 가운데로 흘렀다. **잡은 방법** 스모크 테스트가 「상단에 눌렀는가」 같은 치수를 재게 했다.

6.4 에셋 파이프라인 — 원본에서 화면까지

원본 그림과 화면 사이에 굽기 도구를 한 층 두면, 원본을 갈아 끼워도 화면 코드는 한 줄도 바뀌지 않는다
→ 3장 시스템 엔지니어. 대신 그 도구가 새 실패 지점이 된다. 도구는 「몇 장 만들었다」만 보고하고, 잘 만들었는지는 말하지 않는다. 이 절은 도구가 정보를 잃지 않고, 섞이지 않고, 정직하게 보고하게 만드는 원칙이다.

정보가 가장 많은 원본을 받고, 화면이 이상하면 파이프라인을 거꾸로 짚는다

원본 정보를 지우는 전처리를 먼저 거친 파일을 받으면, 뒤 단계에서는 복원할 근거가 없다. 배경을 이미 지운 PNG보다 크로마키 합성본(단색 배경 위의 그림)을 받아 알파까지 다시 푸는 쪽이 정확하다. 화면이 이상하면 CSS로 우회하지 말고 원본 → 중간 → 결과물을 순서대로 픽셀로 찍어 차이가 생긴 단계를 찾는다. 「같은 그림인데 다르게 보인다」면 증상을 재서 보정하기 전에 둘이 왜 달라졌는지부터 찾는다. 새 에셋 파일이 오면 치수를 다시 켜다. 그림은 아무 말 없이 바뀌어 온다.

사례 · 크로마키

무슨 일이 외부 배경 제거 도구가 알파만 만들고 RGB에는 마젠타(크로마키 색)를 남겼다. 색만 걸으면 과대평가된 알파 탓에 마젠타가 남았고(470 → 216), 수학적 하한을 쓰면 파란 투구가 84% 투명해졌다. **왜 조용했나** 배경 제거가 「이 픽셀이 얼마나 배경이었는지」라는 정보를 지워 버려, 뒤에서는 되살릴 근거가 없었다. **잡은 방법** 알파 구간별 평균 RGB를 재서 원인을 확인했다. 마젠타 배경 위의 합성 JPG를 받아 직접 풀자 잔여 픽셀이 약 30,000에서 약 100으로 줄었다.

사례 · 화면에서 우회하려다

무슨 일이 굽기 도구의 `im.convert("RGB")` 가 원본 알파를 지워 사진이 검은 배경으로 보였는데, 처음에는 화면에서 `mix-blend-mode` 로 우회하려 했다. 장비 제작 화면의 옥색·붉은 단추는 같은 79×40 상자인데 크기가 달라 보여, 옥색만 `border-image-outset` 으로 키웠더니 오히려 옥색이 커 보였다. 원인은 붉은 그림 구석의 알파 8자리 점 하나 때문에 도구가 경계상자를 다르게 자른 것(828×225 vs 846×244)이었다. 액자 그림은 아무 말 없이 927×672 에서 940×691 로 바뀌어 오기도 했다. **잡은 방법** 알파는 `getpixel()` 로 원본 픽셀을 찍어 확인했다. 단추는 기획자가 원본 두 장을 열어 「크기가 똑같고 색만 다르다」고 짚었다. 이후 검사는 배경에 흔들리는 픽셀 측정 대신 구조로 보장되는 값(`borderImageSlice`)을 보게 했다.

저장 형식은 런타임이 소비하는 방식과 대상 플랫폼으로 고른다

에셋 포맷은 요청 수, 상태 전환 비용, 엔진 내장 기능을 쓸 수 있는지를 표로 비교해 정한다. 움직이는 그림을 브라우저에 그대로 재생시키면 역재생·재시작·다른 연출 시계와의 동기화가 안 되므로, 칸 단위 띠(스프라이트)로 퍼서 코드의 시간표가 칸을 고르게 한다. 투명 애니메이션은 모든 대상 플랫폼에서 도는 형식으로 굽는다. 데스크톱 크롬에서 보인다고 판단하지 않는다.

방식 (캐릭터 260명 × 5자세)	요청 수	상태 전환
자세마다 파일 분리	1300	텍스처 교체
한 장 + 런타임 crop	260	crop 연산
가로 5칸 스프라이트 시트	260	<code>setFrame(n)</code> — UV만 바뀜

사례 · 캐릭터 애니메이션

무슨 일이 캐릭터 액션 5자세를 가로 5칸 스프라이트 시트 한 장(칸 110×110)으로 정했다. 웹 게임 엔진 Phaser의 `load.spritesheet` 가 로드할 때 한 번 잘라 주고, 엔진의 애니메이션 기능을 그대로 쓸 수 있다. 캐릭터마다 다른 몸통 높이는 굽는 단계에서 칸의 90%로 통일했고, 그림자는 코드로 그렸다.

사례 · 투명 애니메이션

무슨 일이 두루마리 펼침 연출의 원본은 `.gif` (알파가 1비트라 반투명이 탈락한다)·APNG(1.7MB) 대신 `webp` 를 골랐고, 그마저 그대로 재생하지 않고 띠로 폼다. 생성 AI로 만든 주사위 영상(1280×720 , 55프레임)은 투명하게 띄워야 했는데, 사파리가 알파 WebM을 재생하지 못한다(iOS가 출시 대상이다). 그래서 알파 WebP 스프라이트 시트 + 별도 음원으로 구웠다.

표시 조정과 정규화는 런타임이 아니라 굽는 단계에서 끝낸다

배경 세기를 낮추려고 화면에 반투명 막을 덮으면 그 위의 하이라이트와 캐릭터까지 뿌예진다. 표시 조정은 가능한 한 굽는 단계에서 끝내 런타임 레이어를 늘리지 않는다. 같은 자리에 들어갈 수 있는 에셋 계열들은 정규화 기준(채움 비율·기준선)을 하나로 정해 문서에 적는다. 반대로 짝으로 맞춰 그린 상태 그림(기본/선택)은 원본이 이미 정렬해 둔 것이므로 따로 트리밍(여백 자르기)하지 않고 예외 목록에 넣는다.

사례 · 배경 굵기

무슨 일이 반투명 막으로 판을 덮는 것이 가장 쉬웠지만 기각하고, 지도 세기는 굵는 단계에서 눌렀다. 시간대별(낮·황혼·밤) 배경이 한 장에 나란히 붙은 때 그림은 CSS `background-position` 으로 잘라 쓰지 않고 칸마다 잘라 JPG로 냈다(PNG 25MB 이상 → JPG 4.2MB). 한 장 잘라 쓰기는 칸 수가 원본마다 달라 같은 사실이 CSS와 코드 두 곳에 적히고, 밤 그림 하나를 띄우려 낮·황혼까지 받는다.

사례 · 정규화 기준 차이

무슨 일이 초상화는 정규화 없이(세로 채움 79~100%, 표준편차 3.6%), 액션 시트는 90%로 정규화해(91~99%, 표준편차 1.4%) 구웠다. 정지 그림을 움직이는 그림으로 바꾸자 「일부 캐릭터만 유독 작다」는 관찰이 나왔다. **잡은 방법** 알파 경계상자로 채움 비율을 실측해, 코드 결함이 아니라 원본 기준 차이임을 확인했다. 따로, 기본 칩과 선택 칩(같은 600×398 캔버스·같은 자리에 그린 그림)은 도구가 각각 여백을 잘라 화면에서 크기가 달랐다. 첫 진단(선택 칩의 빛 번짐이 잘렸으니 문턱값을 올린다)은 틀렸다.

파라미터를 고치면 전체를 다시 굵고, 건너뛰기는 존재가 아니라 내용으로 판단한다

한 폴더에 서로 다른 규칙으로 만든 파일이 섞이면 눈으로 구분하지 못한다. 파라미터 변경과 전체 재생성을 한 단위 작업으로 묶고, 부분 재생성은 확인용으로만 쓴다. 「출력이 있으면 건너뛴다」는 같은 이름으로 고친 원본을 못 보므로, 증분 판단은 원본의 해시(또는 수정 시각)로 하고, 출력이 여러 벌이면 전부 있어야 건너뛴다. 늘 `--force` 로만 돌리면 기본 경로의 버그가 숨으므로 기본 경로도 가끔 돌린다. 굵기 도구는 첫 입력 하나로 검증하지 말고, 작은 원본·큰 원본처럼 갈래가 다른 입력을 하나씩 넣어 본다. 「안 도는 갈래」는 새 그림이 와야 드러난다.

사례 · 부분 재생성

무슨 일이 초상화 배경 제거 규칙을 고친 뒤 7장만 다시 굵고 넘어갔다가 작업이 끊겨, 253장이 옛 규칙으로 남을 뻔했다. **잡은 방법** 다음 작업에서 작업 이력을 보고 알아채 `--force` 전체 재생성으로 맞췄다.

사례 · 안 도는 갈래

무슨 일이 스프라이트 굵기 함수는 칸을 128px로 잡는데, 그 안의 리사이즈 함수는 줄이기만 했다. 그래서 칸이 56px인 작은 문양 그림이 128px 칸의 왼쪽 위 구석에서 깜빡였다. 앞서 쓰던 그림은 원본이 커서 이 갈래를 한 번도 지나지 않았다. 초상화 도구는 출력의 존재만 봐서 같은 이름으로 바로잡은 원본을 반영하지 못했다. 액자 굵기 도구는 결과가 이미 있으면 죽는 버그(파일 정보 조회 한 줄 누락)가 있었는데, 늘 `--force` 로만 돌려서 아무도 못 봤다. **잡은 방법** 칸 크기를 입력 크기에서 정하게 고쳤다. 나머지는 원본을 고쳤는데 결과가 안 바뀐 것, 기본 옵션으로 한 번 돌려 본 것이 드러났다.

굵기 도구는 자기 이름만 지우고, 경고는 측정값으로 내고, 짝이 될 경고는 나란히 찍는다

같은 폴더를 여러 도구가 쓰면 「내 목록에 없으면 지운다」는 남의 산출물을 지운다. 각 도구는 자기가 만들 수 있는 이름 패턴만 지운다. 품질 경고는 파일 이름이 아니라 측정값(실제 폭)으로 낸다. 오탐이 섞이면 진짜 경고까지 무시된다. 「원본 없음」과 「매칭 실패」처럼 짝이 될 수 있는 경고는 한 화면에 나란히 출력하고, 에셋 이름의 표준은 하나(파일명)로 정한다. 폴백과 삭제는 말없이 하지 말고 이름을 찍어 알린다

→ 3장 시스템 엔지니어.

사례 · 남의 파일을 지운 도구

무슨 일이 배경 굵기 명령이 공용 출력 폴더에서 UI 굵기 도구가 만든 배경 한 장을 「계획에 없으니 묵은 것」으로 쓸어 버려 404가 났다. 같은 배경 도구는 「저해상도」를 파일 이름의 접미사로 판단해서, 해상도가 같은 다른 그림을 저해상도로 찍었다. **왜 조용했나** 배경 그림이 404면 화면이 바탕색으로 물러나 눈에 잘 띄지 않는다. 거짓 경고는 경고 전체를 안 읽게 만든다.

사례 · 짝이었던 두 경고

무슨 일이 액션 시트 도구가 몇 주째 「원본이 없는 캐릭터 8명」과 「대응 캐릭터가 없는 파일 8건」을 따로 찍었다. 나란히 놓자 한 글자씩 틀린 이름 짝이었다. 그동안 그 8명은 전투에서 초상화로 대신 썼다. **왜 조용했나** 두 경고가 각자 「원본이 아직 안 왔다」로 읽혔다. **잡은 방법** 두 목록을 나란히 놓았다. 수가 같은 두 경고는 서로의 짝인지부터 맞춰 본다.

6.5 자르기와 배경 제거 — 이미지 처리를 재는 법

대량 에셋을 다루면 이미지 처리 알고리즘을 직접 만들게 된다. 우리는 캐릭터 초상화 260장의 배경 제거, 그림 시트 자르기, 생성 영상의 배경 걷기를 직접 했다. 이 일은 예제 몇 장에서 좋은 점수가 나와도 전수에서 무너진다. 원칙은 「전수를 돌려 분포의 끝을 보고, 지표를 여럿 세우고, 정답조차 의심한다」로 모인다.

그림 시트는 등분하지 말고 픽셀로 경계를 찾고, 찾은 수를 단언한다

「넉 장을 넉 등분」처럼 당연해 보이는 분할도 실제 경계는 1~3px 어긋나 있다. 등분하면 옆 칸이 딸려 오고, 화면에는 「끝에 이상한 줄」로만 보인다. 등분 자리 근처에서 색이 가장 튀는 열이나 알파 합이 가장 작은 열을 경계로 찾고, 옮긴 값을 로그로 남기며, 찾은 경계 수가 기대값과 다르면 도구가 멈추게 한다. 칸의 순서도 관례를 믿지 말고 원본마다 열어서 확인한다.

사례 · 시트 자르기

무슨 일이 시간대별 배경 띠 그림의 진짜 칸 경계가 268·536이 아니라 266·535여서, 낮 그림 오른쪽에 황혼의 주황 띠가 2px 남았다. 다른 그림은 정확히 등분 자리라 「다 그렇겠지」로 넘어갈 뻔했다. 한 달 뒤 등급 배지 시트(1003px ÷ 6)는 알파 0인 틈조차 없었고, 상점 아이템 시트(681 ÷ 4 = 170.25)는 실제 경계와 최대 3px 어긋났다. 같은 자리를 두 번 밟았다. **왜 조용했나** 도구는 몇 장을 만들었지만 알려 주지, 잘 잘랐는지는 알려 주지 않는다. **잡은 방법** 경계를 찾은 뒤 [268, 536] → [266, 535]처럼 보정량을 찍고, 찾은 경계 수가 기대값이 아니면 멈추게 했다.

사례 · 칸 순서

무슨 일이 기존 관례(시계방향)대로 2×2 연출 그림을 퍼면 꽃이 피는 장면과 지는 장면이 뒤집혀 재생될 뻔했다. 실제로는 읽는 순서(좌상→우상→좌하→우하)로 그려져 있었다. **잡은 방법** 그림 파일을 직접 열어 봤다.

표본은 대표값 하나로 접지 말고, 임계값은 정상값의 최댓값을 재고 정한다

여러 성분이 섞일 수 있는 표본을 중앙값 하나로 접으면, 혼합 표본에서는 어느 쪽도 아닌 값이 나온다. 분포(히스토그램)를 그대로 들고 판정한다. 임계값은 근거 없이 고르지 말고, 정상 사례가 어디까지 올라가는지와 이상

사례가 어디서 시작하는지를 재서 그 사이에 둔다. 연결성(플러드필, 같은 색 영역을 이웃 따라 채워 나가는 방식)만으로 안 잡히는 갇힌 영역에는 색 기준 같은 두 번째 판정을 둔다.

사례 · 배경 제거

무슨 일이 초상화 배경 제거의 첫 판은 그림 테두리 색의 중앙값 하나를 배경색으로 썼다. 예제에서는 IoU(정답 마스크와 겹치는 비율) 0.99였는데, 260장 중 46장이 배경을 절반도 못 지웠다(최악은 95.4% 잔존). **왜 조용했나** 두 색 띠가 섞인 배경이 많았고, 혼합 표본의 중앙값은 어느 쪽도 아닌 색이라 두 색이 다 남았다. 예제 몇 장에는 그런 그림이 없었다. **잡은 방법** 전수 결과의 「남은 넓이」 분포를 보고, 이상한 그림의 원본을 눈으로 확인했다. 히스토그램 방식으로 바꾸자 46장이 0장이 됐다.

사례 · 근거 없는 임계값

무슨 일이 단체 그림에서 「배경 띠」를 판정하는 채움 문턱 0.5를 근거 없이 골랐다. 그런데 발치에서는 캐릭터 5명만으로 폭의 65%가 차서 5명이 한 덩어리로 붙었다. 위아래 띠가 배경을 가둬 플러드필이 닿지 못해 그림 하나가 통째로 지워지기도 했다. **왜 조용했나** 대부분의 입력에서는 0.5로도 정상 결과가 나왔다. **잡은 방법** 실패한 파일을 하나씩 보고 정상 신호(진짜 띠는 폭의 100%)를 다시 켜다.

한 축에서 상충이 풀리지 않으면 다른 축을 찾고, 틀림의 무늬를 본다

상충하는 목표는 각각 지표로 세워 나란히 놓고 훑는다. 같은 축(예: 색 허용치)에서 「느슨하면 A가, 조이면 B가 깨진다」가 보이면 그 축을 더 잘게 훑지 말고, 두 문제를 구별하는 다른 성질을 찾는다. 더 정교한 모델을 쫓다 막히면 「틀림이 어떤 주기·무늬로 나타나는가」를 보고 표적 필터를 생각한다. 알파와 색을 다른 정밀도로 처리하지 않고, 한 지표·한 배율만 보고 끝내지 않는다.

사례 · 축 추가

무슨 일이 히스토그램 방식에서 8장이 얼굴을 잃었다. 외곽선의 틈으로 플러드필이 얼굴 안까지 들어갔다. 색 허용치를 4×4로 훑었지만 느슨하면 얼굴이, 조이면 배경이 남았다. **잡은 방법** 예제 IoU·최소 넓이(얼굴 보존)·최대 넓이(배경 제거) 세 지표를 나란히 놓아 색 축 안의 상충을 확인했다. 누수는 좁은 통로를 지난다는 점에 착안해 「폭」 축(반지름 3 열림 연산)을 더했다. 최소 넓이가 0.178에서 0.319로 올라 양쪽을 만족했다.

사례 · 무늬를 본 필터

무슨 일이 투명 배경을 뜻하는 체커보드 무늬가 아예 그림에 박혀 온 가차 연출 그림의 배경 제거가 세 번 실패했다. 타일 평균은 1px짜리 광선에서 깨졌다. 위치마다 다른 분모(255 - 배경)를 쓴 방식은 가정이 깨지는 픽셀마다 체커를 물려받았다. 알파만 블러하자 색 복원식의 배경 향이 색에 새겨졌다. **왜 조용했나** 구석의 알파만 보면 「배경은 깨끗」이었다. 무늬는 색 채널에 있었다. **잡은 방법** 구석 알파·색 채널 픽셀 배열·실사용 크기 렌더 세 축을 따로 보다가, 틀림이 정확히 30px 주기로 난다는 것을 알아챘다. 그 주파수 성분만 복조·저역통과로 지웠다.

자기 출력을 다시 넣는 단계와 정답 표본은 둘 다 의심하고 쟀다

1차 결과로 모델을 다시 추정하는 반복 정련은 1차의 오류를 다음 단계에 반영해 눈덩이로 키울 수 있다. 「더 정교해 보이는」 단계도 정답 대비 수치가 좋아질 때만 남긴다. 반대로 정답 표본도 완벽하다고 가정하지 않는다. 정답 대비 점수는 참고치로 쓰고, 점수가 낮은 건은 정답 쪽 결함부터 확인한다. 정답을 검수하지 않은 최적화는 정답의 결함까지 학습한다.

사례 · 정련 패스와 정답 표본

무슨 일이 1차 결과로 색 모델을 다시 잡아 2차 플러드필을 도는 정련 패스가 예제 IoU를 0.99에서 0.91로 떨어뜨리고 발밀 풀발을 통째로 지웠다. 1차에 섞여 들어간 초록 기운이 모델에 반영됐다. 측정한 뒤 삭제했다. 같은 날, 점수가 낮던 예제 2건(IoU 0.907, 0.832)은 오히려 우리 결과가 더 깨끗했다. 예제 쪽에 등고선 조각·넙쿨 잔상이 남아 있었고, 서로 다른 도구로 만든 것으로 보였다. **왜 조용했나** IoU를 목표로 최적화했다면 예제의 잔상까지 따라 만들어 점수가 「올라갔을」 것이다.

자동 처리는 다수에 맞추고, 예외는 표시와 수동 통로로 처리한다

260개 중 1개를 위한 휴리스틱은 259개를 망칠 수 있다. 자동 처리는 다수에 맞추고, 정상 범위를 벗어난 결과를 「확인 필요」로 표시하는 검사와 수동 지정 통로로 예외를 처리한다. 수동 값은 원본이 바뀌면 틀어질 수 있음을 적어 둔다. 입력이 같은 생성기·같은 구도로 균질하면, 그 균질성을 쓰는 단순한 규칙이 범용 학습 모델보다 정확하고 의존성도 적다. 다만 그 파라미터는 그 입력 전용이라는 것을 적어 둔다.

사례 · 한 장의 예외

무슨 일이 초상화 한 장은 배경이 인물에 갇혀 있어 색 통계로 피부와 분리되지 않았다(배경의 밝기 편차 $\pm 0 \sim +7$, 피부 $+1 \sim +38$ 로 겹친다). 자동 판별 규칙을 더하는 대신, 배경 시작점 좌표를 손으로 넣는 옵션을 뒀다. 그리고 결과의 배경 비율이 정상 범위(8~55%)를 벗어나면 도구가 표시하게 했다. 배경 제거에 rembg 같은 학습 모델은 쓰지 않았다. 260장이 같은 생성기·같은 구도라 배경 성질을 직접 쓰는 쪽이 더 정확했고, 약 176MB짜리 모델 의존도 안 생겼다.

다른 바탕에 올릴 소재는 가장자리와 그림자를 다시 만든다

배경을 걷은 소재는 원래 바탕을 전제로 그려진 흔적을 안고 있다. 물체가 이미지 가장자리에 걸리면 모서리와 물체 사이의 배경이 바깥과 끊겨 「물체 안의 구멍」으로 보이고, 구멍 채우기가 그것을 정상적으로 채워 버린다. 구멍 채우기 전에 이미지 둘레에 배경색 테를 덧대 바깥을 하나로 이은 뒤 범람시킨다. 원본 그림자가 배경보다 밝으면 걷은 뒤 밝은 얼룩이 되므로, 검정 반투명으로 다시 만들어 어느 바탕에서도 바닥으로 읽히게 한다. 경계는 침식 대신 매팅(경계 픽셀의 알파와 색을 함께 추정하는 처리)으로 복원한다.

사례 · 생성 영상의 배경 걷기

무슨 일이 생성 AI로 만든 주사위 영상의 배경을 밝기 문턱 → 닫기 → 구멍 채우기로 걷었다. 그런데 첫 프레임에서 주사위가 화면 가장자리에 걸려, 모서리와 주사위 사이에 갇힌 배경이 검은 조각으로 붙었다. 원본 그림자는 짙은 회색 배경보다 밝은 회색이라, 걷고 나니 밝은 얼룩으로 남았다. **왜 조용했나** 바깥과 이어지지 않은 배경은 알고리즘에게 「물체 안의 구멍」으로 보여 정상적으로 채워졌다. **잡은 방법** 양피지·회색·먹색 세 바탕 위에 결과를 겹쳐 보는 미리보기 옵션으로 둘 다 찾았다. 화면 표식(워터마크)은 비우고, 물체가 들어오는 가장자리는 페이드로 스며 나오게 했다.

6.6 눈으로 확인하는 법

이 자리의 결함 상당수는 수치로 안 잡히고 눈으로만 보인다. 그런데 눈도 잘못된 조건에서 보면 속는다. 흰 미리보기, 작은 프레임, 기본 줌, 로딩이 덜 된 스크린샷이 모두 결함을 숨기거나 없는 결함을 만든다. 볼 때는 조건을 극단으로 잡고, 판정은 다시 숫자로 한다.

투명 이미지는 모든 바탕 위에서, 크게, 극단 표본으로 보고 판단은 픽셀 통계로 한다

투명 이미지는 실제로 쓰일 모든 배경색 위에 합성하고, 실제 표시 배율보다 크게 본다. 전수 확인이 어려우면 지표의 양 극단 + 무작위 표본을 한 장의 대조 시트로 본다. 수치는 정상이어도 그림이 깨질 수 있고, 반대로 눈에는 깨져 보여도 미리보기 탓일 수 있다. 「안 보인다」 「노이즈가 있다」를 추출 실패로 단정하기 전에 미리보기 배경과 압축을 의심하고, 알파 히스토그램으로 판정한다.

사례 · 어두운 바탕에서만 보인 데

무슨 일이 초상화 이진 마스크가 흰 배경에서는 멀쩡했는데, 어두운 UI 위에서 실루엣 둘레에 황갈색 테 1px이 보였다(원본의 안티에일리어싱 혼합 픽셀). 기본 줌에서는 판의 칸이 30px대라 안 보였다. **왜 조용했나** 넓이·IoU 같은 수치는 정상이었다. **잡은 방법** 실제로 쓰일 판 색 셋(중립·아군 녹색·적군 적색) 위에 3배 확대 합성하고, 지표 양 극단 14장 + 무작위 86장을 어두운 배경 대조 시트로 봤다. 매팅 2px로 해결했다. 얼굴을 잃은 8장도 수치보다 대조 시트에서 먼저 보였다.

사례 · 미리보기의 착시

무슨 일이 불꽃 고리 연출 6장 중 흰색·은색 두 장이 흰 배경 미리보기에서 안 보였다. 가차 그림은 미리보기에서 배경 노이즈가 보였다. **잡은 방법** 둘 다 알파 0~255가 정상 분포임을 픽셀 통계로 확인했다. 앞은 미리보기 배경의 착시, 뒤는 미리보기의 압축 아티팩트였다.

딱 채우는 그림은 가장 큰 프레임에서, 에셋이 다 온 뒤에 본다

작은 프레임에서는 늘린 저해상도 그림도 깨져 보이지 않는다. 해상도 품질 검사는 최대 표시 크기에서 한다(우리 경우 700px). 시각 확인용 캡처는 에셋 로드가 끝난 것을 확인한 뒤에 찍는다. 스크린샷이 그림 다운로드보다 빠르면 「판때기가 안 그려졌다」를 우선순위 문제로 오진한다.

사례 · 저해상도 배경

무슨 일이 처음 받은 화면 배경은 칸 하나가 225~300px짜리 저해상도였는데, 480px 스크린샷으로만 확인해 놓쳤다. **잡은 방법** 기획자가 화면을 보자마자 지적했다. 이후 700px 눈검사를 기준으로 삼았다.

「안 보인다」 「폰트가 다르다」는 굵기·색보다 좌표계를 먼저 본다

확대·축소가 있는 화면에서 월드 좌표 1px 선은 배율이 작아지면 1px 미만이 되어 사라진다. 굵기·색을 올리기 전에 그 선이 화면 좌표 기준인지 월드 좌표 기준인지 정하고, 항상 보여야 하는 선·글자는 배율 역수로 두께를 보정한다. 바탕 밝기 범위 전체에서 대비도 확인한다. SVG 안의 글자 크기가 화면마다 다르다면 `viewBox` 와 원본 해상도부터 본다. 포커스 가능한 SVG 요소에는 초점 스타일을 명시한다.

사례 · 사라진 줄눈

무슨 일이 판의 칸 줄눈을 굵고 진하게 해도 전체 보기 배율에서 안 보였다. 월드 좌표 1px은 0.19배에서 0.2px가 된다. 밝은 바탕에서는 검은 외곽선이 뒤집혀 글자는 날아가고 테만 남았다. **잡은 방법** 이미 해결해 둔 좌표 눈금 글자의 방식(배율 역수)과 비교했다.

사례 · 다른 폰트처럼 보인 글자

무슨 일이 레벨이 오른 도시 그림의 원본이 482픽이라, 그 위의 SVG `<text>`가 `viewBox` 단위로 1.42배 커지면서 「다른 폰트」처럼 보였다. 클릭한 뒤 남던 검은 둥근 네모는 `tabIndex`를 단 `<g>`의 기본 초점 테두리였다. **잡은 방법** 그림마다 자기 `viewBox`를 갖게 하고, 원본 해상도를 메인 화면 그림과 맞췄다.

장식 효과는 결정적으로 만들고, 불변 성질은 좌표로 검사한다

장식 애니메이션을 난수로 만들면 경계를 넘는 경우가 어쩌다 한 번 나와 재현이 안 된다. 정해진 순환으로 만들어 (결정적, 같은 입력이면 늘 같은 결과) 모든 경우를 테스트로 훑을 수 있게 한다. 「글자가 안 움직인다」 같은 픽셀 단위의 불변 성질은 스크린샷 두 장을 눈으로 비교해서는 알 수 없으므로 좌표로 검사한다. DOM 구조를 바꾸면 그 구조를 읽던 스모크 테스트도 함께 고친다. 「있는가」「제자리에 있는가」「몇 줄인가」를 따로 채는 일반 원칙은 → 5장 클라이언트 엔지니어를 본다.

사례 · 움직이는 배경

무슨 일이 메뉴 화면 배경의 미세한 움직임이 2.6초 간격·최대 1.5%·여덟 자세 순환으로 만들었다. 최대 10px 이동이라 글자가 1~2px 따라 움직여도 눈으로는 모른다. 그림을 별도 층으로 옮기자, 배경 그림을 제목 요소에서 읽던 스모크 테스트가 첫 항목에서 막혔다. **잡은 방법** 「그림의 `transform`은 달라지고 제목의 `getBoundingClientRect()`는 같다」를 한 검사로 묶었다. 확대로 만든 여백이 이동 거리보다 큰지는 계산(회귀 테스트)과 실제 배치(브라우저에서 21초를 0.4초마다 훑어 최악의 틈 -12.043px)로 따로 확인했다.

목업은 결함을 찾는 데 쓰고, 느낌은 실제 화면에서 판단한다

인증 같은 외부 의존 때문에 화면에 못 들어가면 「스타일만 얹은 정적 목업」과 「진짜 컴포넌트를 가짜 데이터로 마운트한 임시 엔트리」를 쓰고, 검증 뒤에는 지운다. 목업은 잘림·깜빡임 같은 결함 판단의 근거로만 쓴다. 목업에는 실제 화면의 조상 요소가 없어 구조에 의존하는 버그가 재현되지 않고, 밀도·답답함 같은 감각은 실제 크기 화면에서만 판단할 수 있다. 고치기 전에는 요청받은 화면을 스크린샷으로 확인해, 실제로 렌더하는 컴포넌트를 특정한다.

사례 · 로그인이 막힌 화면

무슨 일이 외부 인증 서비스(Supabase)의 이메일 확인 설정 때문에 자동화 세션이 시험 계정을 만들 수 없었다. 대신 스타일시트를 정적 파일 폴더에 잠깐 복사하고(그래야 그 안의 상대 경로 그림이 풀린다) 마크업만 흉내 낸 HTML을 Playwright로 찍었다. 이것으로 효과 글 잘림·음절 중간에서 끊긴 한글·스프라이트의 구석 깜빡임·그림 사라짐 넷을 잡았다. 반면 340px 목업에서 답답해 보여 줄을 두 줄로 눌렀다가, 기획자가 실제 화면에서 「안 답답하다」고 해서 되돌렸다. **잡은 방법** 결함은 목업으로 잡고, 느낌은 실제 화면 스크린샷으로 기획자에게 확인받았다.

사례 · 영동한 컴포넌트

무슨 일이 「캐릭터 카드를 고쳐 달라」는 요청에 이름이 비슷한 상세 화면 컴포넌트를 고쳤는데, 실제로 뜨는 것은 카드 팝업 컴포넌트였다. **잡은 방법** 고친 뒤 스크린샷을 보니 바뀐 것이 없었다. 화면 이름으로 파일을 짐작하지 않는다.

작업 전후 점검표

- ☐ 새로 쓴 클래스 이름에 화면 접두사가 붙어 있는가? 접두사 뒤가 `head · row` 같은 흔한 낱말은 아닌가?
- ☐ 덮어쓴 규칙이 실제로 이겼는지 `getComputedStyle` 로 확인했는가? `:not()` 안의 클래스까지 썼는가?
- ☐ 그림 테두리를 입힌 요소에 `border:` 단축 속성을 쓰는 규칙이 같은 세기로 겹치지 않는가?
- ☐ 리스킨 뒤 「선택됨」 「눌림」 상태가 여전히 보이는가?
- ☐ 잘리면 안 되는 줄과 저장 단추가, 영역을 극단적으로 좁히거나 내용이 넘치는 데이터에서도 남는가?
- ☐ 인원·모드·언어 등 구성마다 화면을 따로 띄워 봤는가? 가장 긴 번역으로도 돌렸는가?
- ☐ 팝업이 「떠 있는가」 뿐 아니라 프레임을 제자리에서 덮는지 좌표로 확인했는가?
- ☐ 빈 자리 요소와 띄운 판이 아래 클릭을 가로채지 않는가? 실제 클릭·호버로 확인했는가?
- ☐ 크기 요구를 실측한 상자에 곱했는가? 고치기 전과 후의 치수가 실제로 달라졌는가?
- ☐ 값을 줄이거나 넣은 이유가 이번 변경으로 사라지지 않았는가? 사라졌다면 값과 주석을 함께 지웠는가?
- ☐ 원본 → 중간 → 결과물을 픽셀로 확인했는가? CSS로 우회하고 있지는 않은가?
- ☐ 굵기 파라미터를 고친 뒤 전체를 다시 구웠는가? 기본 경로(`--force` 없이)로도 돌려 봤는가?
- ☐ 시트 자르기가 찾은 경계 수를 단언하는가? 칸 순서를 원본에서 직접 확인했는가?
- ☐ 이미지 처리 결과를 예제 몇 장이 아니라 전수 분포의 양 끝으로 확인했는가?
- ☐ 투명 결과를 밝은·어두운·실사용 바탕 위에서 실제보다 크게 봤는가? 「안 보인다」를 알파 히스토그램으로 판정했는가?

조용히 틀리는 것들 — 게임 개발 현장 백서

제2판 · 2026년 10월

작은 팀이 9주 동안 웹·모바일 턴제 전술 게임을 만들며 남긴 개발 기록에서 교훈 353개를 뽑아, 다섯 직무의 원칙 180개로 다시 묶었다.